



แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอน
และงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ

นายฐิติพงษ์ ตะเพียนทอง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย ภาควิชาเทคโนโลยีสารสนเทศ
คณะเทคโนโลยีและการจัดการอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ

นายฐิติพงษ์ ตะเพียนทอง

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย ภาควิชาเทคโนโลยีสารสนเทศ
คณะเทคโนโลยีและการจัดการอุตสาหกรรม มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2568
ลิขสิทธิ์ของมหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

CLUSTER-BASED PLATFORM FOR SUPPORTING TEACHING AND ACADEMIC ACTIVITIES
IN THE DEPARTMENT OF INFORMATION TECHNOLOGY

MR.THITIPONG TAPIANTHONG

PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE BACHELOR'S DEGREE OF ENGINEERING
PROGRAM IN INFORMATION AND NETWORK ENGINEERING
DEPARTMENT OF INFORMATION TECHNOLOGY
FACULTY OF INDUSTRIAL TECHNOLOGY AND MANAGEMENT
KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK
ACADEMIC YEAR 2025
COPYRIGHT OF KING MONGKUT'S UNIVERSITY OF TECHNOLOGY NORTH BANGKOK



ใบรับรองปริญญาโท
คณะเทคโนโลยีและการจัดการอุตสาหกรรม
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เรื่อง แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการ
ภาควิชาเทคโนโลยีสารสนเทศ
โดย นายฐิติพงษ์ ตะเพียนทอง

ได้รับอนุมัติให้นับเป็นส่วนหนึ่งของการศึกษาตาม
หลักสูตรวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมสารสนเทศและเครือข่าย

_____ คณบดี
(ผู้ช่วยศาสตราจารย์ ดร.กฤษฎากร บุคตาจันทร์)

คณะกรรมการสอบปริญญาโท

_____ ประธานกรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ศรายุทธ ธเนศสกุลวัฒนา)

_____ กรรมการ
(อาจารย์ ดร.วัชรชัย คงศิริวัฒนา)

_____ กรรมการ
(ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ)

ชื่อ : นายฐิติพงษ์ ตะเพียนทอง
ชื่อปริญญาบัตร : แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงาน
วิชาการภาควิชาเทคโนโลยีสารสนเทศ
สาขาวิชา : วิศวกรรมสารสนเทศและเครือข่าย
: มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
อาจารย์ที่ปรึกษาปริญญาบัตร : ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ
ปีการศึกษา : 2568

บทคัดย่อ

ปริญญาบัตร "แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาควิชาเทคโนโลยีสารสนเทศ" ฉบับนี้ มีวัตถุประสงค์เพื่อพัฒนาแพลตฟอร์มคลัสเตอร์เสมือนที่สามารถสร้างและจัดการเครื่องเสมือนลินุกซ์ได้อย่างรวดเร็วและมีประสิทธิภาพ เพื่อสนับสนุนการเรียนการสอนและการทำโครงงานในภาควิชาเทคโนโลยีสารสนเทศ และพัฒนาเว็บแอปพลิเคชันแบบ Role-Based Access Control (RBAC) ที่รองรับการใช้งานของทั้งนักศึกษาและอาจารย์ โดยคำนึงถึงความง่ายในการใช้งาน เสถียรภาพของระบบ และความปลอดภัยในการเข้าถึง ระบบประกอบด้วยสามส่วนหลัก ได้แก่ ส่วนติดต่อผู้ใช้งาน บริการ API สำหรับจัดการการยืนยันตัวตน การจัดการผู้ใช้ และเชื่อมต่อระหว่างส่วนติดต่อผู้ใช้และบริการจัดการเครื่องเสมือน และบริการจัดการเครื่องเสมือนที่เชื่อมต่อกับระบบ Hypervisor เพื่อสร้าง กำหนดค่า และจัดการเครื่องเสมือน ระบบสามารถลดความซับซ้อนในการตั้งค่าสภาพแวดล้อมสำหรับการใช้งานระบบลินุกซ์ เพิ่มประสิทธิภาพการใช้ทรัพยากรเซิร์ฟเวอร์ และสนับสนุนการเรียนการสอนที่เกี่ยวข้องกับระบบเซิร์ฟเวอร์ การพัฒนาแอปพลิเคชัน และการจัดการโครงสร้างพื้นฐานด้านไอทีได้อย่างมีประสิทธิภาพ

(ปริญญาบัตรมีจำนวนทั้งสิ้น 81 หน้า)

Name : Mr.Thitipong Tapianthong
Project Title : Cluster-Based Platform for Supporting Teaching and Academic Activities in the Department of Information Technology
Major Field : Information and Network Engineering
: King Mongkut's University of Technology North Bangkok
Project Advisor : Asst. Prof. Dr.Khanista Namee
Academic Year : 2025

Abstract

This project, "Cluster-Based Platform for Supporting Teaching and Academic Activities in the Department of Information Technology," aims to develop a virtual cluster platform capable of rapidly and efficiently creating and managing Linux virtual machines to support teaching and project work in the Department of Information Technology. The project also focuses on developing a Role-Based Access Control (RBAC) web application that supports both students and faculty, emphasizing ease of use, system stability, and access security. The system consists of three main components: a user interface, an API service for handling authentication and user management and communication between the frontend and VM operations service, and a VM management service that interfaces with the Hypervisor system to create, configure, and manage virtual machines. The system reduces the complexity of setting up Linux environments, improves server resource utilization efficiency, and effectively supports teaching related to server systems, application development, and IT infrastructure management.

(Total 81 pages)

Project Advisor

กิตติกรรมประกาศ

ปริญญานิพนธ์ ”แพลตฟอร์มคลัสเตอร์เพื่อสนับสนุนการเรียนการสอนและงานวิชาการภาค
วิชาเทคโนโลยีสารสนเทศ” จะไม่สามารถสำเร็จลุล่วงไปได้ด้วยดีหากไม่ได้รับการช่วยเหลือและความ
อนุเคราะห์จากบุคคลหลายท่าน ทางผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.ชนิษฐา นามิ ซึ่งเป็น
อาจารย์ที่ปรึกษา ที่ได้กรุณาให้คำปรึกษาแนะนำ และให้แนวทางในการพัฒนาแพลตฟอร์มคลัสเตอร์และ
การจัดทำปริญญานิพนธ์ รวมทั้งคณาจารย์ภาควิชาเทคโนโลยีสารสนเทศที่ได้ให้ความรู้ด้านเทคโนโลยี
และการพัฒนาระบบให้กับผู้จัดทำเพื่อนำมาประยุกต์ในการพัฒนาโครงการนี้ อีกทั้งขอขอบคุณภาควิชา
เทคโนโลยีสารสนเทศที่ให้การสนับสนุนโครงสร้างพื้นฐานและทรัพยากรที่จำเป็นสำหรับการพัฒนาและ
ทดสอบระบบ

ท้ายที่สุดนี้ ทางผู้จัดทำขอน้อมรำลึกพระคุณบิดา มารดา ผู้ซึ่งมีพระคุณอย่างสูงสุดที่ให้ความ
อุปการะผู้จัดทำมาโดยตลอด รวมทั้งผู้มีพระคุณทุกท่าน คณะผู้จัดทำรู้สึกซาบซึ้งเป็นอย่างยิ่ง จึงใคร่ขอ
ขอบพระคุณเป็นอย่างสูงมา ณ โอกาสนี้ด้วย

ฐิติพงษ์ ตะเพียนทอง

สารบัญ

หน้า

บทคัดย่อภาษาไทย	ข
บทคัดย่อภาษาอังกฤษ	ค
กิตติกรรมประกาศ	ง
สารบัญ	จ
สารบัญภาพ	ช
สารบัญตาราง	ฌ
บทที่ 1 บทนำ	1
1.1 ความเป็นมาและความสำคัญ	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของการทำโครงการ	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 แนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้อง	5
2.1 แนวคิดพื้นฐานเกี่ยวกับโครงการ	5
2.2 แนวคิดและทฤษฎีที่เกี่ยวข้องกับแพลตฟอร์มคลัสเตอร์เสมือน	7
2.3 รายงานการค้นคว้า การศึกษา หรืองานวิจัยที่เกี่ยวข้อง	10
บทที่ 3 วิธีการดำเนินงานและการออกแบบระบบ	13
3.1 การออกแบบสถาปัตยกรรมระบบ	13
3.2 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา	40
บทที่ 4 ผลการดำเนินงาน	43
4.1 โครงสร้างโครงการหลัก	43
4.2 โครงสร้างแอปพลิเคชันและบริการ	46
4.3 สถาปัตยกรรมระบบและเทคโนโลยี	47
4.4 เอกสารประกอบการใช้งาน API และส่วนติดต่อผู้ใช้	48
4.5 ตัวอย่างการใช้งานระบบตามบทบาทผู้ใช้	50
4.6 ผลการทดสอบประสิทธิภาพระบบ	55
บทที่ 5 สรุปผลการดำเนินงาน	64
5.1 สรุปผลการดำเนินงาน	64
5.2 ปัญหาและอุปสรรค	65
5.3 การแก้ปัญหา	66
5.4 ข้อเสนอแนะ	67
บรรณานุกรม	68

สารบัญ (ต่อ)

	หน้า
ภาคผนวก	70
ภาคผนวก ก: การเตรียม Workspace สำหรับโครงการแบบหลายรีโพสิทอรี	71
ก.1 ภาพรวมของโครงสร้างแบบหลายรีโพสิทอรี	72
ก.2 ข้อกำหนดของระบบ	72
ก.3 การโคลนรีโพสิทอรีทั้งหมด	72
ก.4 การติดตั้ง dependencies ของแต่ละรีโพสิทอรี	73
ก.5 การเตรียมไฟล์ Environment	73
ก.6 ข้อสังเกตสำหรับการพัฒนาใน workspace เดียวกัน	73
ภาคผนวก ข: การเริ่มต้นระบบแบบ Development ในสภาพแวดล้อมหลายรีโพสิทอรี	74
ข.1 การเริ่มต้นบริการประกอบจากรีโพสิทอรี Yuuka	75
ข.2 การเริ่มต้นบริการหลักของระบบ	75
ข.3 ลำดับการเริ่มต้นที่เหมาะสม	75
ข.4 จุดเชื่อมต่อสำคัญในสภาพแวดล้อมพัฒนา	76
ข.5 การอัปเดตชนิดข้อมูลระหว่าง frontend และ backend	76
ข.6 การแก้ไขปัญหาเบื้องต้นในสภาพแวดล้อมพัฒนา	76
ภาคผนวก ค: หน้าที่ของแต่ละรีโพสิทอรีและแนวทางการตรวจสอบปัญหาเบื้องต้น	78
ค.1 หน้าที่ของแต่ละรีโพสิทอรีในระบบ	79
ค.2 ส่วนประกอบสนับสนุนที่เกี่ยวข้องกับหลายรีโพสิทอรี	79
ค.3 แนวทางการตรวจสอบปัญหาเบื้องต้น	79
ค.4 ตัวอย่างรายการตรวจสอบอย่างเป็นลำดับ	80
ค.5 ข้อควรระวังเรื่องความสอดคล้องระหว่างรีโพสิทอรี	80
ค.6 เอกสารอ้างอิงที่ควรใช้ประกอบ	81

สารบัญภาพ

ภาพที่	หน้า
3-1 แผนภาพการทำงานและลำดับขั้นตอนโดยรวมของโปรแกรม	14
3-2 แผนภาพบริบทของระบบแพลตฟอร์มคลัสเตอร์เสมือน	15
3-3 แผนภาพการไหลของข้อมูลระดับ 0: ภาพรวมทั้งระบบ	16
3-4 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 1.0 การยืนยันตัวตนและกำหนดสิทธิ์ผู้ใช้	17
3-5 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 2.0 การจัดการคำร้องและข้อมูลวิชาการ	18
3-6 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 3.0 การจัดการข้อมูลเครื่องเสมือน	19
3-7 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 4.0 การจัดการไฟล์และสิทธิ์การเข้าถึง	20
3-8 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 5.0 การประมวลผลงานจัดสรรเครื่องเสมือน	21
3-9 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 2.3 การพิจารณาอนุมัติคำร้อง	22
3-10 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 3.1 การรับคำร้องที่อนุมัติแล้วและจัดเตรียมงาน	23
3-11 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 3.3 การส่งงานเข้าคิว	24
3-12 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 5.3 การดำเนินการบน Proxmox VE	25
3-13 หน้าจอเข้าสู่ระบบ (Login Page)	26
3-14 หน้าจอแดชบอร์ด (Dashboard)	27
3-15 หน้าจอจัดการเครื่องเสมือน (Instance Management)	27
3-16 หน้าจอคำร้องขอเครื่องเสมือน (Instance Request)	28
3-17 หน้าจอจัดการคำร้องขอ (Request Management)	28
3-18 แผนภาพโครงสร้างฐานข้อมูลของระบบภาพรวม (ER Diagram Overview)	30
3-19 แผนภาพ ER กลุ่มข้อมูลการยืนยันตัวตนและผู้ใช้งาน	31
3-20 แผนภาพ ER กลุ่มข้อมูลโครงสร้างวิชาการ	32
3-21 แผนภาพ ER กลุ่มข้อมูลโครงสร้างพื้นฐาน Proxmox VE	33
3-22 แผนภาพ ER กลุ่มข้อมูลการจัดการเครื่องเสมือน	34

สารบัญภาพ (ต่อ)

ภาพที่	หน้า
3-23 แผนภาพ ER กลุ่มข้อมูลคำร้องขอและกระบวนการอนุมัติ	35
3-24 แผนภาพ ER กลุ่มข้อมูลพื้นที่จัดเก็บไฟล์	36
4-1 หน้าเอกสาร API Documentation	48
4-2 หน้าแรกของระบบ (Landing Page)	49
4-3 Dashboard หลักสำหรับนักศึกษา	51
4-4 หน้าจัดการ VM Instances สำหรับนักศึกษา	51
4-5 หน้าจอขอสร้าง VM Instance ใหม่สำหรับนักศึกษา	52
4-6 Dashboard หลักสำหรับอาจารย์และเจ้าหน้าที่	53
4-7 หน้าจัดการหลักสูตรและรายวิชา	53
4-8 หน้าจัดการเทอมการศึกษา	54
4-9 หน้าจัดการรายชื่ออาจารย์	54
4-10 หน้าอนุมัติคำขอที่รอดำเนินการ	55
ก-1 ตัวอย่างคำสั่งสำหรับโคลนรีโพซิทอรีของระบบ	72
ก-2 การติดตั้ง dependencies ของแต่ละบริการ	73
ข-1 การเริ่มต้นบริการประกอบจากรีโพซิทอรี Yuuka	75
ข-2 การเริ่มต้น Momoi ในโหมดพัฒนา	75
ข-3 การเริ่มต้น Yuzu ในโหมดพัฒนา	75
ข-4 การเริ่มต้น Midori ในโหมดพัฒนา	76
ข-5 การสร้างชนิดข้อมูล API ของ Midori ใหม่จาก OpenAPI	76

สารบัญตาราง

ตารางที่	หน้า
3-1	สรุปกลุ่มข้อมูลหลักจาก Prisma Schema ฉบับปัจจุบัน
3-2	รายการ Enum สำคัญในสคีมาฐานข้อมูลฉบับปัจจุบัน
3-3	สรุปกลุ่ม API หลักของบริการ Momoi
3-4	สรุปคิวงานหลักและโครงสร้างข้อมูลงาน
4-1	สรุปทรัพยากรของ Proxmox Cluster ที่ใช้ในระบบ
4-2	สรุปผลการรวมการทดสอบ Midori
4-3	ผลการทดสอบรายเส้นทาง (Midori)
4-4	สรุปผลการรวมการทดสอบ Momoi API
4-5	ผลการทดสอบรายเส้นทาง (Momoi API)
4-6	สรุปผลการทดสอบการโคลน VM: Linked vs Full
4-7	พฤติกรรมของ Yuzu เมื่อจำนวนงานพร้อมกันสัมพันธ์กับจำนวน Pod และ ค่า concurrency
4-8	กลไกควบคุมการทำงานพร้อมกันของ Yuzu ฉบับปัจจุบัน

บทที่ 1

บทนำ

1.1 ความเป็นมาและความสำคัญ

การเรียนการสอนในภาควิชาเทคโนโลยีสารสนเทศมีความจำเป็นต้องใช้ระบบปฏิบัติการลินุกซ์ในหลายรายวิชา โดยเฉพาะในรายวิชาที่เกี่ยวข้องกับระบบเซิร์ฟเวอร์ การพัฒนาแอปพลิเคชัน และการจัดการโครงสร้างพื้นฐานด้านเทคโนโลยีสารสนเทศ อย่างไรก็ตาม การเตรียมสภาพแวดล้อมสำหรับการใช้งานระบบลินุกซ์นั้นมักมีความยุ่งยากและใช้เวลานาน ทั้งในด้านการติดตั้ง การตั้งค่า และการจัดการทรัพยากร ส่งผลให้การเรียนการสอนไม่สามารถดำเนินไปได้อย่างเต็มประสิทธิภาพ

ที่ผ่านมา นักศึกษาและอาจารย์ต้องใช้วิธีการจำลองระบบลินุกซ์ผ่านเครื่องเสมือน (Virtual Machine), ระบบย่อยของวินโดวส์สำหรับลินุกซ์ (WSL), หรือบริการคลาวด์จากภายนอก ซึ่งแต่ละวิธีมีข้อจำกัด เช่น ความยุ่งยากในการตั้งค่า ความไม่เสถียรของระบบ หรือค่าใช้จ่ายเพิ่มเติมในการใช้บริการจากผู้ให้บริการภายนอก โดยเฉพาะในการทำโครงการที่ต้องใช้เซิร์ฟเวอร์ นักศึกษามักต้องจัดหาเครื่องคอมพิวเตอร์ส่วนตัวหรือเซิร์ฟเวอร์ด้วยตนเอง ซึ่งเป็นภาระทั้งด้านเวลาและงบประมาณ

เพื่อแก้ไขปัญหาดังกล่าวข้างต้น โครงการนี้จึงมีแนวคิดในการพัฒนาแพลตฟอร์มคลัสเตอร์ภายในภาควิชา โดยใช้เทคโนโลยี Virtualization ร่วมกับระบบ cloud-init เพื่อสร้างเครื่องจำลองลินุกซ์ที่สามารถใช้งานได้อย่างรวดเร็วและง่ายดาย พร้อมทั้งมีระบบ Web Application สำหรับการจัดการและใช้งานเครื่องเสมือน ทั้งในส่วนของนักศึกษาและอาจารย์

1.2 วัตถุประสงค์ของโครงการ

- 1.2.1 เพื่อออกแบบและพัฒนาแพลตฟอร์มคลัสเตอร์เสมือน (Virtual Cluster Platform) ที่สามารถสร้างและจัดการเครื่องเสมือนลินุกซ์ได้อย่างรวดเร็วและมีประสิทธิภาพ สำหรับสนับสนุนการเรียนการสอนและการทำโครงการในภาควิชาเทคโนโลยีสารสนเทศ
- 1.2.2 เพื่อพัฒนา Web Application แบบ Role-Based Access Control (RBAC) ที่รองรับการใช้งานของทั้งนักศึกษาและอาจารย์ ในการร้องขอ จัดการ และเข้าถึงเครื่องเสมือน โดยคำนึงถึงความสะดวกในการใช้งาน เสถียรภาพของระบบ และความปลอดภัยในการเข้าถึงแพลตฟอร์ม

- 1.2.3 เพื่อเพิ่มประสิทธิภาพการใช้ทรัพยากรเซิร์ฟเวอร์ที่มีอยู่ภายในภาควิชา ผ่านการพัฒนา
ระบบคลัสเตอร์แบบรวมศูนย์ ที่สามารถติดตาม วิเคราะห์ และบริหารจัดการทรัพยากร
ได้อย่างเหมาะสม โดยมีระบบ Monitoring และ Cloud Storage ที่สามารถรองรับการ
ใช้งานในลักษณะ multi-user environment

1.3 ขอบเขตของการทำโครงการ

ภาคการศึกษาที่ 1/2568

- 1) จัดทำระบบการจัดการเครื่องคอมพิวเตอร์เซิร์ฟเวอร์
 - 1.1) จัดทำระบบ Hypervisor สำหรับการทำ Virtualization
 - 1.2) ระบบการจัดการเครื่องเสมือนที่อยู่ภายใต้ระบบ Hypervisor ด้วย API
 - 1.2.1) การสร้างเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.2) การแก้ไขข้อมูลเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.3) การลบเครื่องเสมือนบนระบบ Hypervisor
 - 1.2.4) การจัดการสถานะเครื่องเสมือนบนระบบ Hypervisor
- 2) จัดทำระบบ Web Application สำหรับการเข้าใช้งานระบบ
 - 2.1) ระบบเข้าใช้งานสำหรับนักศึกษา
 - 2.1.1) ระบบการยืนยันตัวตนด้วย Google Account เพื่อเข้าใช้งานระบบด้วยโดเมน @email.kmutnb.ac.th และยืนยันภาควิชาที่นักศึกษาสังกัดด้วยรหัสนักศึกษา ที่ปรากฏบนที่อยู่อีเมลล์
 - 2.1.2) ระบบยื่นคำร้องขอการใช้งานเครื่องเสมือนสำหรับการใช้งานในการเรียนปกติ หรือสำหรับการทำโครงการพิเศษ
 - 2.1.3) ระบบการจัดการเครื่องเสมือนและการเข้าใช้งานเครื่องเสมือนของนักศึกษาที่ยังมีสถานะปกติ
 - 2.1.4) ระบบยื่นคำร้องขอต่ออายุการใช้งานเครื่องเสมือนที่มีอยู่เพื่อขอใช้งานเครื่องเสมือนต่อในภาคการศึกษาถัดไป
 - 2.2) ระบบเข้าใช้งานสำหรับอาจารย์หรือผู้ดูแลระบบ
 - 2.2.1) ระบบการยืนยันตัวตนด้วย Google Account เพื่อเข้าใช้งานระบบด้วยโดเมน @itm.kmutnb.ac.th และจำกัดการเข้าใช้ระบบเฉพาะบุคลากรภายในภาควิชา
 - 2.2.2) ระบบจัดการคำร้องขอใช้งานเครื่องเสมือนและคำร้องขอต่ออายุการใช้งานเครื่องเสมือนของนักศึกษา
 - 2.2.3) ระบบการสร้างเครื่องเสมือนสำหรับการใช้งานของอาจารย์หรือผู้ดูแลระบบ
 - 2.2.4) ระบบการจัดการและเข้าใช้งานเครื่องเสมือนของอาจารย์หรือผู้ดูแลระบบ

- 2.2.5) ระบบการจัดการเครื่องเสมือนของนักศึกษาในระบบ โดยสามารถเปลี่ยนแปลงสถานะของเครื่องเสมือนให้อยู่ในรูปแบบการจับเก็บเครื่องเสมือนถาวรหรือการลบเครื่องเสมือนของนักศึกษา
- 2.2.6) ระบบการกำหนดช่วงเวลาของภาคการศึกษา เพื่อให้ระบบสามารถจัดหมวดหมู่ของเครื่องเสมือนในภาคการศึกษานั้น ๆ ได้
- 2.2.7) ระบบการจัดการรายชื่ออาจารย์หรือผู้ดูแลที่สังกัดภาควิชา

ภาคการศึกษาที่ 2/2568

- 1) จัดทำระบบ Reverse Proxy สำหรับการเข้าใช้งานพอร์ตต่าง ๆ ของเครื่องเสมือนจากระบบเครือข่ายสาธารณะได้
- 2) จัดทำระบบ Cloud Storage สำหรับการจัดเก็บไฟล์สำหรับการใช้งานของอาจารย์หรือผู้ดูแลระบบภายในภาควิชา
- 3) จัดทำระบบ Monitoring ที่จัดเก็บข้อมูล Trace, Metric และ Log โดยสามารถแสดงผลผ่าน Dashboard ได้
- 4) จัดทำการระบบเพิ่มเติมบน Web Application
 - 4.1) ระบบการกำหนดค่า Port Forwarding บนระบบ Reverse Proxy เพื่อกำหนดพอร์ตที่ต้องการใช้งานจากระบบเครือข่ายภายนอก
 - 4.2) ระบบการเข้าใช้งาน Cloud Storage สำหรับอาจารย์หรือผู้ดูแลระบบ เพื่อใช้งานในการจัดเก็บไฟล์
- 5) ประเมินประสิทธิภาพของระบบ
 - 5.1) วัดค่า Provisioning Time วัดค่าเฉลี่ย (Mean) และ ค่ามัธยฐาน (Median) ของเวลาที่ใช้ในการสร้างเครื่องเสมือนใหม่ ตั้งแต่เริ่มต้นกระบวนการจนพร้อมใช้งานเปรียบเทียบกับวิธีการเดิม
 - 5.2) วัดค่า Concurrent User Performance ทดสอบความเร็วการตอบสนองของระบบเมื่อมีผู้ใช้งานพร้อมกันหลายราย วัดค่าการตอบสนองของระบบ (Response Time) เฉลี่ย (ms) และอัตราความล้มเหลว (Error Rate) (%)

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1.4.1 ได้ระบบแพลตฟอร์มคลัสเตอร์ที่สามารถใช้งานได้จริงภายในภาควิชาเทคโนโลยีสารสนเทศ เพื่อสนับสนุนการเรียนการสอนและการทำโครงงานของนักศึกษาและอาจารย์
- 1.4.2 ได้ระบบเครื่องเสมือนลินุกซ์ที่สามารถสร้างและจัดการได้อย่างรวดเร็วผ่าน Web Application โดยไม่ต้องติดตั้งระบบปฏิบัติการเพิ่มเติมบนเครื่องของผู้ใช้งาน

- 1.4.3 ได้ใช้ประโยชน์จากเครื่องเซิร์ฟเวอร์ที่มีอยู่ในภาควิชาอย่างเต็มประสิทธิภาพ โดยนำมาให้บริการในรูปแบบคลัสเตอร์สำหรับการเรียนการสอน
- 1.4.4 ได้โครงการที่สามารถนำไปใช้งานจริงภายในภาควิชา และสามารถต่อยอดพัฒนาเพิ่มเติมในอนาคต
- 1.4.5 ได้โครงการอื่น ๆ ที่สามารถใช้งานได้จริงในภาควิชา โดยนำระบบที่นักศึกษาที่พัฒนาในโครงการมาติดตั้งบนแพลตฟอร์มคลัสเตอร์

บทที่ 2

แนวคิด ทฤษฎี และงานวิจัยที่เกี่ยวข้อง

2.1 แนวคิดพื้นฐานเกี่ยวกับโครงการ

2.1.1 ความสำคัญของการเรียนการสอนด้วยระบบปฏิบัติการลินุกซ์ในการศึกษาด้านเทคโนโลยีสารสนเทศ

การเรียนการสอนในสาขาเทคโนโลยีสารสนเทศ โดยเฉพาะในระดับอุดมศึกษา มีความจำเป็นอย่างยิ่งที่จะต้องให้นักศึกษาได้ฝึกฝนการใช้งานระบบปฏิบัติการลินุกซ์ เนื่องจากระบบปฏิบัติการดังกล่าวเป็นพื้นฐานสำคัญในการทำงานด้านเซิร์ฟเวอร์ การพัฒนาระบบ การจัดการโครงสร้างพื้นฐานด้านไอที และเทคโนโลยีคลาวด์ที่กำลังเป็นที่ต้องการของตลาดแรงงานในปัจจุบัน (Anderson, 2008)

การเรียนรู้และฝึกฝนการใช้งานระบบลินุกซ์มีความสำคัญในหลายประการ ได้แก่

- **พัฒนาทักษะการจัดการระบบ** ช่วยให้นักศึกษาเข้าใจและสามารถจัดการระบบปฏิบัติการที่ใช้กันอย่างแพร่หลายในโลกธุรกิจ โดยเฉพาะในด้านเซิร์ฟเวอร์และระบบคลาวด์
- **เสริมสร้างความเข้าใจในการทำงานของระบบคอมพิวเตอร์** การใช้งานลินุกซ์ผ่าน Command Line Interface ช่วยให้นักศึกษาเข้าใจการทำงานของระบบในระดับที่ลึกซึ้ง
- **เตรียมความพร้อมสู่การทำงาน** ระบบลินุกซ์เป็นมาตรฐานในอุตสาหกรรมเทคโนโลยี การฝึกฝนการใช้งานจะช่วยให้ นักศึกษามีความพร้อมในการทำงานจริง
- **สนับสนุนการเรียนรู้เทคโนโลยีใหม่** หลายเทคโนโลยีสมัยใหม่ เช่น Docker, Kubernetes, และระบบ DevOps ต่าง ๆ ต้องอาศัยความรู้พื้นฐานเกี่ยวกับลินุกซ์ (Lee & Chen, 2021)

อย่างไรก็ตาม การจัดเตรียมสภาพแวดล้อมสำหรับการเรียนการสอนด้วยระบบลินุกซ์นั้นมักเผชิญกับความท้าทายหลายประการ ได้แก่

- **ความยุ่งยากในการติดตั้งและตั้งค่า** การติดตั้งระบบลินุกซ์บนเครื่องคอมพิวเตอร์ส่วนบุคคลของนักศึกษา มักต้องใช้เวลานานและมีความซับซ้อน โดยเฉพาะสำหรับผู้ที่ไม่มีความชำนาญ
- **ข้อจำกัดด้านฮาร์ดแวร์** เครื่องคอมพิวเตอร์ส่วนบุคคลของนักศึกษาอาจมีข้อจำกัดด้านหน่วยความจำหรือพื้นที่จัดเก็บข้อมูล ทำให้การใช้งานเครื่องเสมือนไม่เสถียร
- **ความไม่สม่ำเสมอของสภาพแวดล้อม** นักศึกษาแต่ละคนอาจมีการตั้งค่าและเวอร์ชันของซอฟต์แวร์ที่แตกต่างกัน ทำให้เกิดปัญหาในการเรียนการสอนแบบเดียวกัน
- **การบำรุงรักษาและแก้ไขปัญหา** เมื่อเกิดปัญหาทางเทคนิค อาจารย์และนักศึกษาต้องใช้เวลามากในการแก้ไขปัญหาแทนที่จะมุ่งเน้นไปที่การเรียนรู้เนื้อหาหลัก

- **ค่าใช้จ่ายในการใช้บริการภายนอก** การใช้บริการคลาวด์จากผู้ให้บริการภายนอกมักมีค่าใช้จ่ายที่สูง โดยเฉพาะเมื่อต้องใช้งานตลอดภาคการศึกษา

2.1.2 บทบาทของเทคโนโลยี Virtualization และ Cloud Computing ในการศึกษา

ในยุคที่เทคโนโลยีสารสนเทศพัฒนาอย่างรวดเร็ว การนำเทคโนโลยี Virtualization และ Cloud Computing มาประยุกต์ใช้ในการศึกษาได้กลายเป็นแนวทางที่สำคัญในการแก้ไขปัญหาและเพิ่มประสิทธิภาพการเรียนการสอน โดยเฉพาะอย่างยิ่งในสาขาเทคโนโลยีสารสนเทศที่ต้องการสภาพแวดล้อมที่หลากหลายและซับซ้อนสำหรับการฝึกปฏิบัติ (Clark & Kwinn, 2016)

เทคโนโลยี Virtualization และ Cloud Computing มีบทบาทสำคัญในการเพิ่มประสิทธิภาพการเรียนการสอนหลายประการ ได้แก่

- **ลดเวลาและความซับซ้อนในการตั้งค่าระบบ** เทคโนโลยีเครื่องเสมือนสามารถสร้างสภาพแวดล้อมที่พร้อมใช้งานได้อย่างรวดเร็ว โดยไม่ต้องให้นักศึกษาแต่ละคนติดตั้งและตั้งค่าระบบด้วยตนเอง
- **เพิ่มความยืดหยุ่นในการจัดการทรัพยากร** สามารถปรับเปลี่ยนขนาดและสเปกของเครื่องเสมือนตามความต้องการของแต่ละรายวิชาหรือโครงการ โดยไม่ต้องลงทุนซื้อฮาร์ดแวร์เพิ่มเติม
- **รองรับการเรียนรู้แบบ Remote Learning** นักศึกษาสามารถเข้าถึงและใช้งานระบบได้จากที่ไหนก็ได้ トラバใดที่มีการเชื่อมต่ออินเทอร์เน็ต ทำให้การเรียนการสอนไม่ถูกจำกัดด้วยสถานที่และเวลา
- **สร้างความสม่ำเสมอของสภาพแวดล้อม** ทุกคนจะได้ใช้งานระบบที่มีการตั้งค่าเหมือนกัน ลดปัญหาที่เกิดจากความแตกต่างของสภาพแวดล้อมในเครื่องของแต่ละคน
- **ประหยัดค่าใช้จ่าย** การใช้ทรัพยากรแบบรวมศูนย์ช่วยลดค่าใช้จ่ายในการซื้อและบำรุงรักษาฮาร์ดแวร์ รวมถึงลดค่าใช้จ่ายสำหรับนักศึกษาในการจัดหาเครื่องมือสำหรับการเรียน

แนวคิดของแพลตฟอร์มคลัสเตอร์สำหรับการศึกษาจึงเป็นการต่อยอดจากข้อดีของเทคโนโลยีเหล่านี้ โดยการสร้างระบบที่สามารถ

- **จัดการเครื่องเสมือนแบบรวมศูนย์** ผ่านระบบ Web Application ที่ใช้งานง่าย ทำให้อาจารย์และนักศึกษาสามารถร้องขอ สร้าง และจัดการเครื่องเสมือนได้โดยไม่ต้องมีความรู้ลึกด้านระบบ
- **ใช้ประโยชน์จากทรัพยากรที่มีอยู่อย่างเต็มที่** การรวมเครื่องเซิร์ฟเวอร์หลายเครื่องเข้าเป็นคลัสเตอร์เดียว ช่วยให้สามารถใช้งานทรัพยากรได้อย่างมีประสิทธิภาพสูงสุด
- **รองรับการใช้งานหลากหลายรูปแบบ** ทั้งการเรียนการสอนปกติ การทำโครงการ การวิจัย และการพัฒนาระบบต่าง ๆ ภายในภาควิชา

- มีระบบติดตามและบริหารจัดการ ด้วยระบบ Monitoring ที่ช่วยในการติดตามการใช้งาน และแก้ไขปัญหาได้อย่างรวดเร็ว

2.2 แนวคิดและทฤษฎีที่เกี่ยวข้องกับแพลตฟอร์มคลัสเตอร์เสมือน

2.2.1 เทคโนโลยี Virtualization

Virtualization Technology เป็นเทคนิคการจำลองซอฟต์แวร์หรือฮาร์ดแวร์บนซอฟต์แวร์อื่น โดยสภาพแวดล้อมจำลองนี้เรียกว่าเครื่องเสมือน (Virtual Machine) ซึ่งเป็นเทคโนโลยีพื้นฐานสำคัญที่ทำให้สามารถแบ่งทรัพยากรฮาร์ดแวร์เครื่องเดียวออกเป็นหลาย ๆ ระบบที่ทำงานได้อิสระจากกัน (Garcia & Rodriguez, 2023)

Hypervisor และประเภทของ Virtualization

- **Type 1 Hypervisor (Bare-metal)** เป็น Hypervisor ที่ทำงานโดยตรงบนฮาร์ดแวร์ โดยไม่ต้องผ่านระบบปฏิบัติการหลัก เช่น VMware vSphere, Microsoft Hyper-V, และ Proxmox VE ซึ่งมีประสิทธิภาพสูงและเหมาะสำหรับการใช้งานในระดับองค์กร
- **Type 2 Hypervisor (Hosted)** เป็น Hypervisor ที่ทำงานบนระบบปฏิบัติการหลัก เช่น VMware Workstation, VirtualBox ซึ่งง่ายต่อการใช้งานแต่มีประสิทธิภาพต่ำกว่า Type 1
- **Container-based Virtualization** เป็นรูปแบบของ Virtualization ที่แบ่งปันระบบปฏิบัติการเดียวกัน เช่น Docker, LXC ซึ่งมีความเร็วและประสิทธิภาพสูงกว่าแต่มีความแยกตัวน้อยกว่าเครื่องเสมือนแบบเต็มรูปแบบ

ข้อดีของเทคโนโลยี Virtualization

- **การใช้ทรัพยากรอย่างมีประสิทธิภาพ** สามารถใช้ทรัพยากรฮาร์ดแวร์ได้เต็มประสิทธิภาพ เนื่องจากสามารถรันหลายระบบพร้อมกันได้บนเครื่องเดียว
- **ความยืดหยุ่นในการจัดการ** สามารถสร้าง ลบ หรือปรับแต่งเครื่องเสมือนได้อย่างรวดเร็ว โดยไม่ต้องแตะต้องฮาร์ดแวร์จริง
- **การแยกตัวและความปลอดภัย** แต่ละเครื่องเสมือนทำงานแยกจากกัน หากเครื่องหนึ่งเกิดปัญหาจะไม่ส่งผลกระทบต่อเครื่องอื่น
- **ความสะดวกในการบำรุงรักษา** สามารถสำรองข้อมูล ย้าย หรือกู้คืนระบบได้ง่าย โดยไม่ต้องหยุดการทำงานของเครื่องอื่น

2.2.2 Cloud Computing และ Container Orchestration

Cloud Computing เป็นแนวคิดการประมวลผลแบบคลาวด์ที่ช่วยให้สามารถเข้าถึงทรัพยากรการประมวลผลที่กำหนดค่าได้ตามต้องการ ซึ่งสามารถจัดเตรียมและปล่อยทรัพยากรได้อย่างรวดเร็ว

โดยใช้ความพยายามในการจัดการหรือการโต้ตอบระหว่างผู้ให้บริการน้อยที่สุด โดยมีรูปแบบการให้บริการหลัก ได้แก่

รูปแบบการให้บริการ Cloud Computing

- **Infrastructure as a Service (IaaS)** การให้บริการโครงสร้างพื้นฐานด้านไอที เช่น เครื่องเสมือน พื้นที่จัดเก็บข้อมูล เครือข่าย โดยผู้ใช้งานสามารถควบคุมระบบปฏิบัติการและแอปพลิเคชันได้
- **Platform as a Service (PaaS)** การให้บริการแพลตฟอร์มสำหรับการพัฒนาและปรับใช้แอปพลิเคชัน โดยไม่ต้องจัดการโครงสร้างพื้นฐานด้านล่าง
- **Software as a Service (SaaS)** การให้บริการซอฟต์แวร์ที่พร้อมใช้งานผ่านอินเทอร์เน็ต โดยไม่ต้องติดตั้งหรือจัดการด้วยตนเอง

Container และ Container Orchestration

Container คือการรวมแอปพลิเคชันและรหัสไบনারี ไบเบรารี และการกำหนดค่าเข้าด้วยกันในขณะแบ่งปันอิมเมจระบบปฏิบัติการโฮสต์ ดังนั้นคอนเทนเนอร์จึงแบ่งปันทรัพยากรและดำเนินการไมโครเซอร์วิสขนาดเล็ก โปรแกรมซอฟต์แวร์ และแอปพลิเคชันที่ครอบคลุมยิ่งขึ้นได้อย่างมีประสิทธิภาพ ด้วยค่าใช้จ่ายด้านการจัดการน้อยกว่าเครื่องเสมือน

- **ข้อดีของ Container** มีขนาดเล็ก เริ่มต้นเร็ว ใช้ทรัพยากรน้อย และสามารถพกพาได้ง่าย เนื่องจากบรรจุทุกสิ่งที่จำเป็นสำหรับการทำงานของแอปพลิเคชัน
- **Docker** เป็นแพลตฟอร์ม Container ที่ได้รับความนิยมสูงสุด ช่วยให้การสร้าง จัดการ และปรับใช้ Container เป็นเรื่องง่าย

Container Orchestration คือการปรับใช้และจัดการแอปพลิเคชันที่เป็น Container ที่อยู่ในคลัสเตอร์ที่เชื่อมต่อถึงกัน โดยมีโหนดที่คอยควบคุมการทำงานที่เรียกว่าโหนดมาสเตอร์ (Master node) ซึ่งมีหน้าที่ดูแลคลัสเตอร์และจัดการการปรับใช้คอนเทนเนอร์บนโหนดเวิร์กเกอร์ (Worker node)

- **การจัดการทรัพยากรอัตโนมัติ** ระบบ จะ จัดสรร ทรัพยากร ให้กับ Container ตาม ความ ต้องการและสถานะของคลัสเตอร์
- **High Availability** สามารถกระจาย Container ไปยังหลายโหนดเพื่อป้องกันการหยุดทำงานเมื่อโหนดใดโหนดหนึ่งเกิดปัญหา
- **Auto Scaling** สามารถปรับขนาดของแอปพลิเคชันขึ้นลงอัตโนมัติตามปริมาณการใช้งาน
- **Service Discovery และ Load Balancing** ช่วยในการเชื่อมต่อ Container ต่าง ๆ และกระจายโหลดการทำงาน

2.2.3 Web Application และเทคโนโลยีสนับสนุน

Web Application คือการพัฒนาระบบงานบนเว็บ ข้อมูลต่าง ๆ ในระบบสามารถเข้าถึงได้ทั้งเครือข่ายภายในหรือเครือข่ายสาธารณะ ซึ่งทำให้ใช้งานง่ายผ่านเว็บเบราว์เซอร์ โดยในการใช้งานนั้นไม่จำเป็นต้องติดตั้งโปรแกรมใด ๆ เพิ่มเติม ทำให้เหมาะสำหรับการใช้งานในสภาพแวดล้อมที่มีผู้ใช้งานหลากหลาย

คุณสมบัติสำคัญของ Web Application สำหรับการจัดการระบบคลัสเตอร์

- **Cross-platform Compatibility** สามารถใช้งานได้บนระบบปฏิบัติการต่าง ๆ โดยไม่ต้องพัฒนาแยกสำหรับแต่ละแพลตฟอร์ม
- **Centralized Management** ช่วยให้สามารถจัดการระบบจากจุดเดียว ลดความซับซ้อนในการดูแลระบบหลายเครื่อง
- **Role-Based Access Control (RBAC)** สามารถกำหนดสิทธิ์การเข้าถึงตามบทบาทของผู้ใช้ เช่น นักศึกษา อาจารย์ และผู้ดูแลระบบ
- **Real-time Monitoring** สามารถแสดงสถานะของระบบแบบเรียลไทม์ ช่วยในการติดตามและแก้ไขปัญหาได้รวดเร็ว

Cloud-Init และการตั้งค่าเครื่องเสมือนอัตโนมัติ

Cloud-Init คือการตั้งค่าที่ถูกเตรียมไว้ล่วงหน้าสำหรับการจัดเตรียมเครื่องจำลองใหม่โดยอัตโนมัติ ซึ่งรวมถึงการสร้างข้อมูลประจำตัวผู้ใช้และการตั้งค่าระบบขั้นพื้นฐาน ด้วยการสร้างไฟล์ cloud-config ที่ปรับให้เหมาะกับเครื่องเสมือนแต่ละเครื่องอย่างไดนามิก

- **การตั้งค่าผู้ใช้งานอัตโนมัติ** สามารถสร้างบัญชีผู้ใช้ กำหนดรหัสผ่าน และตั้งค่า SSH keys ได้โดยอัตโนมัติ
- **การติดตั้งซอฟต์แวร์เริ่มต้น** สามารถกำหนดให้ติดตั้งแพ็คเกจต่าง ๆ ที่จำเป็นสำหรับการใช้งานเบื้องต้น
- **การตั้งค่าเครือข่าย** สามารถกำหนดค่า IP Address, DNS, และการตั้งค่าเครือข่ายอื่น ๆ ได้
- **การรันสคริปต์เริ่มต้น** สามารถกำหนดให้รันคำสั่งหรือสคริปต์ต่าง ๆ หลังจากที่ยระบบบูตขึ้นมาแล้ว

Reverse Proxy และการจัดการการเข้าถึง

Reverse Proxy คือตัวกลางที่ส่งข้อมูลเครือข่ายที่แลกเปลี่ยนข้อมูลระหว่างไคลเอนต์และเซิร์ฟเวอร์ โดยจะรับคำขอจากไคลเอนต์บนเครือข่ายสาธารณะ (อินเทอร์เน็ต) ส่งต่อคำขอไปยังเซิร์ฟเวอร์ในเครือข่ายภายใน จากนั้นจึงส่งคำตอบที่สร้างโดยเซิร์ฟเวอร์ไปยังไคลเอนต์

- **Port Forwarding** ช่วยให้สามารถเข้าถึงบริการต่าง ๆ ในเครื่องเสมือนจากภายนอกได้โดยผ่านพอร์ตที่กำหนด

- **Load Balancing** สามารถกระจายโหลดไปยังเซิร์ฟเวอร์หลายตัวเพื่อเพิ่มประสิทธิภาพ
- **SSL Termination** สามารถจัดการการเข้ารหัส SSL/TLS ได้ที่ตัว Proxy โดยไม่ต้องตั้งค่าในแต่ละเซิร์ฟเวอร์
- **Security** ช่วยป้องกันการเข้าถึงโดยตรงไปยังเซิร์ฟเวอร์ภายใน และสามารถกำหนดกฎการเข้าถึงได้

สรุปได้ว่า การนำเทคโนโลยี Virtualization, Cloud Computing, Container Orchestration, Web Application และเทคโนโลยีสนับสนุนต่าง ๆ มาใช้ร่วมกันในการพัฒนาแพลตฟอร์มคลัสเตอร์สำหรับการศึกษา จะช่วยให้สามารถสร้างสภาพแวดล้อมการเรียนการสอนที่มีประสิทธิภาพ ยืดหยุ่น และตอบสนองความต้องการของผู้ใช้งานได้หลากหลายรูปแบบ โดยเฉพาะอย่างยิ่งในสาขาเทคโนโลยีสารสนเทศที่ต้องการการฝึกปฏิบัติในสภาพแวดล้อมที่เสมือนจริง

2.3 รายงานการค้นคว้า การศึกษา หรืองานวิจัยที่เกี่ยวข้อง

2.3.1 การพัฒนาระบบ Virtualization สำหรับการศึกษา

การศึกษาวิจัยเรื่อง "Guide to Security for Full Virtualization Technologies" โดย Karen Scarfone, Murugiah Souppaya, และ Paul Hoffman ได้นำเสนอแนวทางและมาตรฐานด้านความปลอดภัยสำหรับเทคโนโลยี Virtualization แบบเต็มรูปแบบ (Karen Scarfone, 2011). งานวิจัยนี้มีความสำคัญต่อการพัฒนาแพลตฟอร์มคลัสเตอร์เนื่องจากให้ข้อมูลเกี่ยวกับการจำแนกประเภทของ Virtualization ตามชั้นสถาปัตยกรรมคอมพิวเตอร์ และแนวทางการรักษาความปลอดภัยที่จำเป็นซึ่งเป็นองค์ความรู้พื้นฐานสำคัญในการออกแบบระบบที่ปลอดภัยและเหมาะสมสำหรับสภาพแวดล้อมการศึกษา

2.3.2 การประยุกต์ใช้ Cloud Computing ในการศึกษา

Danairat T. ได้นำเสนอเอกสารเรื่อง "Cloud Computing Technology The Architecture Overview" ซึ่งให้ภาพรวมของสถาปัตยกรรม Cloud Computing และแนวคิดการประมวลผลแบบคลาวด์ที่ช่วยให้สามารถเข้าถึงทรัพยากรการประมวลผลที่กำหนดค่าได้ตามต้องการ (T., 2015). งานศึกษานี้เป็นแนวทางสำคัญในการทำความเข้าใจรูปแบบการให้บริการ เช่น IaaS, PaaS, และ SaaS ซึ่งเป็นแนวคิดที่นำมาประยุกต์ใช้ในการพัฒนาแพลตฟอร์มคลัสเตอร์เพื่อให้บริการเครื่องเสมือนแก่ผู้ใช้งานในสถาบันการศึกษา

2.3.3 การพัฒนาระบบติดตามความก้าวหน้าสำหรับเอกสารโครงการพิเศษ

Rawiporn Suamsiri และ Kamonwan Janmanee ได้พัฒนาระบบ **"Progress Tracking System for Special Project Documents"** ซึ่งเป็นระบบ Web Application ที่ช่วยในการติดตามความก้าวหน้าของการจัดทำเอกสารโครงการพิเศษ (Suamsiri & Janmanee, 2024). งานวิจัยนี้แสดงให้เห็นถึงแนวทางการพัฒนา Web Application ที่ใช้งานง่ายผ่านเว็บเบราว์เซอร์โดยไม่ต้องติดตั้งโปรแกรมเพิ่มเติม ซึ่งเป็นแนวคิดที่สอดคล้องกับการพัฒนาส่วนติดต่อผู้ใช้งานสำหรับแพลตฟอร์มคลัสเตอร์ในโครงการนี้

2.3.4 การใช้งาน Container Technologies ในสถาปัตยกรรมต่าง ๆ

งานวิจัยของ SHAHIDULLAH KAISER, MD.SADUNHAQ, ALI AMAN TOSUN, และ TURGAY KORKMAZ เรื่อง **"Container Technologies for ARM Architecture: A Comprehensive Survey of the State-of-the-Art"** นำเสนอการสำรวจเทคโนโลยี Container บนสถาปัตยกรรม ARM อย่างครอบคลุม (Kaiser et al., 2022). แสดงให้เห็นถึงการพัฒนาและการใช้งาน Container ที่มีประสิทธิภาพสูง ใช้ทรัพยากรน้อย และสามารถดำเนินการไมโครเซอร์วิสได้อย่างมีประสิทธิภาพ ซึ่งเป็นข้อมูลสำคัญในการออกแบบระบบที่ใช้ Container เป็นส่วนหนึ่งของโครงสร้างพื้นฐาน

2.3.5 Container Orchestration ในอุตสาหกรรมยานยนต์

Naresh Nayak, Dennis Grewe, และ Sebastian Schildt ได้นำเสนอการศึกษาเรื่อง **"Automotive Container Orchestration: Requirements, Challenges and Open Directions"** ซึ่งอธิบายแนวคิดของ Container Orchestration ในการปรับใช้และจัดการแอปพลิเคชันที่เป็น Container ในคลัสเตอร์ (Nayak et al., 2023). งานวิจัยนี้ให้ความรู้เกี่ยวกับการทำงานของโหนดมาสเตอร์และโหนดเวิร์กเกอร์ รวมถึงคุณสมบัติสำคัญ เช่น High Availability, Auto Scaling, และ Load Balancing ซึ่งเป็นแนวคิดที่นำมาประยุกต์ใช้ในการพัฒนาระบบจัดการคลัสเตอร์ในโครงการนี้

2.3.6 การสร้างโครงสร้างพื้นฐานคลาวด์สำหรับการจัดตารางเครื่องเสมือน

Jonathan Chya และ Xunfei Jiang ได้นำเสนอการศึกษาเรื่อง **"Building a Cloud Infrastructure for Virtual Machine Scheduling in Datacenters"** ซึ่งเป็นการศึกษาเกี่ยวกับการสร้างโครงสร้างพื้นฐานคลาวด์สำหรับการจัดตารางเครื่องเสมือนในศูนย์ข้อมูล รวมถึงการใช้ Cloud-Init ในการจัดเตรียมเครื่องเสมือนใหม่โดยอัตโนมัติ (Chya & Jiang, 2024). งานวิจัยนี้แสดงให้เห็นถึงการใช้ไฟล์ cloud-config ที่ปรับให้เหมาะกับเครื่องเสมือนแต่ละเครื่องอย่างไดนามิก ซึ่งช่วยเพิ่มประสิทธิภาพด้านพลังงาน ความเร็ว และการปรับแต่งกระบวนการจัดเตรียมเครื่องเสมือนได้อย่างมาก

2.3.7 การพัฒนาสถาปัตยกรรม Reverse Proxy

Yu TAO และ Gang CHEN ได้เผยแพร่งานวิจัยเรื่อง **"An Extensible Universal Reverse Proxy Architecture"** ซึ่งนำเสนอสถาปัตยกรรม Reverse Proxy ที่มีความยืดหยุ่นและสามารถขยายได้ (Tao & Chen, 2016). งานวิจัยนี้อธิบายการทำงานของ Reverse Proxy ในการรับคำขอจากไคลเอนต์บนเครือข่ายสาธารณะและส่งต่อไปยังเซิร์ฟเวอร์ในเครือข่ายภายใน รวมถึงความสามารถในการแคชทรัพยากรเพื่อเพิ่มประสิทธิภาพ ซึ่งเป็นแนวคิดสำคัญที่นำมาใช้ในการพัฒนาระบบ Port Forwarding และการจัดการการเข้าถึงเครื่องเสมือนจากภายนอกในโครงการนี้

บทที่ 3

วิธีการดำเนินงานและการออกแบบระบบ

3.1 การออกแบบสถาปัตยกรรมระบบ

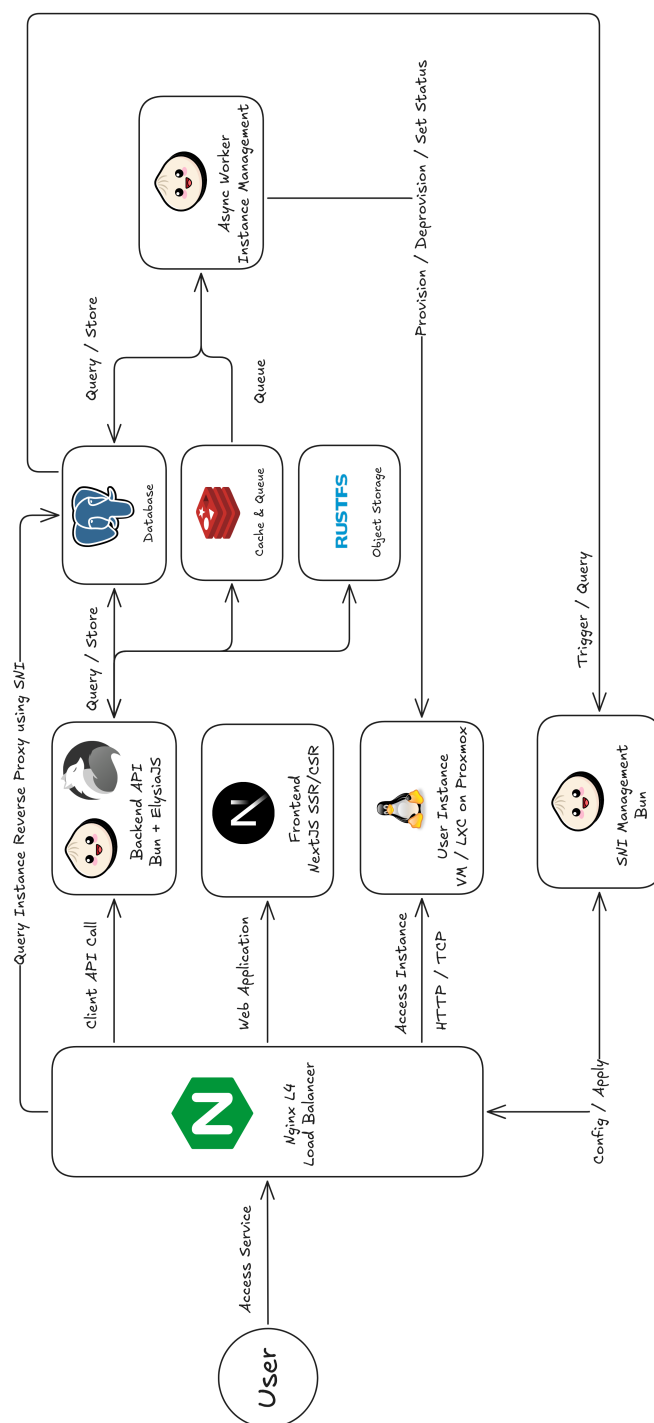
3.1.1 สถาปัตยกรรมระบบโดยรวม

ระบบแพลตฟอร์มคลัสเตอร์เสมือนในฉบับพัฒนาปัจจุบันยังคงยึดแนวคิด Microservices Architecture แต่เมื่อพิจารณาทั้งบริการหลักและบริการสนับสนุนที่ใช้ในการติดตั้งจริง สามารถแบ่งองค์ประกอบออกเป็น 5 บทบาทสำคัญ คือ

- **Frontend (Midori)** เว็บแอปพลิเคชันสำหรับผู้ใช้งาน พัฒนาด้วย Next.js และ React ทำหน้าที่เป็นส่วนติดต่อผู้ใช้สำหรับนักศึกษา อาจารย์ และผู้ดูแลระบบ
- **API Service (Momoi)** เซิร์ฟเวอร์หลักของระบบ พัฒนาด้วย Bun, Elysia.js และ Prisma รับผิดชอบโดเมนด้านข้อมูลวิชาการ คำร้องขอเครื่องเสมือน การจัดการเครื่องเสมือน พื้นที่จัดเก็บไฟล์ การยืนยันตัวตนผ่านเส้นทาง `/api/auth` และการเชื่อมต่อกับบริการภายนอก
- **Worker / VM Management (Yuzu)** เซอร์วิสประมวลผลงานเบื้องหลังสำหรับการสร้าง ลบ และเปลี่ยนสถานะเครื่องเสมือน โดยสื่อสารกับ Proxmox VE แบบ Asynchronous รองรับการทำงาน concurrency ของแต่ละคิวผ่านตัวแปรสภาพแวดล้อม และมี distributed locks เพื่อป้องกันงานที่ชนทรัพยากรเดียวกัน
- **Reverse Proxy and Access Automation (Satsuki)** บริการที่ติดตั้งบนเครื่อง Nginx ภายนอกซึ่งทำหน้าที่เป็น reverse proxy หน้าด่านของระบบ ใช้สร้างไฟล์ reverse proxy configuration และ `authorized_keys` จากข้อมูลในฐานข้อมูล พร้อมติดตามการเปลี่ยนแปลงด้วย PostgreSQL LISTEN/NOTIFY และสั่ง reload บริการปลายทางเมื่อข้อมูลเปลี่ยน
- **Supporting Services and Deployment (Yuuka)** ชุดโครงสร้างพื้นฐานและไฟล์สำหรับการติดตั้งใช้งานจริง เช่น PostgreSQL, Redis, RustFS และระบบสังเกตการณ์การทำงานของแพลตฟอร์ม

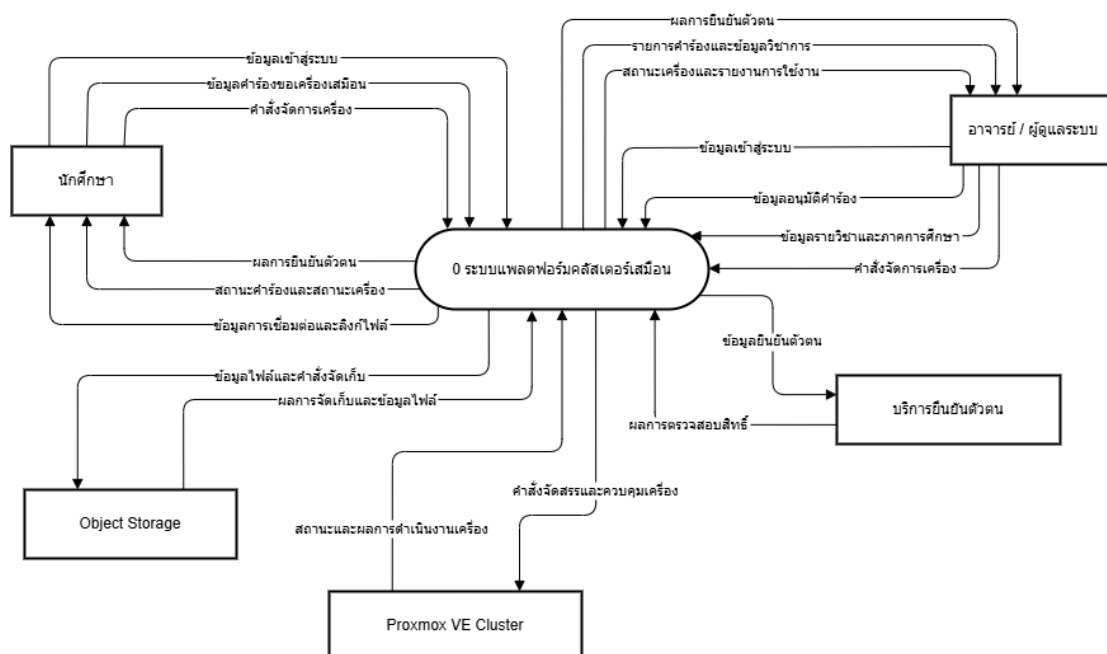
ในสภาพแวดล้อมที่ติดตั้งใช้งานจริง เส้นทางการทำงานหลักของระบบสามารถสรุปได้เป็น Browser > Midori > Momoi สำหรับคำขอด้านข้อมูล ธุรกรรมทางธุรกิจ และการยืนยันตัวตนผ่าน `/api/auth` ส่วนงานที่ใช้เวลานาน เช่น การสร้าง ลบ หรือเปลี่ยนสถานะเครื่องเสมือน จะถูก Momoi ส่งเข้าสู่ Redis-backed BullMQ ก่อนที่ Yuzu จะรับงานไปดำเนินการกับ Proxmox VE ต่อไป และเมื่อข้อมูล

reverse proxy หรือ SSH key เปลี่ยนแปลง Satsuki ซึ่งทำงานอยู่บนเครื่อง Nginx ภายนอกจะรับสัญญาณจากฐานข้อมูลเพื่อสร้างไฟล์กำหนดค่าที่เกี่ยวข้องใหม่และ reload reverse proxy หน้าด้านโดยอัตโนมัติ การแยกเส้นทางแบบนี้ทำให้ระบบส่วนติดต่อผู้ใช้และ API ตอบสนองได้รวดเร็ว แม้งานด้านโครงสร้างพื้นฐานจะมีระยะเวลาดำเนินการยาวนานกว่ามาก



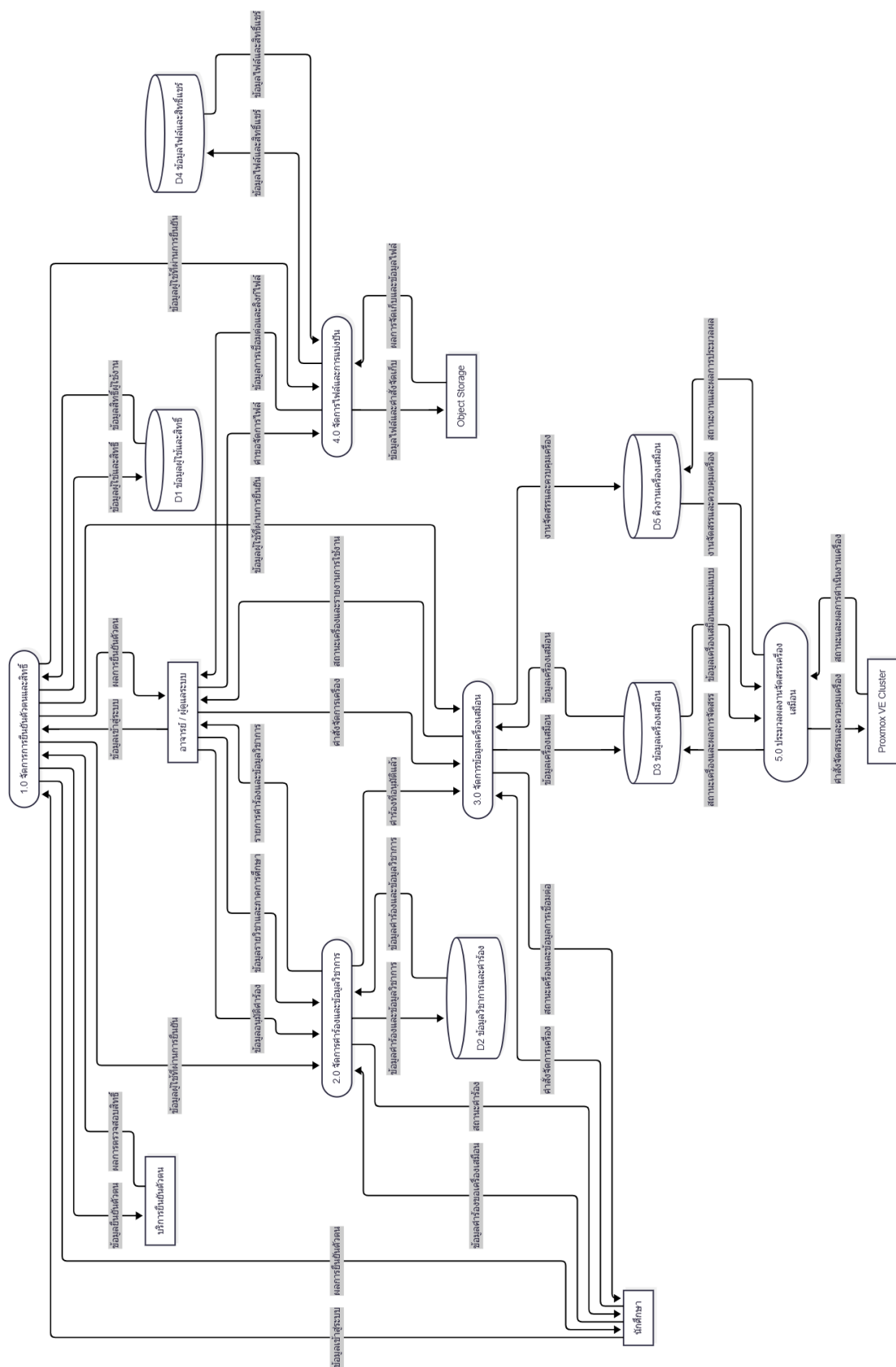
ภาพที่ 3-1 แผนภาพการทำงานและลำดับขั้นตอนโดยรวมของโปรแกรม

จากภาพที่ 3-1 จะเห็นลำดับการทำงานหลักของแพลตฟอร์มอย่างเป็นขั้นตอน โดยผู้ใช้เริ่มต้นจากการใช้งานผ่าน Midori ซึ่งเป็นส่วนติดต่อผู้ใช้ จากนั้นคำขอจะถูกส่งต่อไปยัง Momoi เพื่อประมวลผลธุรกรรมทางธุรกิจ ตรวจสอบสิทธิ์ และบันทึกข้อมูลลงในฐานข้อมูล หากเป็นงานที่ต้องใช้เวลานานหรือเกี่ยวข้องกับการจัดสรรทรัพยากรเครื่องเสมือน Momoi จะส่งงานเข้าสู่คิวเพื่อให้ Yuzu รับไปดำเนินการกับ Proxmox VE แบบอะซิงโครนัส ขณะเดียวกันข้อมูลที่เกี่ยวข้องกับ reverse proxy และ SSH access จะถูก Satsuki ซึ่งติดตั้งอยู่บนเครื่อง Nginx ภายนอกนำไปสร้างไฟล์กำหนดค่าและ reload บริการหน้าด่านให้สอดคล้องกับสถานะล่าสุดของระบบ ภาพนี้จึงช่วยสรุปความสัมพันธ์ระหว่างบริการหลัก เส้นทางข้อมูล และจุดที่เกิดการประมวลผลเบื้องหลัง ก่อนจะลงรายละเอียดในแผนภาพบริบทและแผนภาพการไหลของข้อมูลระดับถัดไป



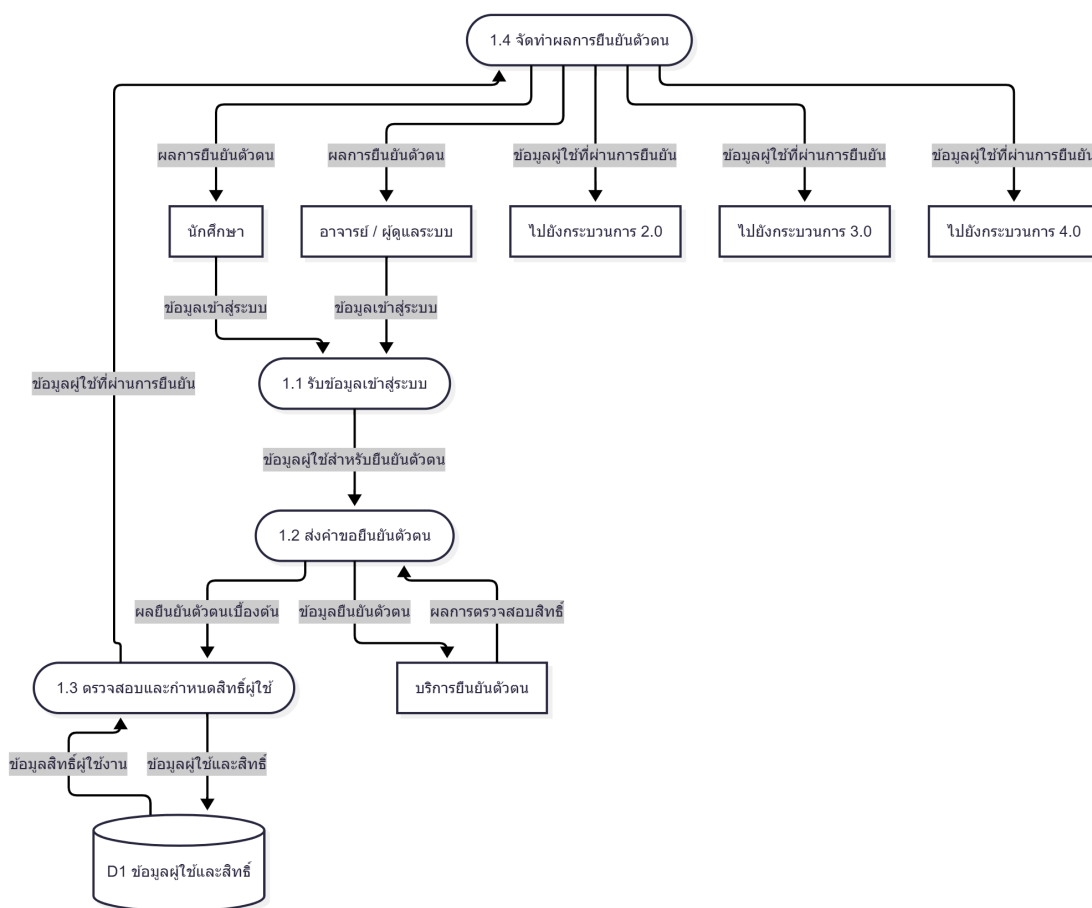
ภาพที่ 3-2 แผนภาพบริบทของระบบแพลตฟอร์มคลัสเตอร์เสมือน

จากภาพที่ 3-2 ระบบถูกมองเป็นกระบวนการเดียวในลักษณะ Black Box เพื่อแสดงเฉพาะความสัมพันธ์ระหว่างระบบกับเอนทิตีภายนอก ได้แก่ นักศึกษา อาจารย์หรือผู้ดูแลระบบ บริการยืนยันตัวตน Proxmox VE Cluster และ Object Storage โดยไม่มีการแสดงแหล่งเก็บข้อมูลภายในตามหลักของ Context Diagram จากนั้นในภาพที่ 3-3 ได้ทำการแตกกระบวนการหลักของระบบออกเป็น 5 กระบวนการ พร้อมเพิ่มแหล่งเก็บข้อมูลที่จำเป็น โดยแยกแสดงเป็น 3 มุมมองเพื่อความชัดเจน และในภาพที่ 3-4 – 3-8 ได้ขยายรายละเอียดทั้ง 5 กระบวนการหลัก เพื่อรักษาความสมดุลของกระแสข้อมูลระหว่างแต่ละระดับของ DFD ให้สอดคล้องกัน



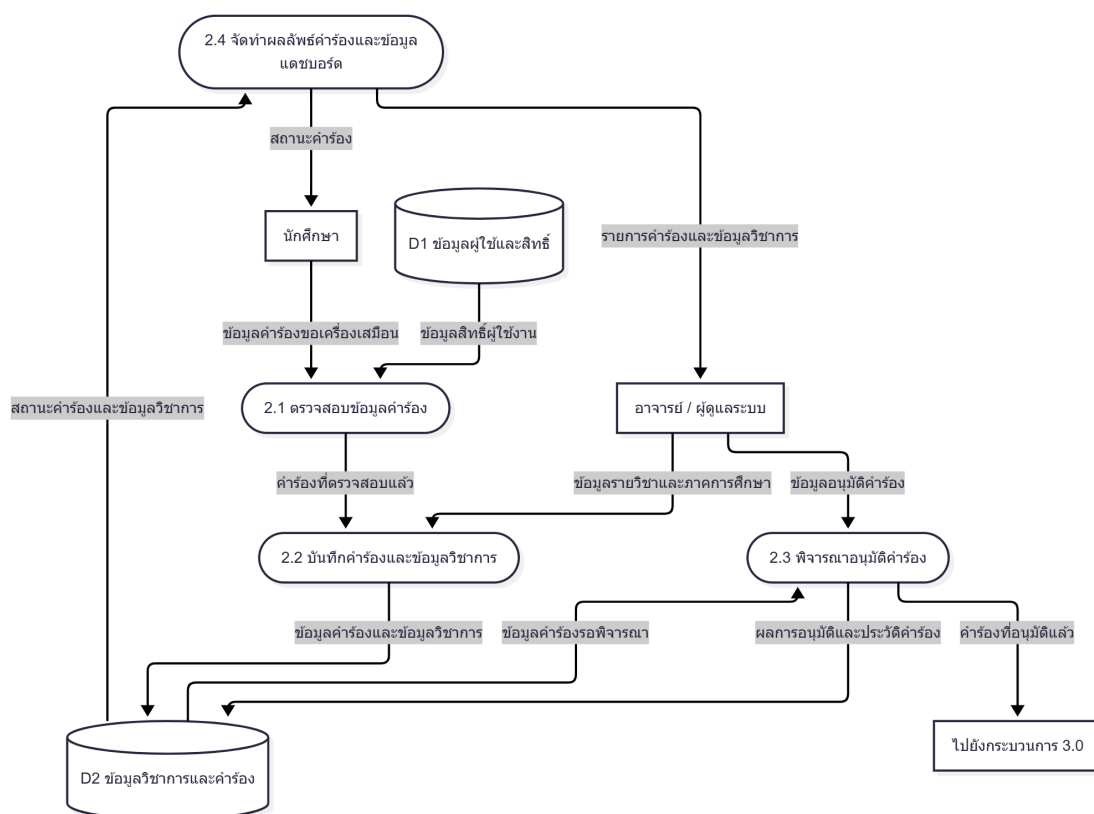
ภาพที่ 3-3 แผนภาพการไหลของข้อมูลระดับ 0: ภาพรวมทั้งระบบ

ในระดับแผนภาพการไหลของข้อมูลระดับที่ 1 (DFD Level 1) ตามภาพที่ 3-4 – 3-8 ได้ทำการขยายกระบวนการหลักจากระดับ 0 ให้เห็นรายละเอียดของกระแสข้อมูลภายในระบบมากขึ้น โดยยังคงรักษาหลัก *Balancing* กล่าวคือ ข้อมูลเข้าและข้อมูลออกของแต่ละกระบวนการหลักต้องสอดคล้องกับสิ่งที่ปรากฏใน DFD ระดับ 0 และไม่สร้าง/ลบกระแสข้อมูลที่ไม่มีอยู่ในระดับก่อนหน้า ทั้งนี้สามารถสรุปแนวคิดสำคัญของกระบวนการระดับที่ 1 ได้ดังนี้



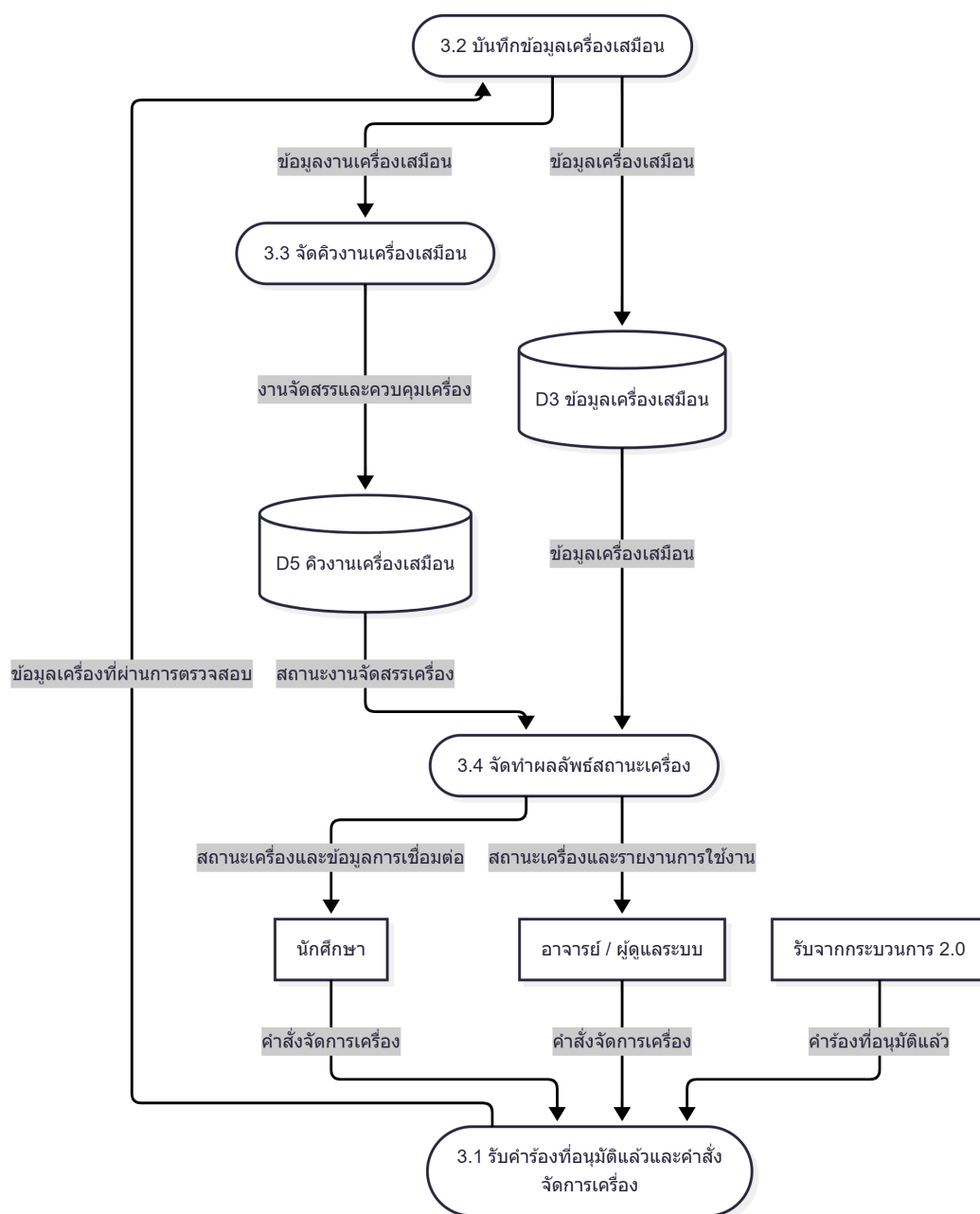
ภาพที่ 3-4 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 1.0 การยืนยันตัวตนและกำหนดสิทธิ์ผู้ใช้

กระบวนการ 1.0 การยืนยันตัวตนและกำหนดสิทธิ์ผู้ใช้ (ภาพที่ 3-4) ทำหน้าที่เป็น *gatekeeper* ของ API ทั้งระบบ โดยรับข้อมูลยืนยันตัวตนจากผู้ใช้ผ่าน Midori จากนั้นสร้าง/ตรวจสอบ Session และผูกกับข้อมูลผู้ใช้ในฐานข้อมูลเพื่อกำหนดบทบาท (ADMIN/INSTRUCTOR/STUDENT) เส้นทางนี้ให้บริการผ่านโมดูล *better-auth* ภายใน Momoi และใช้ฐานข้อมูลพร้อมกติกากำหนดสิทธิ์ RBAC ชุดเดียวกับบริการ API อื่น ๆ ผลลัพธ์คือ *token/session* ที่ใช้เข้าได้ในการเรียก API อื่น ๆ และข้อมูลสิทธิ์ที่ใช้ประกอบการอนุมัติการเข้าถึงทรัพยากร



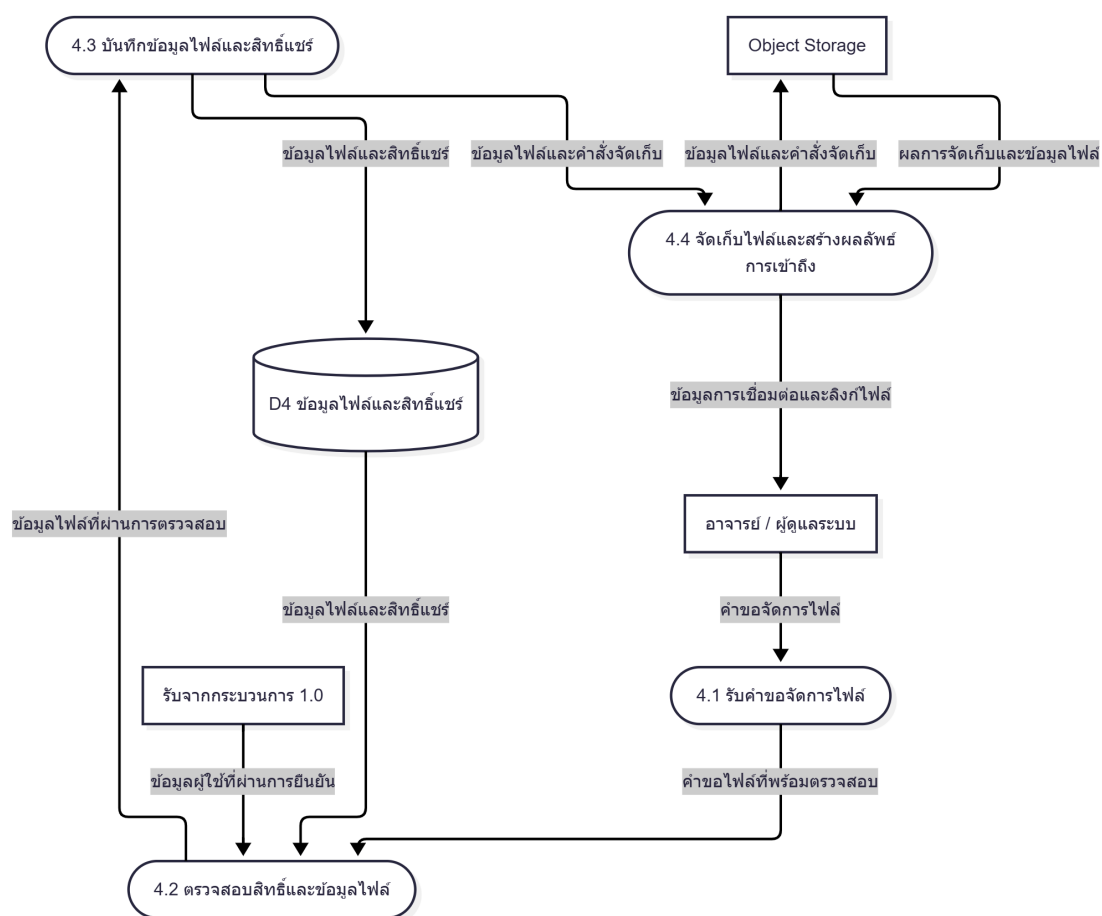
ภาพที่ 3-5 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 2.0 การจัดการคำร้องและข้อมูลวิชาการ

กระบวนการ 2.0 การจัดการคำร้องและข้อมูลวิชาการ (ภาพที่ 3-5) เน้นการจัดการข้อมูลเชิงโดเมนที่เกี่ยวข้องกับผู้ใช้และการเรียนการสอน เช่น รายวิชา/ภาคการศึกษา/ผู้สอน รวมถึงวงจรคำร้องขอเครื่องเสมือนและการตรวจสอบสิทธิ์ของผู้ยื่นคำร้อง จุดสำคัญคือข้อมูลคำร้องและประวัติการอนุมัติจะถูกจัดเก็บแบบเป็นโครงสร้างเพื่อให้สามารถตรวจสอบย้อนหลังได้ และสถานะคำร้องถูกส่งกลับไปยัง UI เพื่อสื่อสารความคืบหน้าให้ผู้ใช้



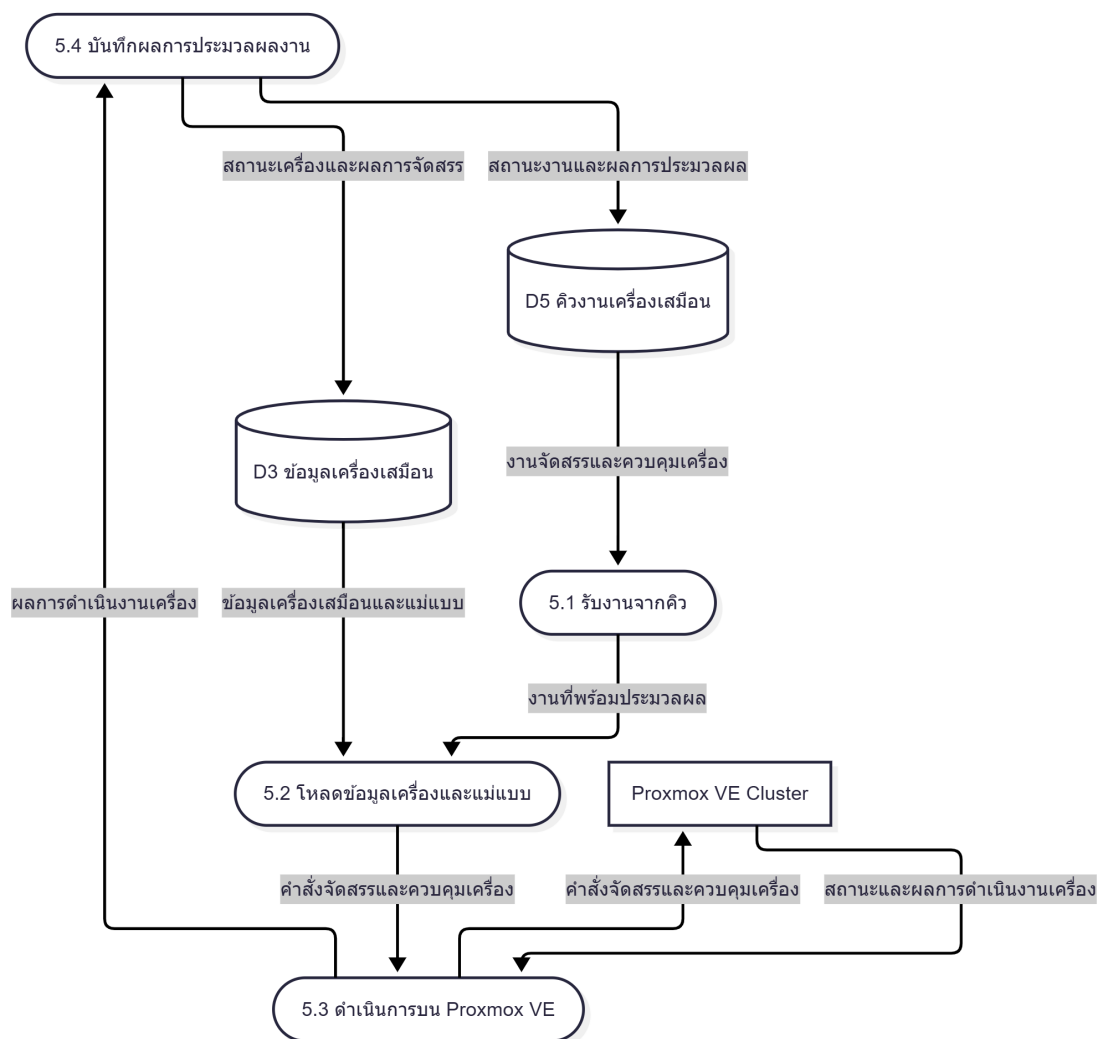
ภาพที่ 3-6 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 3.0 การจัดการข้อมูลเครื่องเสมือน

กระบวนการ 3.0 การจัดการข้อมูลเครื่องเสมือน (ภาพที่ 3-6) เป็นส่วนกลางของการจัดการ VM ในเชิงธุรกรรม โดยอ่าน/เขียนข้อมูล Instance และความสัมพันธ์กับทรัพยากร Proxmox VE ในฐานะข้อมูล รวมถึงการตัดสินใจว่างานใดควรประมวลผลแบบทันที (เช่นการอ่านสถานะจากฐานข้อมูล) และงานใดควรส่งไปประมวลผลแบบอะซิงโครนัสผ่านคิว (เช่นการสร้าง/ลบ/สั่งเริ่ม-หยุด VM) เพื่อให้ API ตอบสนองรวดเร็วและรองรับการ retry ได้อย่างปลอดภัย



ภาพที่ 3-7 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 4.0 การจัดการไฟล์และสิทธิ์การเข้าถึง

กระบวนการ 4.0 การจัดการไฟล์และสิทธิ์การเข้าถึง (ภาพที่ 3-7) แยกการจัดการ เมทาดาทา (เช่นชื่อไฟล์, owner, permission, เวลาอัปโหลด) ออกจาก ข้อมูลไฟล์จริง ที่อยู่ใน Object Storage โดยกระบวนการนี้จึงต้องจัดการทั้งสิทธิ์เข้าถึงและการออกลิงก์/โทเคนการเข้าถึงตามนโยบายของระบบ เพื่อให้การควบคุมสิทธิ์สอดคล้องกับบทบาทผู้ใช้และบริบทของรายวิชา/คำร้อง



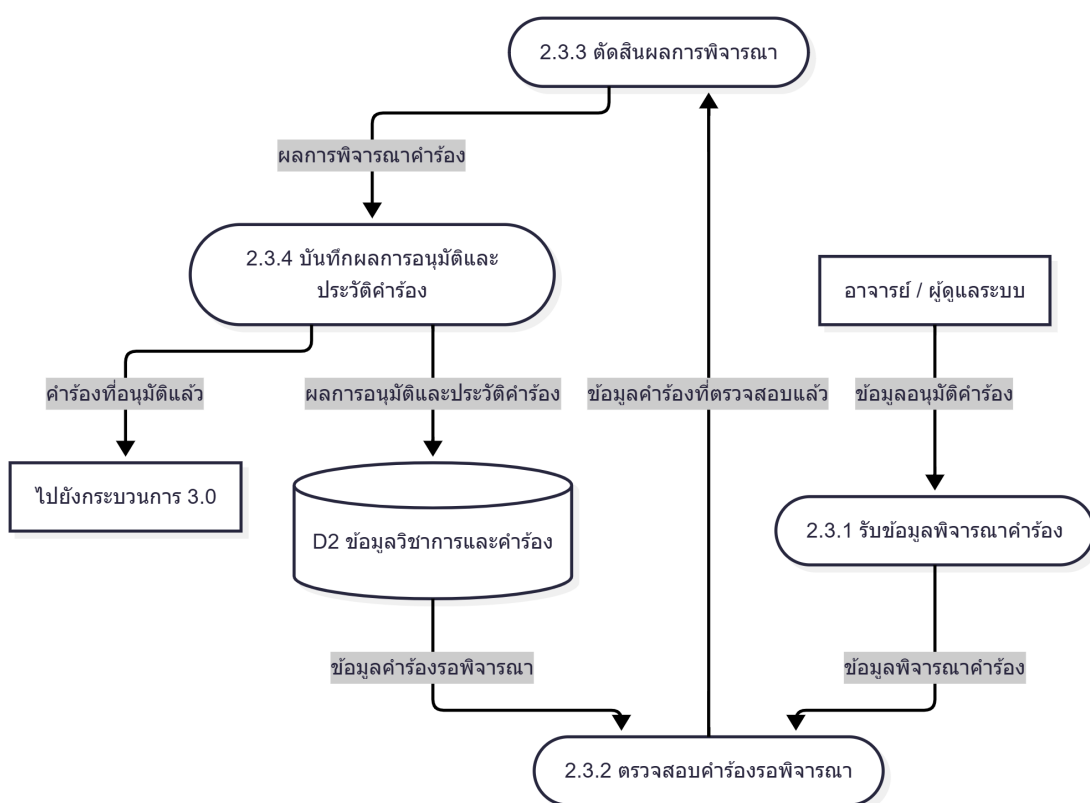
ภาพที่ 3-8 แผนภาพการไหลของข้อมูลระดับที่ 1 ของกระบวนการ 5.0 การประมวลผลงานจัดสรรเครื่องเสมือน

กระบวนการ 5.0 การประมวลผลงานจัดสรรเครื่องเสมือน (ภาพที่ 3-8) แสดงบทบาทของ worker (Yuzu) ในการรับงานจากคิวและเรียก Proxmox VE จริง โดยผลลัพธ์ที่ได้จาก Proxmox VE (สำเร็จ/ล้มเหลว/สถานะระหว่างทาง) จะถูกนำกลับมาอัปเดตฐานข้อมูล เพื่อให้ Midori และ Momoi แสดงสถานะล่าสุดแก่ผู้ใช้ได้จากแหล่งข้อมูลเดียวกัน (single source of truth) ใน implementation ปัจจุบัน Yuzu ยังรองรับ การ ประมวล ผล หลายงาน พร้อม กัน ภายใน หนึ่ง instance และ ใช้ กลไก distributed lock บน Redis เพื่อป้องกันไม่ให้ job ที่อ้างถึง ‘instancetype’ เดียวกันหรือ ‘VMID’ เดียวกันทำงานทับซ้อนกันจนเกิดปัญหา resource contention โดยไม่ปิดกั้นการทำงานพร้อมกันบน โหนดเดียวกันหากเป็นคนละเครื่อง

สำหรับการวิเคราะห์เชิงลึกเพิ่มเติม ได้จัดทำแผนภาพการไหลของข้อมูลระดับที่ 2 เพื่อขยายกระบวนการย่อยที่มีความซับซ้อน ได้แก่ การพิจารณาอนุมัติคำร้องในกระบวนการ 2.3 การรับคำร้องที่

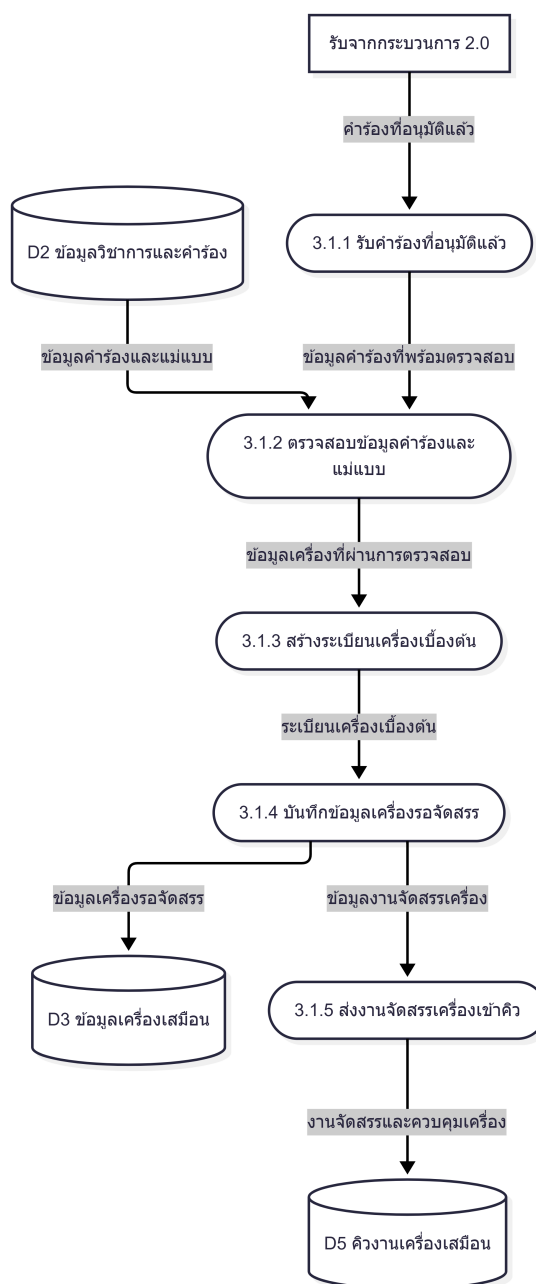
อนุมัติแล้วและจัดเตรียมงานในกระบวนการ 3.1 การส่งงานเข้าคิวในกระบวนการ 3.3 และการดำเนินการบน Proxmox VE ในกระบวนการ 5.3 ทั้งนี้การกำหนดชื่อกระแสข้อมูลในแต่ละระดับถูกจัดให้สอดคล้องกันตามหลัก Balancing ของ DFD ดังแสดงในภาพที่ 3-9 – 3-12

ในระดับ DFD Level 2 การขยายรายละเอียดมีเป้าหมายเพื่อให้เห็น **จุดตัดสินใจ (decision points)** และ **จุดที่ต้องรักษาความถูกต้องของข้อมูล (consistency points)** ของระบบอย่างชัดเจน โดยเฉพาะกระบวนการที่เกี่ยวข้องกับการอนุมัติ (ต้องตรวจสอบสิทธิ์และบันทึกประวัติ) และกระบวนการที่เกี่ยวข้องกับงานระยะยาว (ต้องส่งเข้าคิวและรองรับการ retry) ซึ่งสามารถอธิบายกลไกสำคัญได้ดังนี้



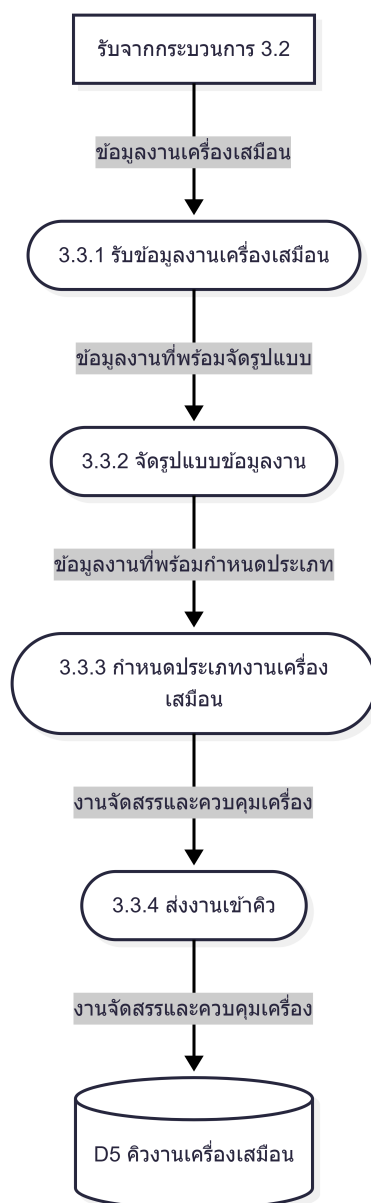
ภาพที่ 3-9 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 2.3 การพิจารณาอนุมัติคำร้อง

กระบวนการ 2.3 การพิจารณาอนุมัติคำร้อง (ภาพที่ 3-9) เริ่มจากผู้ดูแลระบบ/อาจารย์ส่งคำสั่งอนุมัติหรือปฏิเสธคำร้อง ระบบจะตรวจสอบสิทธิ์ของผู้ดำเนินการและสถานะคำร้องปัจจุบันก่อน (เพื่อป้องกันการอนุมัติซ้ำหรืออนุมัติคำร้องที่ถูกยกเลิกแล้ว) จากนั้นจึงอัปเดตสถานะคำร้องในฐานข้อมูล พร้อมบันทึกประวัติการตัดสินใจลงในตาราง audit log เพื่อใช้ตรวจสอบย้อนหลัง และส่งผลลัพธ์/สถานะใหม่กลับไปยัง UI



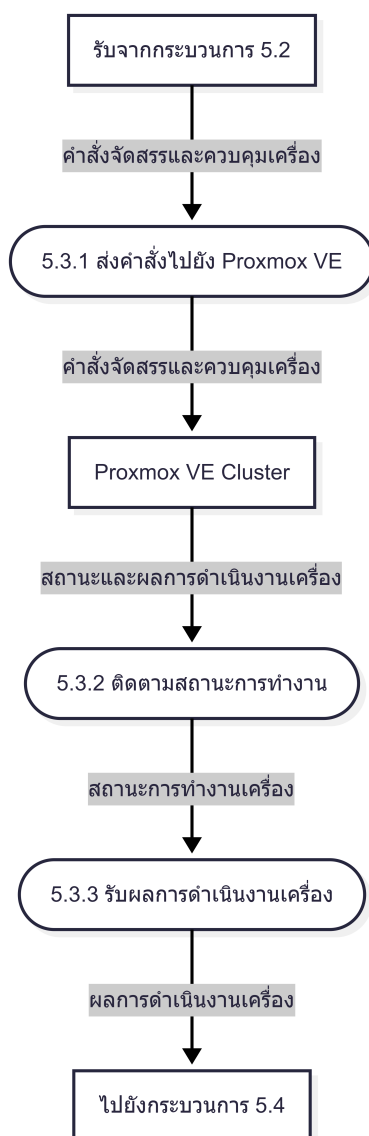
ภาพที่ 3-10 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 3.1 การรับคำสั่งที่อนุมัติแล้ว และจัดเตรียมงาน

กระบวนการ 3.1 การรับคำสั่งที่อนุมัติแล้วและจัดเตรียมงาน (ภาพที่ 3-10) เมื่อคำสั่งถูกอนุมัติ ระบบจะดึงข้อมูลที่จำเป็นจากฐานข้อมูล (เช่น รายละเอียดผู้ใช้ รายวิชา เทมเพลต VM โหนด/เครือข่ายที่เกี่ยวข้อง) เพื่อจัดเตรียม *แผนการดำเนินงาน* (provision plan) และสร้างระเบียบ Instance/งานที่เกี่ยวข้องในสถานะเริ่มต้น จุดนี้มักเน้นความสอดคล้องของข้อมูล เช่น ต้องผูกคำสั่งกับ instanceId ที่ชัดเจน และกำหนดสถานะเพื่อรองรับการติดตามความคืบหน้าในขั้นถัดไป



ภาพที่ 3-11 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 3.3 การส่งงานเข้าคิว

กระบวนการ 3.3 การส่งงานเข้าคิว (ภาพที่ 3-11) ระบบจะสร้าง payload ของงานโดยเก็บเฉพาะตัวระบุ (เช่น instanceId, userId, action) และข้อมูลที่จำเป็นขั้นต่ำ เพื่อหลีกเลี่ยงข้อมูลซ้ำซ้อนหรือข้อมูลล้าสมัยในคิว จากนั้นจึงส่งงานเข้าสู่ Redis-backed BullMQ พร้อมกำหนดชนิดงาน (เช่น provision/deprovision/toggle status) และอัปเดตสถานะงานในฐานข้อมูลเพื่อสะท้อนว่าอยู่ระหว่างรอประมวลผล วิธีนี้ช่วยให้การ retry มีความปลอดภัย เพราะ worker สามารถอ่านข้อมูลฉบับล่าสุดจากฐานข้อมูลก่อนลงมือปฏิบัติจริง



ภาพที่ 3-12 แผนภาพการไหลของข้อมูลระดับที่ 2 ของกระบวนการ 5.3 การดำเนินการบน Proxmox VE

กระบวนการ 5.3 การดำเนินการบน Proxmox VE (ภาพที่ 3-12) เริ่มจาก worker (Yuzu) รับงานจากคิวและโหนดรายละเอียด instance จากฐานข้อมูลเพื่อประกอบคำสั่ง จากนั้นเรียก Proxmox VE API เพื่อดำเนินการตามชนิดงาน (สร้าง/ลบ/เริ่ม/หยุด/รีสตาร์ท) พร้อมตรวจสอบผลลัพธ์และบันทึกสถานะงานเป็นช่วง ๆ หากเกิดข้อผิดพลาด worker จะบันทึกเหตุการณ์และสถานะล้มเหลวเพื่อให้ระบบแจ้งกลับผู้ใช้อย่างถูกต้อง และหากเป็นกรณีที่รองรับการ retry จะสามารถหยิบงานขึ้นมาทำซ้ำได้โดยอ้างอิงสถานะล่าสุดในฐานข้อมูลเพื่อหลีกเลี่ยงการสร้าง VM ซ้ำหรือเปลี่ยนสถานะผิดพลาด นอกจากนี้ก่อนเข้าสู่ช่วงงานสำคัญบางประเภท worker จะเข้าครอบครอง lock ของทรัพยากรที่เกี่ยวข้อง เช่น 'instanceId' ของคำร้องงานเดิม หรือ 'VMID' ของเครื่องปลายทาง เพื่อป้องกันไม่ให้

คำสั่งที่กระทบเครื่องเสมือนเดียวกันวิ่งพร้อมกันเกินความเหมาะสม ขณะเดียวกันการทำงานบนโหนด Proxmox เดียวกันยังสามารถเกิดขึ้นแบบขนานได้ หากเป็นคนละ ‘VMID’ และไม่ใช้งานที่ชนกับเครื่องเดิม

3.1.2 การออกแบบหน้าจอการทำงาน

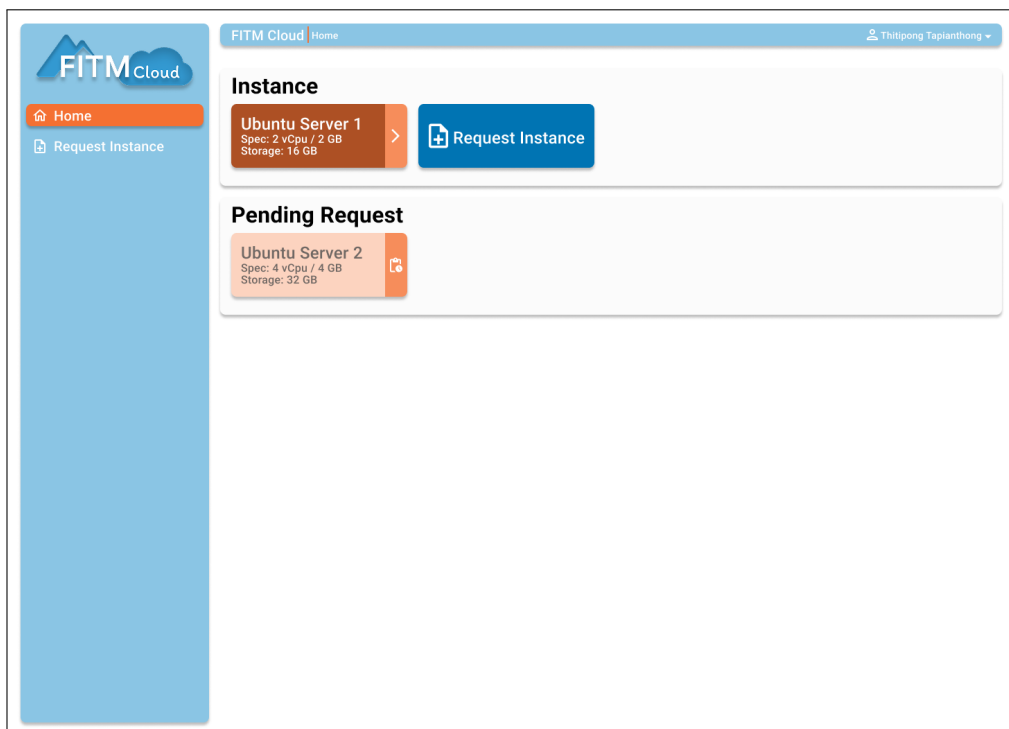
การออกแบบหน้าจอการทำงานของระบบแพลตฟอร์มคลัสเตอร์เสมือนได้รับการออกแบบให้มีความสะดวกในการใช้งานและการเข้าถึงข้อมูลที่จำเป็น โดยแบ่งออกเป็นหน้าจอหลัก ๆ ดังนี้

- **หน้าจอเข้าสู่ระบบ (Login Page)** หน้าจอสำหรับผู้ใช้ในการเข้าสู่ระบบผ่านการยืนยันตัวตน
- **หน้าจอแดชบอร์ด (Dashboard)** แสดงภาพรวมสถานะเครื่องเสมือน รายการคำร้องขอ และข้อมูลผู้ใช้
- **หน้าจอจัดการเครื่องเสมือน (Instance Management)** สำหรับผู้ใช้ในการดูรายละเอียดเครื่องเสมือน เช่น สถานะ การเชื่อมต่อ และการจัดการเครื่อง
- **หน้าจอคำร้องขอเครื่องเสมือน (Instance Request)** สำหรับผู้ใช้ในการส่งคำร้องขอการใช้งานเครื่องเสมือน
- **หน้าจอจัดการคำร้องขอ (Request Management)** สำหรับเจ้าหน้าที่ในการตรวจสอบและอนุมัติคำร้องขอเครื่องเสมือน

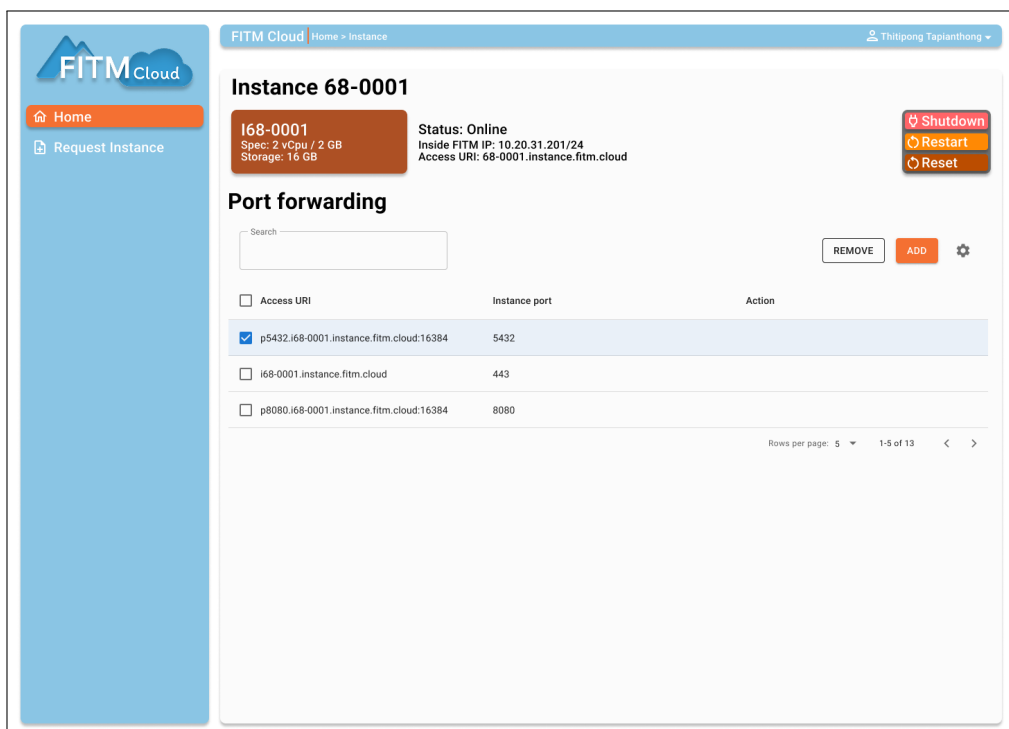
โดยมีการออกแบบหน้าจอหลัก ๆ ดังแสดงในภาพที่ 3-13 – 3-17



ภาพที่ 3-13 หน้าจอเข้าสู่ระบบ (Login Page)



ภาพที่ 3-14 หน้าจอแดชบอร์ด (Dashboard)



ภาพที่ 3-15 หน้าจอจัดการเครื่องเสมือน (Instance Management)

FITM Cloud Home > Request Instance Thitipong Tapianthong

Request Instance

Subject

Details
It is a long established fact that a reader will be distracted

Request type ☐ Standard ☐ Project / Thesis

Select Instance

Instance OS Value Instance Preset Value

Create Request

ภาพที่ 3-16 หน้าจอคำร้องขอเครื่องเสมือน (Instance Request)

FITM Cloud Home > Manage Request Thitipong Tapianthong

Manage Request

Search Student ID, Subject Type -

REJECT APPROVE

Request ID	Student ID	Subject	Type	Action
☒ 68-0001	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0002	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0003	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0004	650602xxxxxx	Working in thesis 681xxx	Project / Thesis	Cell
☐ 68-0005	680602xxxxxx	Study in subject 0602xx101	Standard	Cell

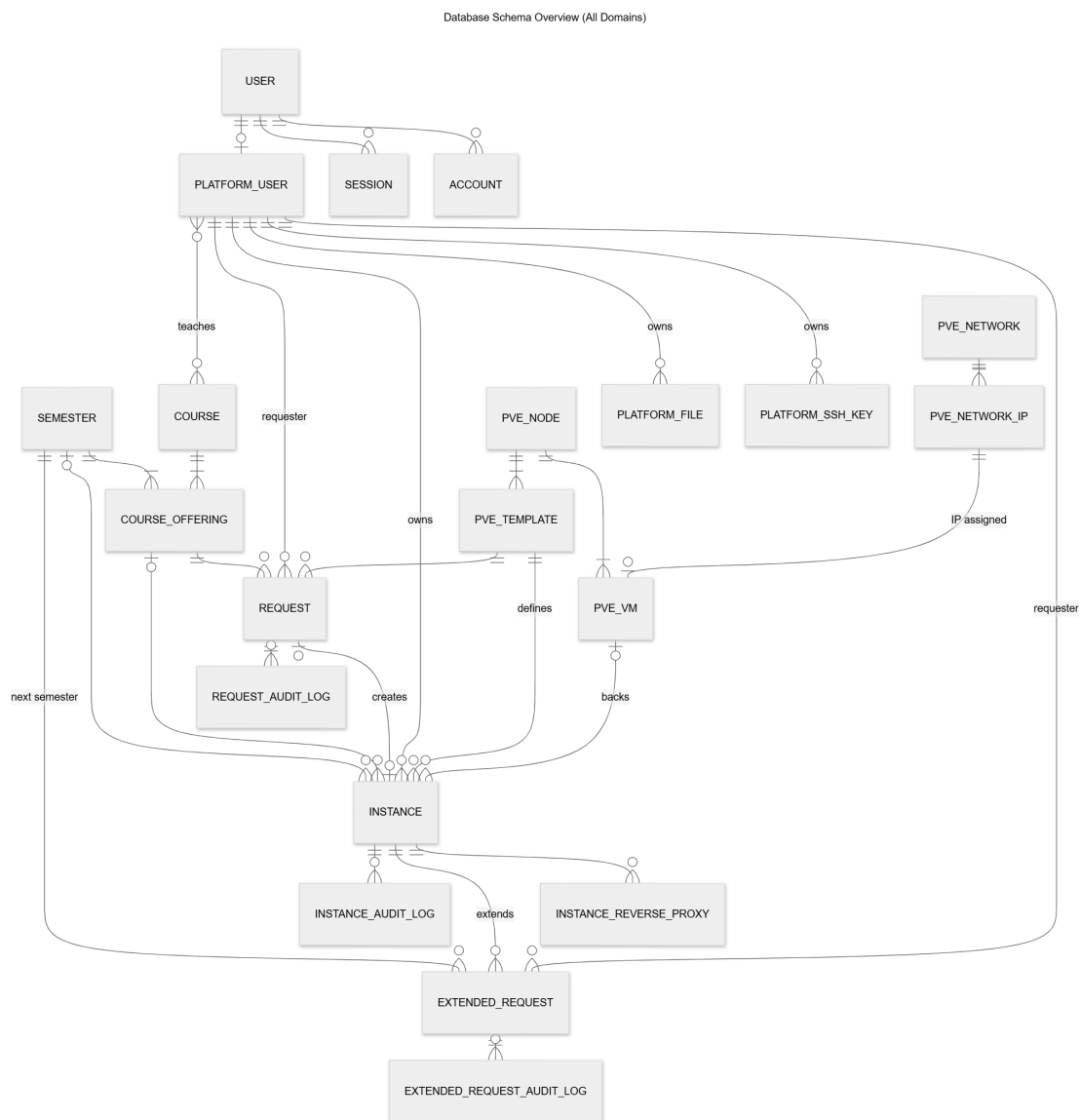
Rows per page: 5 1-5 of 13

ภาพที่ 3-17 หน้าจอจัดการคำร้องขอ (Request Management)

3.1.3 การออกแบบฐานข้อมูล

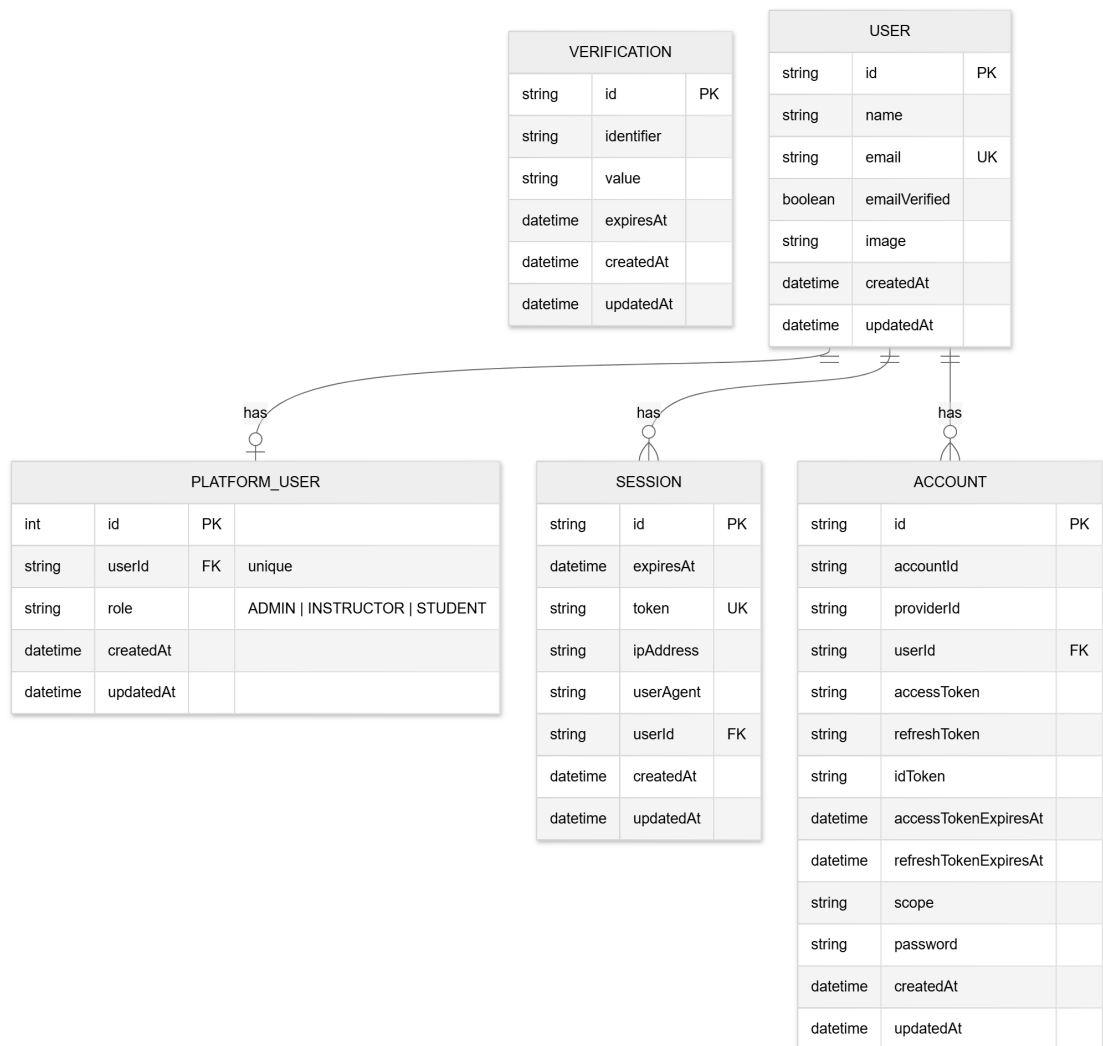
ระบบใช้ PostgreSQL เป็นฐานข้อมูลหลัก และใช้ Prisma ORM ในการกำหนดสคีมาและสร้างชั้นเข้าถึงข้อมูล โดยในฉบับพัฒนาปัจจุบันโครงสร้างฐานข้อมูลสามารถสรุปเป็นกลุ่มข้อมูลหลักดังนี้

- **Authentication and Role Management** เช่น User, Session, Account, Verification และ PlatformUser สำหรับจัดการตัวตนผู้ใช้งาน เซสชัน และบทบาท ADMIN, INSTRUCTOR, STUDENT
- **Academic Structure** เช่น Course, Semester, CourseOffering และ InstructorSearch สำหรับเชื่อมโยงรายวิชา ภาคการศึกษา และผู้สอน
- **Virtualization Resources** เช่น PVENode, PVENetwork, PVENetworkIP, PVETemplate และ PVEVM สำหรับสะท้อนทรัพยากรที่เกี่ยวข้องกับ Proxmox VE
- **Request Workflow** เช่น Request, ExtendedRequest, RequestAuditLog และ ExtendedRequestAuditLog สำหรับคำร้องขอเครื่องเสมือน การขยายสิทธิ์การใช้งาน และประวัติการอนุมัติ
- **Instance Management** เช่น Instance, InstanceReverseProxy, InstanceAuditLog และ PlatformSSHKey สำหรับข้อมูลเครื่องเสมือน ช่องทางการเชื่อมต่อ และบันทึกการเปลี่ยนแปลง
- **Platform Storage** เช่น PlatformFile สำหรับการจัดเก็บไฟล์



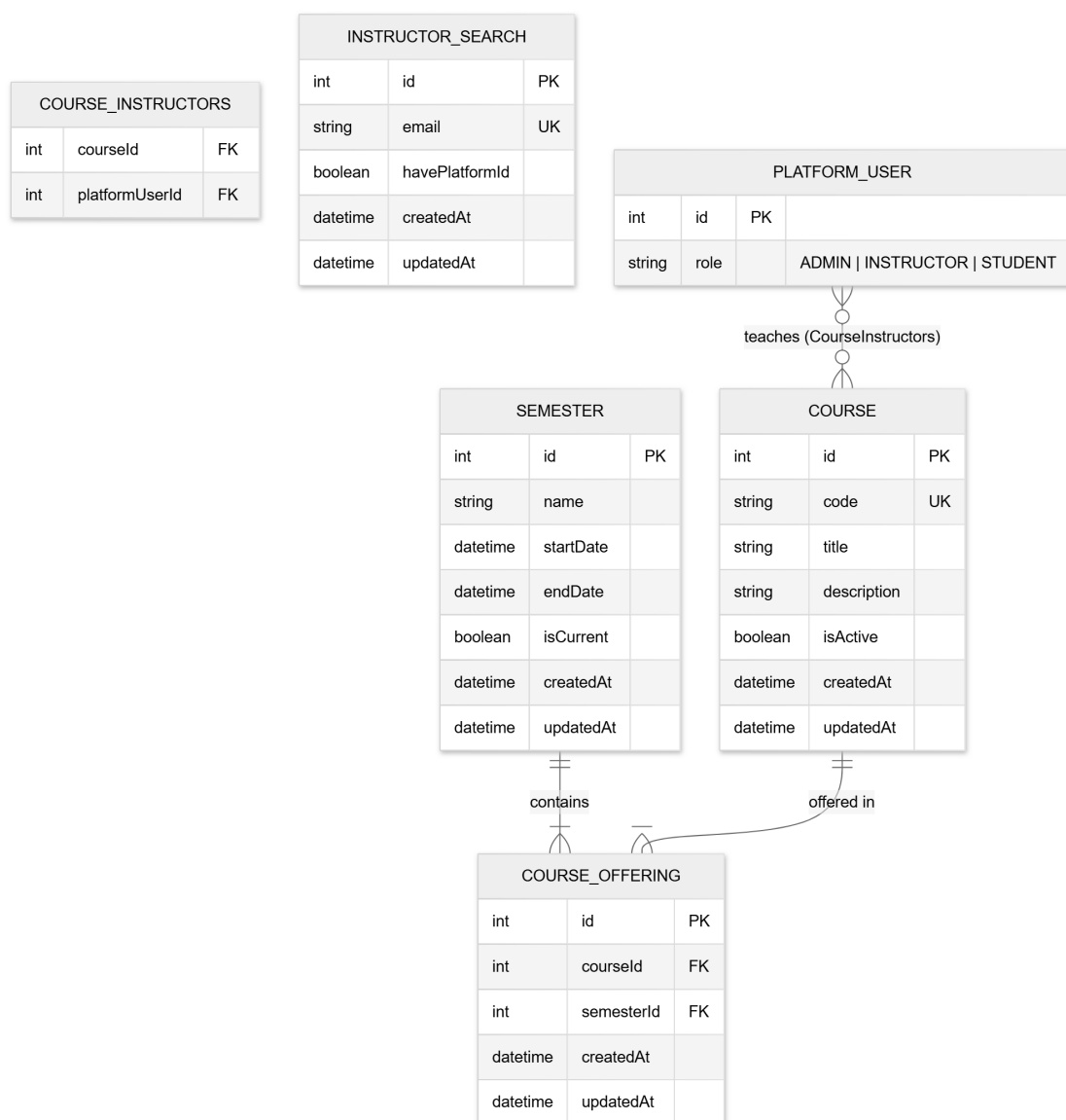
ภาพที่ 3-18 แผนภาพโครงสร้างฐานข้อมูลของระบบภาพรวม (ER Diagram Overview)

Authentication & User Domain



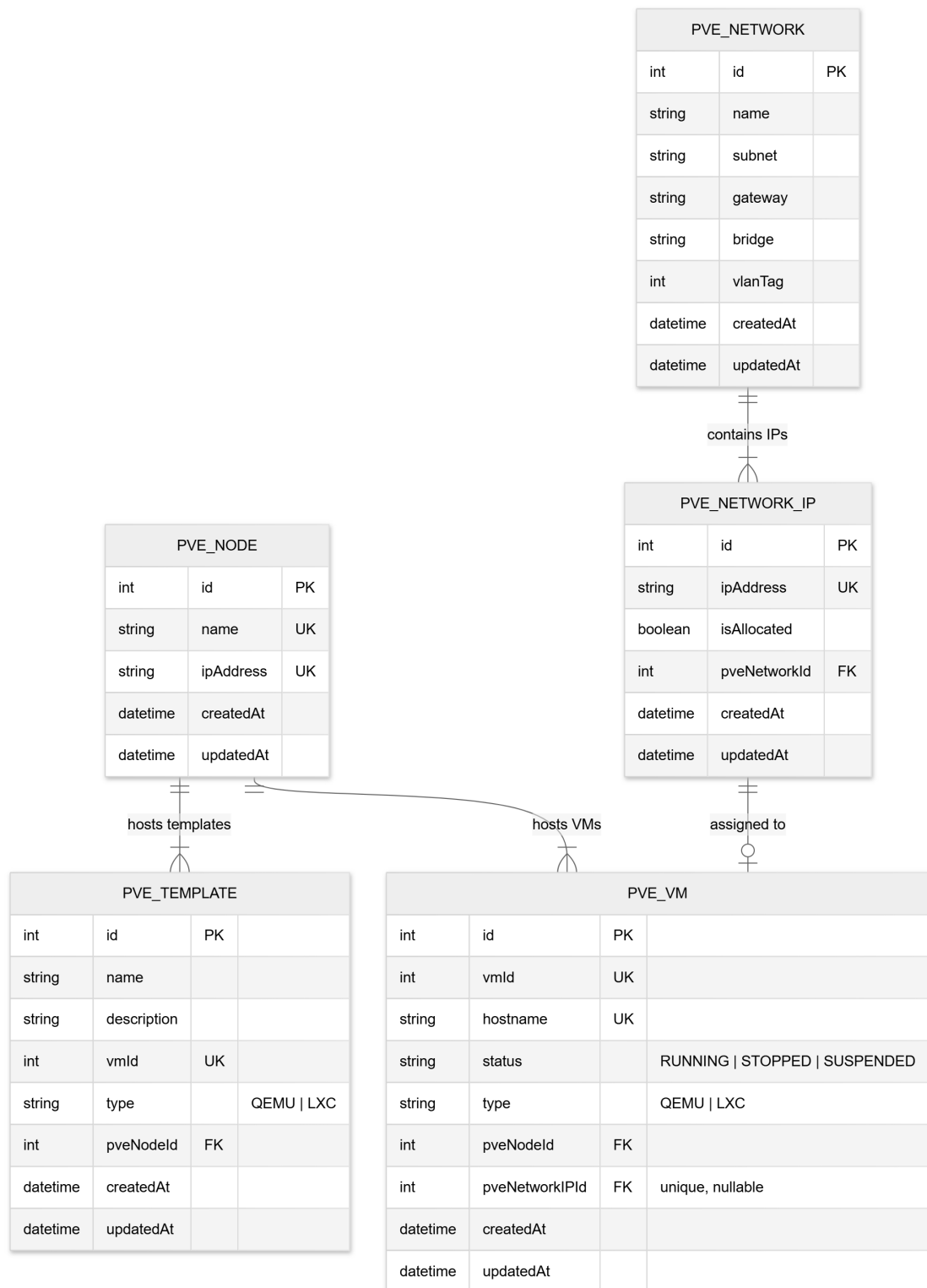
ภาพที่ 3-19 แผนภาพ ER กลุ่มข้อมูลการยืนยันตัวตนและผู้ใช้งาน

Academic Domain

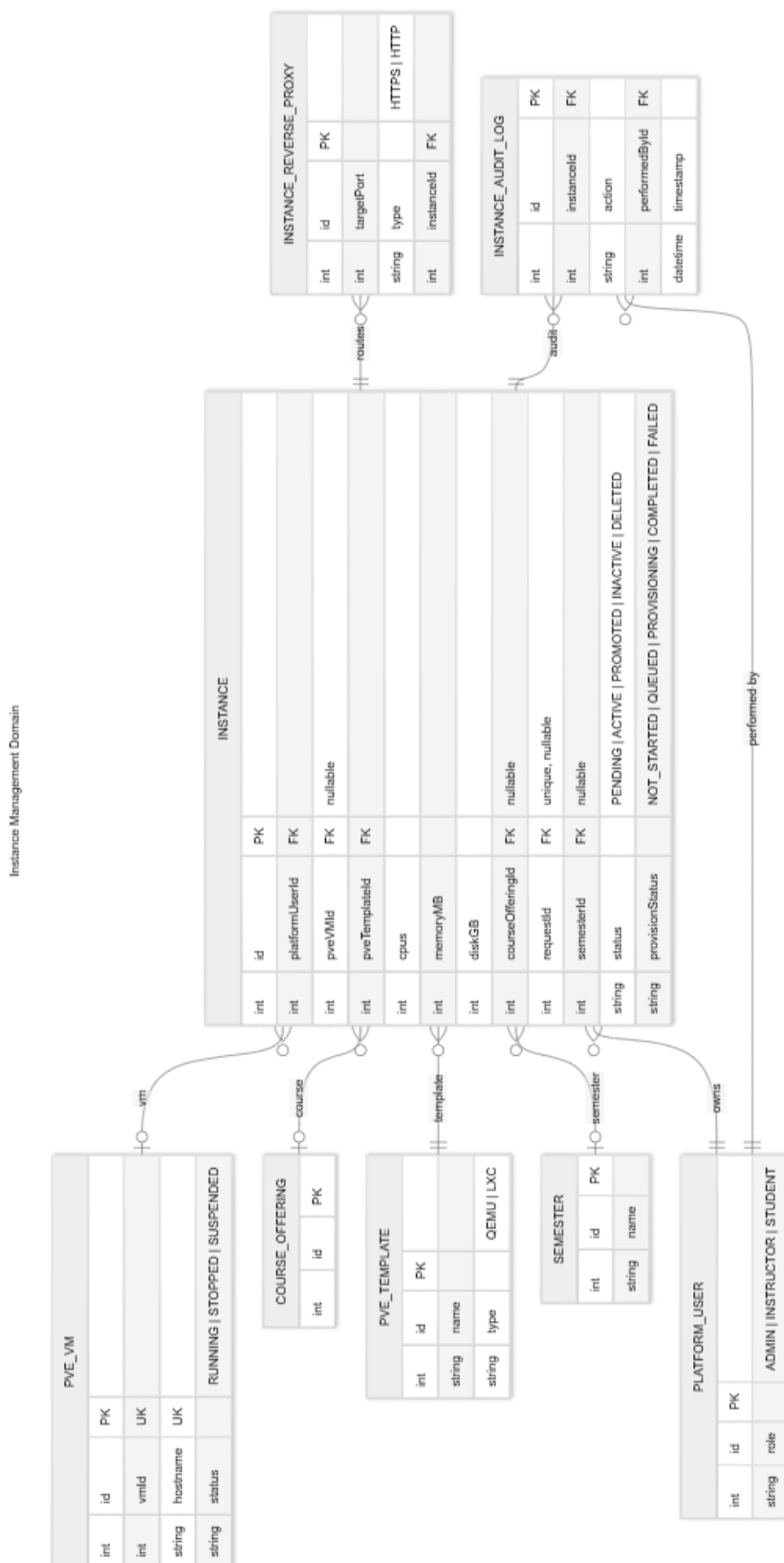


ภาพที่ 3-20 แผนภาพ ER กลุ่มข้อมูลโครงสร้างวิชาการ

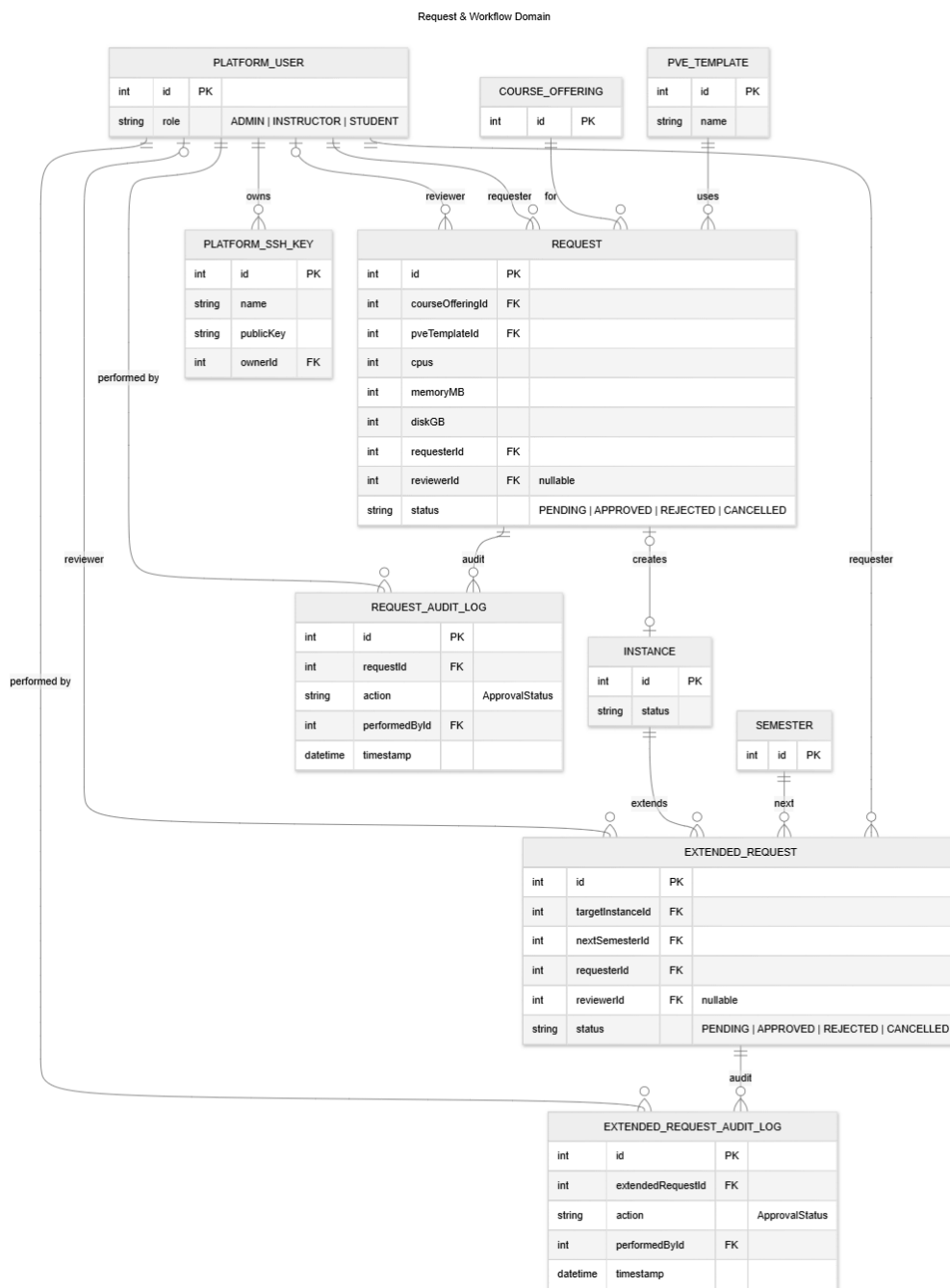
Proxmox VE Infrastructure Domain



ภาพที่ 3-21 แผนภาพ ER กลุ่มข้อมูลโครงสร้างพื้นฐาน Proxmox VE

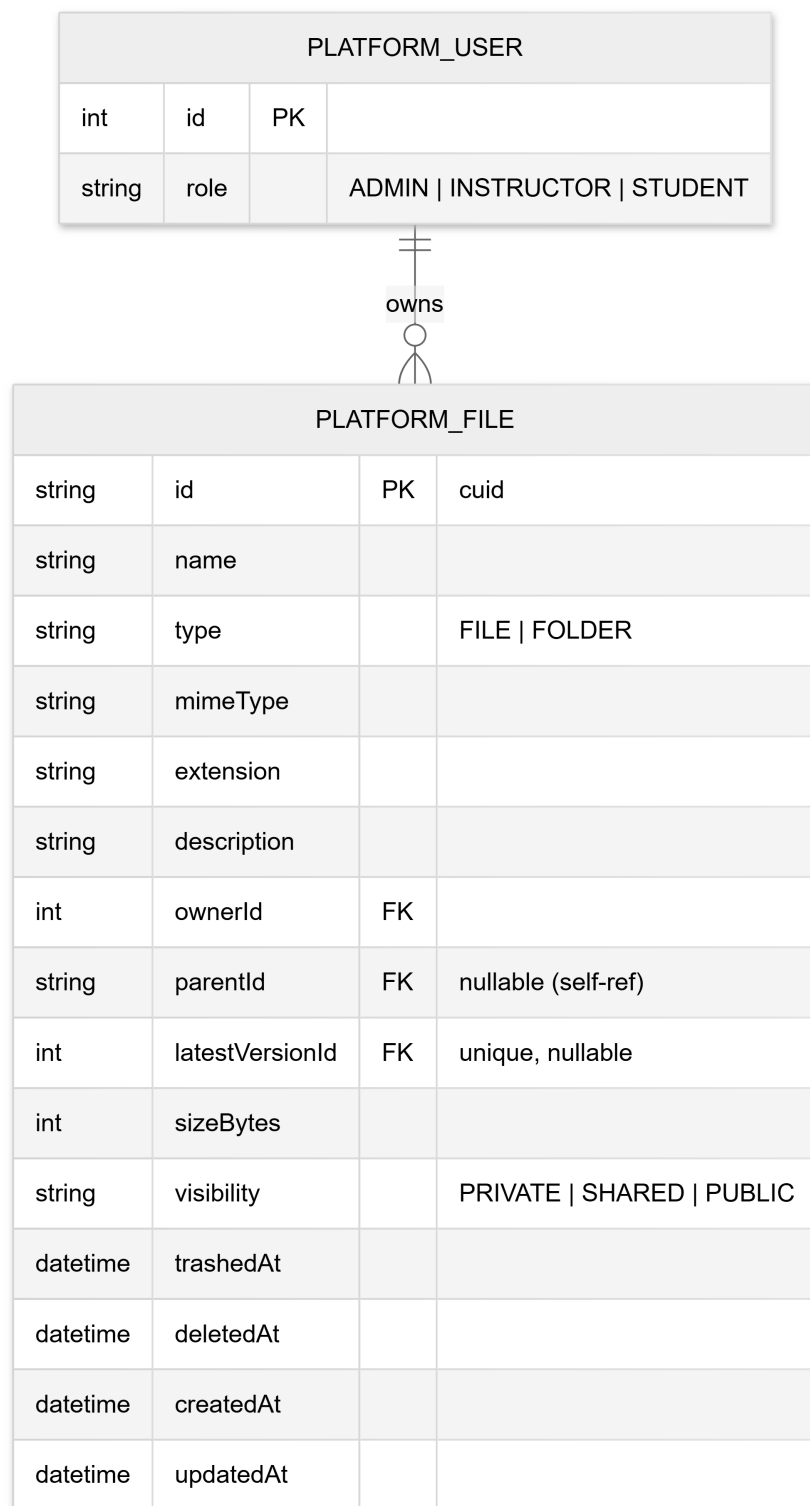


ภาพที่ 3-22 แผนภาพ ER กลุ่มข้อมูลการจัดการเครื่องเสมือน



ภาพที่ 3-23 แผนภาพ ER กลุ่มข้อมูลคำร้องขอและกระบวนการอนุมัติ

Platform File Storage Domain



ภาพที่ 3-24 แผนภาพ ER กลุ่มข้อมูลพื้นที่จัดเก็บไฟล์

จากภาพที่ 3-18 – 3-24 จะเห็นได้ว่าการออกแบบฐานข้อมูลไม่ได้ครอบคลุมเฉพาะข้อมูลผู้ใช้และเครื่องเสมือนเท่านั้น แต่ยังรวมถึงโครงสร้างทางวิชาการ ประวัติการอนุมัติ การจัดการ reverse proxy และระบบจัดเก็บไฟล์ภายในแพลตฟอร์มด้วย ทั้งนี้ได้แบ่งแผนภาพเป็น 6 กลุ่มตามโดเมนข้อมูล ได้แก่ การยืนยันตัวตน โครงสร้างวิชาการ โครงสร้างพื้นฐาน Proxmox VE การจัดการเครื่องเสมือน กระบวนการคำร้องขอ และพื้นที่จัดเก็บไฟล์ การออกแบบในลักษณะนี้ทำให้ Momoi สามารถทำหน้าที่เป็นศูนย์กลางของข้อมูลธุรกรรม ขณะที่ Yuzu ใช้ข้อมูลเดียวกันในการอ้างอิงระหว่างการผลิตผลคำสั่งบน Proxmox VE

ตารางที่ 3-1 สรุปลักษณะข้อมูลหลักจาก Prisma Schema ฉบับปัจจุบัน

กลุ่มข้อมูล	เอนทิตีหลัก
Authentication และ RBAC	User, Session, Account, Verification, PlatformUser
โครงสร้างวิชาการ	Course, Semester, CourseOffering, InstructorSearch
ทรัพยากร Proxmox VE	PVNode, PVNetwork, PVNetworkIP, PVTemplate, PVEVM
กระบวนการคำร้องขอ	Request, ExtendedRequest, RequestAuditLog, ExtendedRequestAuditLog
การจัดการเครื่องเสมือน	Instance, InstanceReverseProxy, InstanceAuditLog, PlatformSSHKey
พื้นที่จัดเก็บไฟล์	PlatformFile

ตารางที่ 3-2 รายการ Enum สำคัญในสคีมารฐานข้อมูลฉบับปัจจุบัน

ชื่อ Enum	ค่า
PlatformRole	ADMIN, INSTRUCTOR, STUDENT
PVEVMStatus	RUNNING, STOPPED, SUSPENDED
PVEVMType	QEMU, LXC
InstanceStatus	PENDING, ACTIVE, PROMOTED, INACTIVE, DELETED
InstanceProvisionStatus	NOT_STARTED, QUEUED, PROVISIONING, COMPLETED, FAILED
ReverseProxyType	HTTPS, HTTP
ApprovalStatus	PENDING, APPROVED, REJECTED, CANCELLED

3.1.4 การออกแบบ API และการสื่อสาร

ระบบใช้ RESTful API เป็นหลักในการสื่อสารระหว่างส่วนประกอบ โดยมีการออกแบบดังนี้

- **Authentication Endpoints** ให้บริการผ่านเส้นทาง ‘/api/auth’ โดย Momoi เป็นผู้ดูแลการยืนยันตัวตน Session และ OAuth callback ขณะที่ Midori จะใช้งานเส้นทางดังกล่าวผ่าน better-auth client
- **Platform API Endpoints** ให้บริการโดย Momoi ครอบคลุมโดเมนหลัก เช่น Academic Management, Request Workflow, Instance Management, Reverse Proxy Management และ Storage Management
- **Documentation and Telemetry Endpoints** ใช้สำหรับสร้าง OpenAPI Documentation, Metrics และ Traces เพื่อสนับสนุนการทดสอบและการติดตามสถานะของระบบ โดยกระจายอยู่ตามบทบาทของแต่ละบริการ

ตารางที่ 3-3 สรุปกลุ่ม API หลักของ Momoi พร้อมระดับการยืนยันตัวตนที่ต้องการ โดย Momoi เปิดเผย API ด้านแพลตฟอร์มเหล่านี้ผ่าน Elysia.js ซึ่งสร้าง OpenAPI Specification โดยอัตโนมัติ ทำให้ Midori สามารถนำข้อมูล Schema ไปสร้างชนิดข้อมูลฝั่ง Client ได้โดยตรงโดยไม่ต้องเขียนซ้ำ

ตารางที่ 3-3 สรุปกลุ่ม API หลักของบริการ Momoi

กลุ่ม API	คำอธิบาย
Public	ข้อมูลสาธารณะ เช่น ภาคการศึกษาปัจจุบันและที่กำลังจะมาถึง
Auth	Login, Logout, Session, OAuth callback
Academic	รายวิชา ภาคการศึกษา ผู้สอน และการเปิดสอน
Request	สร้างและจัดการคำร้องขอเครื่องเสมือน ทั้ง Request และ ExtendedRequest
Instance	เครื่องเสมือน Reverse Proxy Audit Log และ SSH Key
Storage	ไฟล์ โฟลเดอร์ การแบ่งปัน และสิทธิ์การเข้าถึง
Admin	จัดการ PVE Node PVE Template PVE Network และ Platform User

การยืนยันตัวตนของระบบออกแบบให้ใช้ Session ผ่านชุด endpoint ‘/api/auth’ ซึ่งใช้ ‘better-auth’ และข้อมูลผู้ใช้ในฐานะข้อมูลร่วมกัน จากนั้นเมื่อ Midori เรียก API ของ Momoi ต่อไป Momoi จะตรวจสอบ session และดึงข้อมูล PlatformUser จากฐานข้อมูลเพื่อกำหนดบทบาท (ADMIN, INSTRUCTOR, STUDENT) ก่อนอนุมัติคำขอ แนวทางนี้ทำให้การยืนยันตัวตนและการกำหนดสิทธิ์เชื่อมต่อกับ business logic ของแพลตฟอร์มได้โดยตรง และลดความซับซ้อนของ topology ที่ต้องติดตามในเอกสารและการติดตั้งใช้งาน

3.1.5 การออกแบบระบบ Message Queue

ระบบรองรับการประมวลผลแบบ Asynchronous โดยใช้ Redis ร่วมกับ BullMQ เป็นกลไก Message Queue ระหว่าง Momoi และ Yuzu คิวหลักในฉบับพัฒนาปัจจุบันประกอบด้วย

- **Provision Queue** สำหรับงานสร้างและจัดสรรเครื่องเสมือนใหม่
- **Deprovision Queue** สำหรับงานลบหรือคืนทรัพยากรของเครื่องเสมือน
- **Toggle Status Queue** สำหรับงานเปลี่ยนสถานะของเครื่อง เช่น Start, Stop และ Restart

ในระดับการติดตั้งใช้งานจริง คิวเหล่านี้จะถูกตั้งชื่อแบบผูกกับ environment นำหน้าชื่อคิวเพื่อแยกงานของแต่ละสภาพแวดล้อมออกจากกันอย่างชัดเจน

ข้อมูลที่ถูกส่งเข้าแต่ละงานจะเก็บเฉพาะตัวระบุที่จำเป็น เช่น ‘instanceId’, ‘userId’ และ action ที่ต้องการเปลี่ยนแปลง จากนั้น Yuzu จะอ่านข้อมูลฉบับเต็มจากฐานข้อมูลก่อนดำเนินการบน Proxmox VE วิธีการนี้ช่วยลดความซ้ำซ้อนของข้อมูลในคิว ลดโอกาสใช้ข้อมูลที่ล้าสมัย และทำให้การ retry งานมีความปลอดภัยมากขึ้น

ระบบกำหนดนโยบายการทำงาน (retry policy) เพื่อรองรับความล้มเหลวชั่วคราว เช่น การเชื่อมต่อ Proxmox VE ชัดข้อง หรือ VM อยู่ระหว่างเปลี่ยนสถานะ โดยงานที่ล้มเหลวจะถูก retry ตามจำนวนครั้งที่กำหนด พร้อมการหน่วงเวลาแบบ exponential backoff ระหว่างครั้ง หากยังล้มเหลวเกินจำนวนที่กำหนดจะถูกย้ายเข้า failed job queue เพื่อให้ผู้ดูแลระบบตรวจสอบในภายหลัง ในฉบับพัฒนาปัจจุบัน worker ของแต่ละคิวยังสามารถตั้งค่า concurrency แยกกันผ่าน environment variables ทำให้ปรับระดับการประมวลผลพร้อมกันให้สอดคล้องกับสภาพแวดล้อมพัฒนา ทดสอบ หรือ production ได้ โดยยังใช้ distributed lock บน Redis เพื่อกันไม่ให้ job ที่ชน ‘instanceId’ เดียวกันหรืออ้างอิง ‘VMID’ เดียวกันทำงานพร้อมกัน ขณะเดียวกันงานที่ใช้โหนดเดียวกันแต่เป็นคนละ VM ยังสามารถทำงานแบบขนานได้ตามปกติ ตารางที่ 3-4 สรุปคิวงานหลักพร้อมข้อมูลที่ส่งพร้อมแต่ละงานและผลลัพธ์ที่คาดหวัง

ตารางที่ 3-4 สรุปคิวงานหลักและโครงสร้างข้อมูลงาน

ชื่อคิว	ข้อมูลหลักในงาน	ผลลัพธ์ที่คาดหวัง
Provision	instanceId, userId	VM ถูกสร้างและพร้อมใช้งาน สถานะ Instance อัปเดตเป็น ACTIVE
Deprovision	instanceId, userId	VM ถูกลบออกจากระบบ สถานะ Instance อัปเดตเป็น DELETED
Toggle Status	instanceId, userId, action (START STOP RESTART)	สถานะ VM และ Instance ถูกอัปเดตให้สอดคล้องกับการสั่ง Start Stop หรือ Restart

3.2 เครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา

3.2.1 เทคโนโลยี Frontend

Next.js 16 และ React 19 เป็นเทคโนโลยีหลักของส่วน Frontend เนื่องจากมีคุณสมบัติที่เหมาะสมกับการพัฒนาเว็บแอปพลิเคชันเชิงระบบดังนี้

- **App Router** สำหรับการจัดการ Routing แบบใหม่
- **Built-in TypeScript Support** รองรับ Type Safety
- **Integration with API Proxy and OpenAPI Types** ช่วยให้ส่วน Frontend สามารถเชื่อมต่อกับ Momoi ได้อย่างมีชนิดข้อมูลที่ชัดเจนและลดความซ้ำซ้อนของ schema ระหว่าง frontend กับ backend

Tailwind CSS และชุดคอมโพเนนต์บน Radix UI ใช้สำหรับการจัดการส่วนติดต่อผู้ใช้ เนื่องจาก

- **Utility-first** เขียน CSS ได้เร็วและมีประสิทธิภาพ
- **Responsive Design** รองรับการแสดงผลบนอุปกรณ์ต่าง ๆ
- **Composable UI Primitives** ช่วยให้สร้างฟอร์ม แดชบอร์ด และกล่องโต้ตอบที่ปรับใช้ซ้ำได้ง่าย

3.2.2 เทคโนโลยี Backend

Elysia.js เป็น Web Framework ที่ใช้กับ Bun Runtime โดยมีเหตุผลในการเลือกใช้ดังนี้

- **High Performance** มีประสิทธิภาพสูงกว่า Node.js Framework ทั่วไป
- **Type Safety** รองรับ TypeScript แบบเต็มรูปแบบ
- **OpenAPI Integration** สร้าง API Documentation อัตโนมัติ
- **Observability Integration** รองรับการเชื่อมต่อกับ OpenTelemetry และการเก็บข้อมูลด้านประสิทธิภาพของบริการ

Prisma ORM ใช้สำหรับการจัดการฐานข้อมูล เนื่องจาก

- **Type-safe Database Client** ป้องกันข้อผิดพลาดในการเขียน Query
- **Database Migrations** จัดการการเปลี่ยนแปลง Schema อย่างเป็นระบบ
- **Centralized Domain Model** ช่วยให้สคีมาของข้อมูลด้านผู้ใช้ คำร้อง เครื่องเสมือน และพื้นที่จัดเก็บอยู่ในรูปแบบเดียวกัน

3.2.3 Infrastructure และ DevOps

Docker ใช้สำหรับการ Containerization ของแอปพลิเคชันและบริการสนับสนุน เพื่อ

- **Environment Consistency** สภาพแวดล้อมการพัฒนาและ Production เหมือนกัน
- **Easy Deployment** ง่ายต่อการ Deploy และ Scale
- **Resource Isolation** แยกทรัพยากรระหว่างแอปพลิเคชัน

Docker Compose และ Kubernetes Manifests ใช้สำหรับการจัดการ Multi-container Application และสภาพแวดล้อมการติดตั้งจริง

- **Service Orchestration** จัดการการเริ่มต้นและหยุดทำงานของ Service
- **Network Management** จัดการเครือข่ายระหว่าง Container
- **Volume Management** จัดการการแชร์ข้อมูลระหว่าง Container
- **Platform Deployment** กำหนดการติดตั้งบริการหลัก เช่น PostgreSQL, Redis, RustFS และชุดระบบสังเกตการณ์การทำงาน

Observability Stack ถูกนำมาใช้เพื่อวัดผลและติดตามสถานะของระบบอย่างต่อเนื่อง

- **Prometheus** สำหรับเก็บ Metrics จากแต่ละบริการ
- **Grafana** สำหรับแสดง Dashboard และภาพรวมของระบบ
- **Loki และ Jaeger** สำหรับเก็บ Logs และ Traces
- **InfluxDB และ Exporters** สำหรับรองรับข้อมูลเชิงสังเกตการณ์เพิ่มเติมในระดับโครงสร้างพื้นฐาน

3.2.4 เครื่องมือการพัฒนา

Version Control และ Collaboration

- **Git** สำหรับการควบคุมเวอร์ชันของโค้ด
- **GitHub** สำหรับการเก็บโค้ดและการทำงานร่วมกัน
- **GitHub Actions** สำหรับ CI/CD Pipeline

Code Quality และ Testing

- **TypeScript-first Development** สำหรับ Static Type Checking ตลอดทุกบริการ
- **OpenAPI Code Generation** สำหรับสร้างชนิดข้อมูลและลดความคลาดเคลื่อนระหว่าง Frontend กับ Backend
- **Repository-specific Linting and Formatting** ใช้เครื่องมือที่เหมาะสมกับแต่ละบริการ เพื่อควบคุมคุณภาพของโค้ดก่อนนำขึ้นใช้งาน

บทที่ 4

ผลการดำเนินงาน

ในการพัฒนาระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ ได้ดำเนินการออกแบบและพัฒนาระบบที่ประกอบด้วย 3 บริการซอฟต์แวร์หลัก คือ ส่วนติดต่อผู้ใช้ (Frontend) ส่วนบริการ API (API Service) และส่วนประมวลผลงานจัดการเครื่องเสมือน (VM Operations Service) ร่วมกับบริการสนับสนุนสำหรับการยืนยันตัวตน ฐานข้อมูล ระบบคิวงาน พื้นที่จัดเก็บไฟล์ และระบบติดตามการทำงาน ในบทนี้จึงนำเสนอผลการดำเนินงานในมุมมองของโครงสร้างรีโพสิทอรี องค์ประกอบของแต่ละบริการ เทคโนโลยีที่ใช้ เอกสารประกอบการใช้งาน และตัวอย่างหน้าจอของระบบที่พัฒนาขึ้นจริง โดยยึดตามสภาพแวดล้อมของโค้ดปัจจุบันซึ่งแยกเป็นหลายรีโพสิทอรีและติดตั้งใช้งานร่วมกันในลักษณะ Microservices Architecture

4.1 โครงสร้างโครงการหลัก

จากการออกแบบและพัฒนาระบบแพลตฟอร์มคลาวด์เพื่อสนับสนุนกิจกรรมการเรียนการสอนและงานวิชาการ โครงการฉบับปัจจุบันไม่ได้จัดเก็บแบบ Monorepo แต่แบ่งออกเป็นหลายรีโพสิทอรีที่ทำงานร่วมกัน ได้แก่ Midori, Momoi, Yuzu, Satsuki และ Yuuka เพื่อให้สามารถพัฒนา ทดสอบ และติดตั้งใช้งานแต่ละส่วนได้อย่างเป็นอิสระ โดยในเชิงหน้าที่สามารถมองได้ว่า Midori, Momoi และ Yuzu เป็นบริการหลักเชิงธุรกิจ ขณะที่ Satsuki และ Yuuka เป็นส่วนสนับสนุนด้าน reverse proxy หน้าด่าน การเชื่อมต่อโครงสร้างพื้นฐาน และการติดตั้งใช้งานจริง

4.1.1 รีโพสิทอรี Midori

รีโพสิทอรี Midori เป็นส่วนติดต่อผู้ใช้ของระบบ พัฒนาด้วย Next.js และ React ทำหน้าที่นำเสนอหน้าเว็บสำหรับนักศึกษา อาจารย์ และผู้ดูแลระบบ ภายในรีโพสิทอรีนี้มีโครงสร้างสำคัญ เช่น โฟลเดอร์ src/app สำหรับหน้าและ route หลักของระบบ, src/components สำหรับองค์ประกอบส่วนติดต่อผู้ใช้, src/hooks และ src/lib สำหรับตรรกะร่วม, รวมถึงไฟล์กำหนดค่าอย่าง package.json, next.config.ts และ components.json

- 1) **Landing และ Login Page** สำหรับการเข้าถึงระบบและเข้าสู่กระบวนการยืนยันตัวตน
- 2) **Dashboard ตามบทบาทผู้ใช้** สำหรับนักศึกษา อาจารย์ และผู้ดูแลระบบ
- 3) **Client สำหรับเรียก API** ที่เชื่อมต่อกับ Momoi และบริการยืนยันตัวตนผ่าน OpenAPI

การแยกส่วนติดต่อผู้ใช้ออกมาเป็นรีโพซิทอรีเฉพาะช่วยให้สามารถพัฒนา UI, จัดการ state และ ทดสอบประสิทธิภาพผู้ใช้ได้โดยไม่กระทบกับตรรกะของบริการฝั่ง backend

4.1.2 รีโพซิทอรี Momoi

รีโพซิทอรี **Momoi** เป็นบริการ API หลักของแพลตฟอร์ม พัฒนาด้วย Bun, Elysia.js และ Prisma ทำหน้าที่เป็นศูนย์กลางของข้อมูลธุรกรรมและตรรกะทางธุรกิจ ภายในประกอบด้วยโฟลเดอร์สำคัญ เช่น src/routes สำหรับกำหนด API, src/service สำหรับ business logic, src/database/prisma สำหรับ schema ของฐานข้อมูล, src/queue สำหรับตัวส่งงานเข้าคิว และ test สำหรับชุดทดสอบ

- 1) **Academic Management API** สำหรับข้อมูลรายวิชา ภาคการศึกษา ผู้สอน และโครงสร้างที่เกี่ยวข้อง
- 2) **Request และ Instance API** สำหรับจัดการคำร้อง เครื่องเสมือน reverse proxy และ ประวัติการใช้งาน
- 3) **Storage และ Observability** สำหรับจัดการไฟล์ การเชื่อมต่อ S3-compatible storage และการเปิดเผย OpenAPI, metrics, tracing

รีโพซิทอรีนี้ยังเป็นจุดเชื่อมต่อกับ PostgreSQL, Redis และบริการจัดเก็บไฟล์ ทำให้ Momoi ทำหน้าที่เป็นแกนกลางของแพลตฟอร์มในเชิงข้อมูลและการประสานงานระหว่างบริการ รวมถึงดูแลเส้นทางการยืนยันตัวตน '/api/auth' ภายในบริการเดียวกันเพื่อลดความซับซ้อนของ topology

4.1.3 รีโพซิทอรี Yuzu

รีโพซิทอรี **Yuzu** เป็นบริการประมวลผลงานเบื้องหลังสำหรับการจัดสรร ลบ และเปลี่ยนสถานะ เครื่องเสมือน โดยรับงานจากคิวและเชื่อมต่อกับ Proxmox VE แบบ asynchronous ภายในมีโฟลเดอร์ src/queue สำหรับ worker แต่ละประเภท, src/pve สำหรับ client และชนิดข้อมูลของ Proxmox API, และ src/database สำหรับการอ่านข้อมูลรวมจาก PostgreSQL

- 1) **Provision Worker** สำหรับสร้างและเตรียมเครื่องเสมือนใหม่
- 2) **Deprovision Worker** สำหรับคืนทรัพยากรและลบเครื่องที่ไม่ใช้งานแล้ว
- 3) **Toggle Status Worker** สำหรับสั่งเริ่ม หยุด หรือรีสตาร์ทเครื่องเสมือน
- 4) **Configurable Queue Concurrency** สำหรับปรับระดับการประมวลผลพร้อมกันของแต่ละคิวผ่านตัวแปรสภาพแวดล้อม
- 5) **Redis-based Resource Locks** สำหรับป้องกันงานที่ชน instance เดียวกันหรือ 'VMID' เดียวกันไม่ให้ทำงานซ้อนกัน

การแยก Yuzu ออกมาเป็น worker service ช่วยให้สามารถจัดการงานที่ใช้เวลานานและงานที่ต้อง retry ได้อย่างเหมาะสม โดยไม่ทำให้ API หลักต้องรอผลการทำงานจาก Proxmox VE โดยตรง

อีกทั้งใน implementation ปัจจุบันยังสามารถเพิ่มหรือลด concurrency ของแต่ละคิวได้โดยไม่ต้องแก้ไขโค้ด และใช้ distributed lock เพื่อจำกัดงานที่มีโอกาสชน resource เดียวกัน ช่วยลดความเสี่ยงต่อปัญหา storage lock timeout หรือการเปลี่ยนสถานะซ้ำซ้อนของเครื่องเสมือน

4.1.4 รีโพซิทอรี Satsuki

รีโพซิทอรี **Satsuki** เป็นบริการสนับสนุนด้านโครงสร้างพื้นฐานที่รับผิดชอบการสร้างไฟล์กำหนดค่า reverse proxy และไฟล์ ‘authorized_keys’ แบบอัตโนมัติจากข้อมูลในฐานข้อมูล พัฒนาด้วย Bun และจัดโครงสร้างภายในตามแนว clean architecture โดยมี ‘src/domain’, ‘src/application’, ‘src/infrastructure’ และ ‘src/bootstrap’ แยกตามบทบาทชัดเจน ในการติดตั้งใช้งานจริง Satsuki จะถูกนำไปติดตั้งบนเครื่อง Nginx ภายนอกซึ่งทำหน้าที่เป็น reverse proxy หน้าด่านของระบบ เพื่อให้สามารถเขียนไฟล์ configuration และ reload Nginx ได้โดยตรง

- 1) **Reverse Proxy Config Generator** สำหรับสร้าง Nginx stream/http configuration จากข้อมูล Instance และ reverse proxy mapping บนเครื่อง proxy ภายนอก
- 2) **Authorized Keys Generator** สำหรับสร้างไฟล์กุญแจ SSH ที่ใช้กับ bastion หรือเครื่องกลางของระบบบนโฮสต์เดียวกัน
- 3) **PostgreSQL LISTEN/NOTIFY Integration** สำหรับติดตามการเปลี่ยนแปลงของข้อมูล และ trigger การ regenerate configuration พร้อม reload Nginx โดยอัตโนมัติ

บริการนี้ทำให้ความสามารถด้าน reverse proxy และ SSH access ที่ถูกกล่าวถึงในเชิงฟังก์ชันของแพลตฟอร์มมี implementation ที่แยกเป็นระบบย่อยชัดเจนมากขึ้น และเหมาะกับการทำงานบนเครื่องหน้าด่านที่ต้องถือครองไฟล์กำหนดค่าจริงของ Nginx

4.1.5 รีโพซิทอรี Yuuka

รีโพซิทอรี **Yuuka** ใช้สำหรับเก็บไฟล์กำหนดค่าการติดตั้งระบบและบริการสนับสนุน เช่น manifest สำหรับ Kubernetes, ตัวอย่าง environment file, เอกสาร observability และไฟล์ docker-compose.yaml สำหรับสภาพแวดล้อมพัฒนา ภายในรีโพซิทอรีนี้มีบริการสำคัญ เช่น PostgreSQL, Redis, RustFS, Prometheus, Grafana, Loki, Jaeger, InfluxDB รวมถึงไฟล์ deployment ของ midori, momoi และ yuzu

โครงสร้างลักษณะนี้ช่วยให้การจัดการโครงสร้างพื้นฐานแยกออกจากโค้ดแอปพลิเคชันอย่างชัดเจน และทำให้สามารถปรับใช้ระบบได้ทั้งในระดับ local development และ cluster environment โดยในภาพรวม Yuuka เป็นจุดที่กำหนด ingress routing และการเชื่อมต่อระหว่างบริการ เช่น การนำคำขอจากผู้ใช้ไปยัง Midori และ Momoi รวมถึงการผูกบริการสนับสนุนเข้ากับระบบหลักอย่างเป็นระบบ

4.2 โครงสร้างแอปพลิเคชันและบริการ

4.2.1 ส่วนประกอบหลักของระบบ

นอกจากการแยกรีโพสิทอรีตามหน้าที่แล้ว แต่ละบริการยังมีไฟล์กำหนดค่าหลักของตนเองเพื่อควบคุมการพัฒนาและการติดตั้งใช้งาน เช่น `package.json` สำหรับจัดการ dependency และ `script`, `tsconfig.json` สำหรับกำหนดค่า TypeScript, `Dockerfile` สำหรับการสร้าง container image และ `README.md` สำหรับอธิบายวิธีใช้งานและขั้นตอนการติดตั้ง

ในส่วนของ Momoi, Yuzu และ Satsuki ยังมีไฟล์ `prisma.config.ts` หรือไฟล์กำหนดค่าฐานข้อมูลที่เกี่ยวข้องสำหรับการเชื่อมต่อข้อมูล ขณะที่ Midori มีไฟล์ `next.config.ts`, `components.json` และ `postcss.config.mjs` เพื่อควบคุมการทำงานของเฟรมเวิร์กและระบบออกแบบส่วนติดต่อผู้ใช้ ส่วน Yuuka ใช้ไฟล์ `.yaml` จำนวนมากเพื่ออธิบาย topology ของบริการที่ติดตั้งจริง

4.2.2 โครงสร้างระบบทั้งหมด

โดยในสภาพแวดล้อมจริงของการพัฒนาปัจจุบัน โครงสร้างถูกแยกเป็นหลายรีโพสิทอรีตามความรับผิดชอบของบริการ และแต่ละรีโพสิทอรีมีเอกสาร ไฟล์กำหนดค่า และ source code ของตนเองอย่างชัดเจน

นอกจากนี้ยังประกอบด้วยไฟล์สำคัญอีกหลายไฟล์ ได้แก่

- `Dockerfile` ของแต่ละบริการ สำหรับสร้าง container image แยกตามหน้าที่
- `package.json` และ `bun.lock` หรือไฟล์ `lock` ที่ใช้ควบคุม dependency
- `tsconfig.json`, `prisma.config.ts` และไฟล์กำหนดค่าเฉพาะเฟรมเวิร์ก เช่น `next.config.ts`
- ไฟล์ตัวอย่างสภาพแวดล้อม เช่น `.env.example` และไฟล์โนโพลเดอร์ `yuuka/docs/env`
- ไฟล์ `deployment` และ `observability`
- ไฟล์เอกสาร เช่น `README.md`, `docs/API.md`, `docs/QUEUE.md` และเอกสารประกอบในรีโพสิทอรีต่าง ๆ

การแบ่งโครงสร้างเป็นหลายรีโพสิทอรีช่วยลด coupling ระหว่างบริการ ทำให้สามารถวางแผนการพัฒนา ทดสอบ และติดตั้งใช้งานของแต่ละส่วนได้อย่างยืดหยุ่น เมื่อต้องการปรับปรุง frontend, API, worker หรือ infrastructure ก็สามารถทำได้เฉพาะส่วนโดยไม่ต้องแบกรับความซับซ้อนของโครงสร้างแบบรวมศูนย์ขนาดใหญ่

การจัดโครงสร้างโครงการในลักษณะหลายรีโพสิทอรีนี้ช่วยให้สามารถกำหนดขอบเขตของบริการได้ชัดเจน ลดผลกระทบข้ามระบบเมื่อต้องแก้ไขโค้ด และเหมาะสมกับการดูแลระบบที่มีทั้ง application service และ deployment manifest อยู่ร่วมกันในแพลตฟอร์มเดียว

4.3 สถาปัตยกรรมระบบและเทคโนโลยี

4.3.1 การออกแบบสถาปัตยกรรมแบบ Microservices

ระบบแพลตฟอร์มคลาวด์ที่พัฒนาขึ้นใช้สถาปัตยกรรมแบบ Microservices ซึ่งประกอบด้วยบริการหลักเชิงธุรกิจ 3 ตัว คือ Midori, Momoi และ Yuzu และมีบริการสนับสนุนเพิ่มเติม ได้แก่ Satsuki สำหรับ reverse proxy/SSH configuration บนเครื่อง Nginx ภายนอก, PostgreSQL สำหรับข้อมูลธุรกรรม Redis สำหรับ cache และคิวงาน RustFS สำหรับพื้นที่จัดเก็บไฟล์ และชุด observability สำหรับติดตามสถานะของระบบ แต่ละบริการทำงานอิสระและสื่อสารกันผ่าน HTTP API ระบบคิวงานแบบ asynchronous และกลไก event จากฐานข้อมูล เพื่อให้ระบบมีความยืดหยุ่นและรองรับการขยายตัวได้ง่าย

เมื่อพิจารณาการทำงานจริงของระบบในคลัสเตอร์ ผู้ใช้จะเริ่มจาก Browser ไปยัง Midori จากนั้น Midori จะเรียก Momoi สำหรับ API ด้านข้อมูล ตรรกะทางธุรกิจ และการยืนยันตัวตนผ่าน ‘/api/auth’ ส่วนงานที่ใช้เวลานาน เช่น การสร้าง ลบ หรือเปลี่ยนสถานะเครื่องเสมือน Momoi จะส่งเข้าสู่ BullMQ บน Redis แล้วให้ Yuzu รับงานไปดำเนินการต่อกับ Proxmox VE ขณะเดียวกัน Satsuki ซึ่งทำงานอยู่บนเครื่อง Nginx ภายนอกจะติดตามการเปลี่ยนแปลงของข้อมูล reverse proxy และ SSH key จากฐานข้อมูลเพื่อ regenerate ไฟล์กำหนดค่าที่เกี่ยวข้องและ reload reverse proxy หน้าด้านโดยอัตโนมัติ รูปแบบนี้ทำให้ API หลักตอบสนองต่อผู้ใช้ได้รวดเร็ว แม้งานโครงสร้างพื้นฐานจะใช้เวลาและต้อง retry หลายครั้ง

4.3.2 เทคโนโลยีที่ใช้ในการพัฒนา

ในการพัฒนาระบบแพลตฟอร์มคลาวด์นี้ ได้เลือกใช้เทคโนโลยีที่ทันสมัยและเหมาะสมกับการใช้งานในสภาพแวดล้อมการศึกษา โดยเทคโนโลยีหลักที่ใช้ประกอบด้วย

- **Bun Runtime** - JavaScript/TypeScript Runtime ที่มีประสิทธิภาพสูง
- **Next.js 16 และ React 19** - Framework และ Library สำหรับการพัฒนา Frontend
- **Tailwind CSS และ Radix UI** - เครื่องมือสำหรับออกแบบส่วนติดต่อผู้ใช้
- **Elysia.js** - Web Framework สำหรับการพัฒนา API ของ Momoi
- **PostgreSQL** - ระบบจัดการฐานข้อมูลเชิงสัมพันธ์
- **Prisma ORM** - Object-Relational Mapping สำหรับการจัดการฐานข้อมูล
- **Redis และ BullMQ** - ระบบ cache และคิวงาน
- **Docker และ Kubernetes Manifests** - สำหรับการ deploy
- **Prometheus, Grafana, Loki และ Jaeger** - สำหรับการเก็บ metrics, logs และ traces

การเลือกใช้เทคโนโลยีเหล่านี้มีเหตุผลสำคัญ คือ ความเสถียร ประสิทธิภาพ และการรองรับการขยายระบบในอนาคต รวมถึงการมี Community ที่ใหญ่และเอกสารประกอบที่ครบถ้วน ทำให้การพัฒนาและบำรุงรักษาระบบเป็นไปอย่างราบรื่น

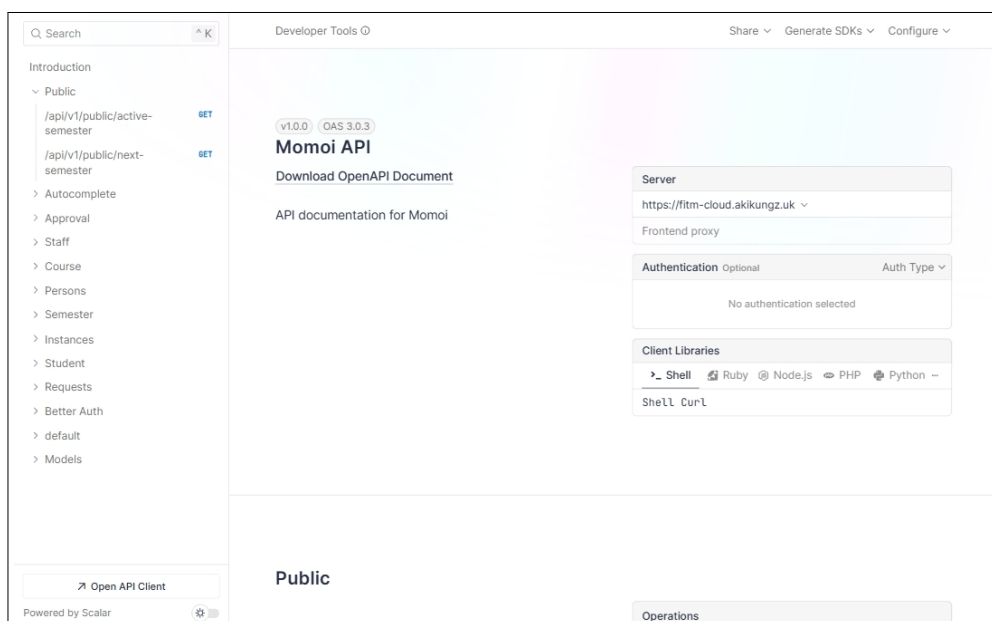
4.4 เอกสารประกอบการใช้งาน API และส่วนติดต่อผู้ใช้

4.4.1 เอกสาร API Documentation

ระบบแพลตฟอร์มคลาวด์ได้จัดทำเอกสาร API Documentation อย่างครบถ้วนโดยใช้ OpenAPI Specification (Swagger) เพื่อให้ผู้พัฒนาและผู้ใช้งานสามารถเข้าใจและใช้งาน API ได้อย่างง่ายดาย เอกสารนี้ประกอบด้วยรายละเอียดของ Endpoints ต่าง ๆ พารามิเตอร์ที่ต้องการ รูปแบบข้อมูลที่ส่งและรับ รวมทั้งตัวอย่างการใช้งานที่ชัดเจน

ภาพที่ 4-1 แสดงหน้าเอกสาร API Documentation ที่สร้างขึ้นโดยอัตโนมัติจากโค้ด TypeScript ผ่าน Elysia.js และ OpenAPI Plugin ซึ่งช่วยให้เห็นโครงสร้างของบริการ Momoi ตามโดเมนงานหลักของระบบ โดยสามารถจัดกลุ่มการใช้งานได้ดังนี้

- **Public API** - สำหรับการใช้งานทั่วไปที่ไม่ต้องมีการยืนยันตัวตน
- **Academic Management API** - สำหรับจัดการรายวิชา ภาคการศึกษา และข้อมูลผู้สอน
- **Request Workflow API** - สำหรับสร้าง ตรวจสอบ และอนุมัติคำร้องขอเครื่องเสมือน
- **Instance API** - สำหรับจัดการเครื่องเสมือนและช่องทางเข้าถึงบริการ
- **Storage API** - สำหรับจัดการไฟล์



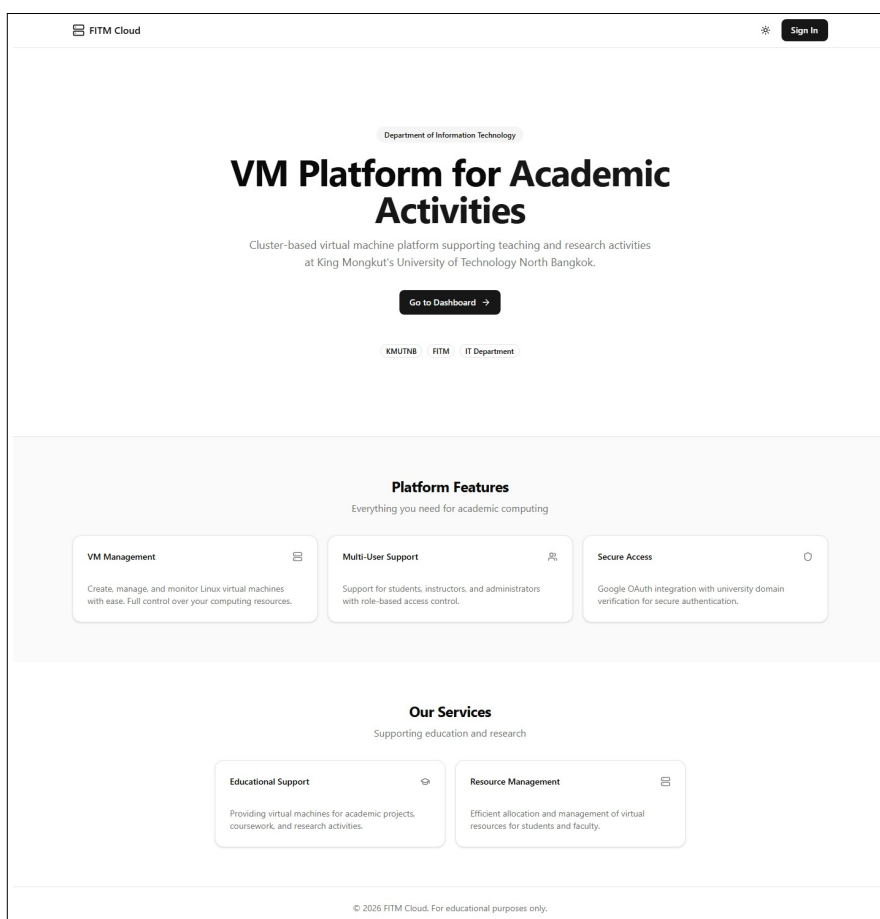
ภาพที่ 4-1 หน้าเอกสาร API Documentation

เอกสาร API นี้มีการอัปเดตอัตโนมัติเมื่อมีการเปลี่ยนแปลงโค้ด และสามารถทดสอบ API ได้โดยตรงผ่านหน้าเว็บ ทำให้ผู้พัฒนาสามารถทำความเข้าใจและใช้งาน API ได้อย่างรวดเร็วและถูกต้อง

4.4.2 ส่วนติดต่อผู้ใช้ (User Interface)

ส่วนติดต่อผู้ใช้ของระบบได้รับการออกแบบให้ใช้งานง่าย สวยงาม และตอบสนองได้ดีบนอุปกรณ์ต่าง ๆ โดยพัฒนาด้วย Next.js, React, Tailwind CSS และชุดองค์ประกอบ UI ที่อิงกับ Radix UI ระบบประกอบด้วยหน้าหลักต่าง ๆ ที่มีการจัดระเบียบข้อมูลและฟังก์ชันการทำงานอย่างชัดเจน ทั้งในส่วนของ dashboard, request workflow, instance management และ storage interface

ภาพที่ 4-2 แสดงหน้าแรกของระบบ (Landing Page) ที่ออกแบบให้ดึงดูดความสนใจและสื่อสารจุดประสงค์ของระบบได้ชัดเจน โดยมีการนำเสนอฟีเจอร์หลักของระบบ ข้อมูลการติดต่อ และลิงก์สำหรับการล็อกอินเข้าสู่ระบบ



ภาพที่ 4-2 หน้าแรกของระบบ (Landing Page)

4.4.3 Dashboard และระบบจัดการ

หลังจากที่ผู้ใช้ล็อกอินเข้าสู่ระบบแล้ว ผู้ใช้จะเข้าสู่หน้า Dashboard ที่แสดงข้อมูลสำคัญและฟังก์ชันการทำงานต่าง ๆ ตามบทบาทของผู้ใช้ โดย Dashboard ได้รับการออกแบบให้แสดงข้อมูลอย่างเป็นระบบและเข้าใจง่าย พร้อมทั้งมีการใช้งานที่สะดวกและรวดเร็ว

ส่วนติดต่อผู้ใช้ที่พัฒนาขึ้นมีลักษณะสำคัญดังนี้

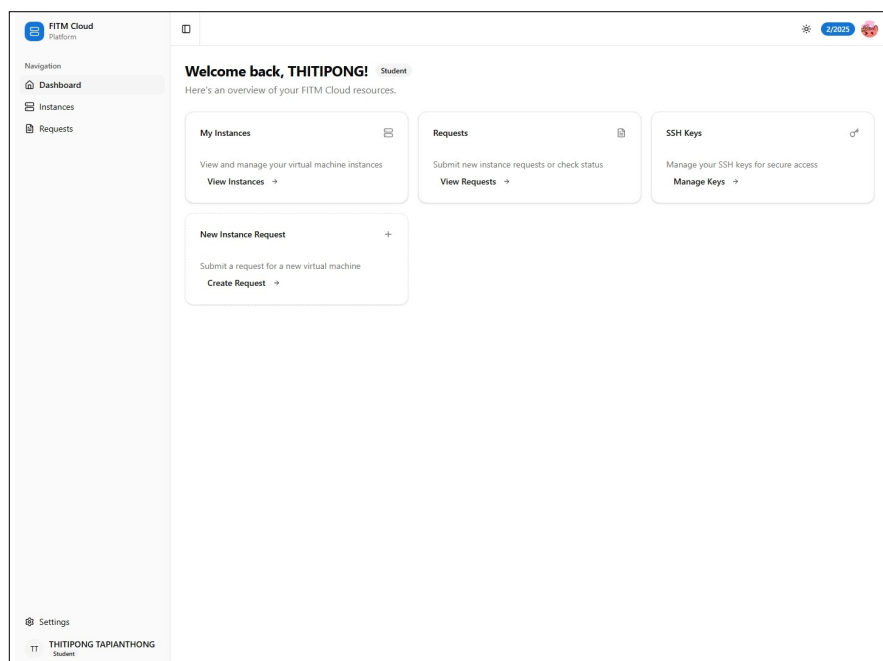
- **Responsive Design** - รองรับการแสดงผลบนหน้าจอขนาดต่าง ๆ
- **Intuitive Navigation** - การนำทางที่เข้าใจง่ายและสอดคล้องกับมาตรฐาน UX
- **Role-based UI Visibility** - เมนูและตัวเลือกปรับแสดงผลโดยอัตโนมัติตามบทบาทของผู้ใช้ STUDENT INSTRUCTOR หรือ ADMIN โดยไม่ต้องกำหนดเพิ่มเติม
- **Request Lifecycle Tracking** - แสดงสถานะคำร้องขอตั้งแต่การยื่น การรอพิจารณา จนถึงการอนุมัติหรือปฏิเสธ พร้อม Audit Log ในแต่ละขั้นตอน
- **Instance Control Panel** - แผงควบคุมสถานะเครื่องเสมือน รองรับการสั่ง Start Stop Restart และดูข้อมูล IP และ Reverse Proxy ได้จากหน้าเดียวกัน
- **File Management Interface** - อินเทอร์เฟซสำหรับอัปโหลด จัดไฟล์เตอร์ และกำหนดสิทธิ์การแบ่งปันไฟล์ภายในแพลตฟอร์ม

การออกแบบส่วนติดต่อผู้ใช้ได้คำนึงถึงประสบการณ์ผู้ใช้ (User Experience) เป็นหลัก โดยมุ่งเน้นให้ผู้ใช้สามารถทำงานได้อย่างมีประสิทธิภาพและลดข้อผิดพลาดในการใช้งาน ทั้งนี้ระบบยังมีการจัดการข้อผิดพลาดและการแจ้งเตือนที่ชัดเจน เพื่อให้ผู้ใช้สามารถแก้ไขปัญหาได้อย่างรวดเร็ว

4.5 ตัวอย่างการใช้งานระบบตามบทบาทผู้ใช้

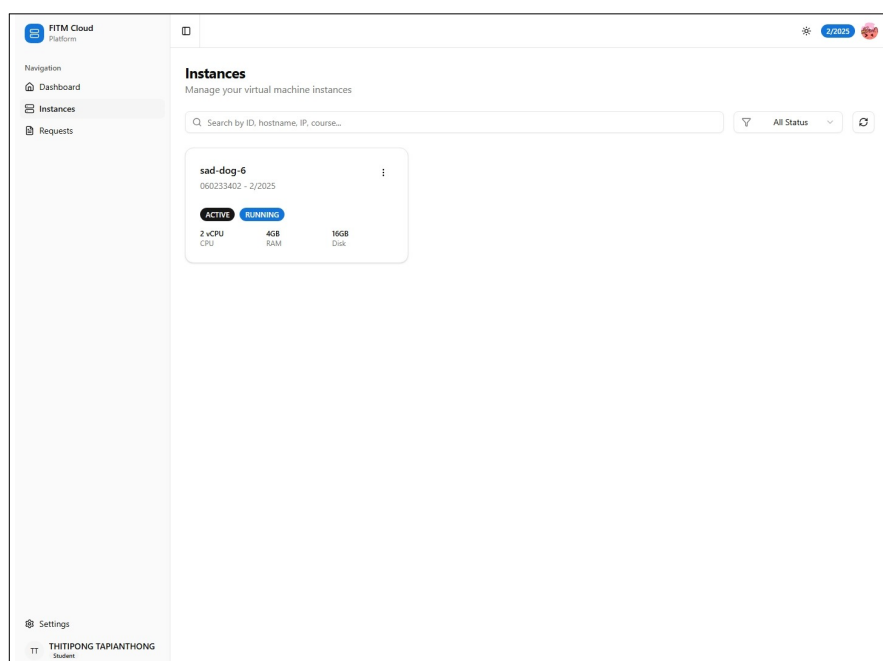
4.5.1 Dashboard สำหรับนักศึกษา

ระบบได้จัดทำ Dashboard เฉพาะสำหรับนักศึกษา เพื่อให้สามารถจัดการเครื่องเสมือนส่วนตัว และติดตามการใช้งานทรัพยากรได้อย่างง่ายดาย ภาพที่ 4-3 แสดง Dashboard หลักของนักศึกษาที่ประกอบด้วยข้อมูลสรุปการใช้งาน สถานะเครื่องเสมือน และฟังก์ชันการจัดการที่สำคัญ



ภาพที่ 4-3 Dashboard หลักสำหรับนักศึกษา

นักศึกษาสามารถดูรายการเครื่องเสมือนทั้งหมดที่ตนเองมีสิทธิ์เข้าถึง พร้อมทั้งสถานะการทำงาน การใช้ทรัพยากร และการจัดการพื้นฐาน ดังแสดงในภาพที่ 4-4 ซึ่งนำเสนอรายละเอียดของแต่ละ Instance อย่างชัดเจน



ภาพที่ 4-4 หน้าจัดการ VM Instances สำหรับนักศึกษา

นอกจากนี้ นักศึกษายังสามารถขอสร้างเครื่องเสมือนใหม่ผ่านฟอร์มที่ออกแบบมาอย่างเรียบง่าย ดังแสดงในภาพที่ 4-5 ซึ่งช่วยให้การขอใช้ทรัพยากรเป็นไปอย่างรวดเร็วและมีประสิทธิภาพ

FITM Cloud Platform

Navigation
Dashboard
Instances
Requests

New Instance Request
Submit a request for a new virtual machine instance

Request Information
Basic information about your instance request

Title
A short title for your request
My Development VM

Course
Select the course this instance is for
Select a course...

Operating System
Select the operating system template
Select an OS template...

Description
Explain what you'll use this instance for
I need this instance to work on my course project...

Instance Specifications
Configure the resources for your virtual machine

CPU Cores
2 vCPU
1 vCPU | 8 vCPUs

Memory
4 GB
1 GB | 16 GB

Disk Size
16 GB
16 GB | 64 GB

Summary

CPU	2 vCPU
Memory	4 GB
Disk	16 GB

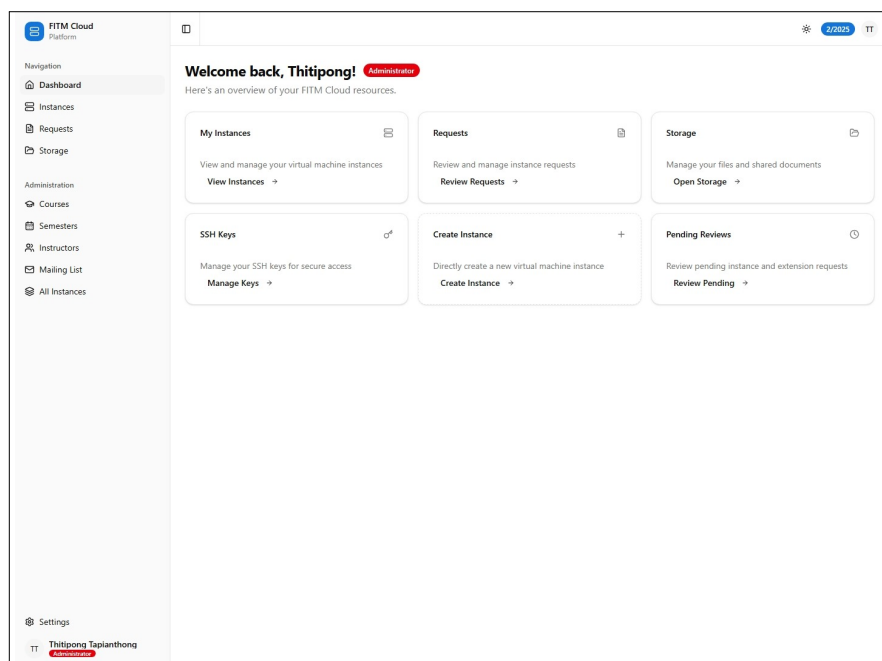
Submit Request
Cancel

Settings
THITIPONG TAPIANTHONG
Student

ภาพที่ 4-5 หน้าจอขอสร้าง VM Instance ใหม่สำหรับนักศึกษา

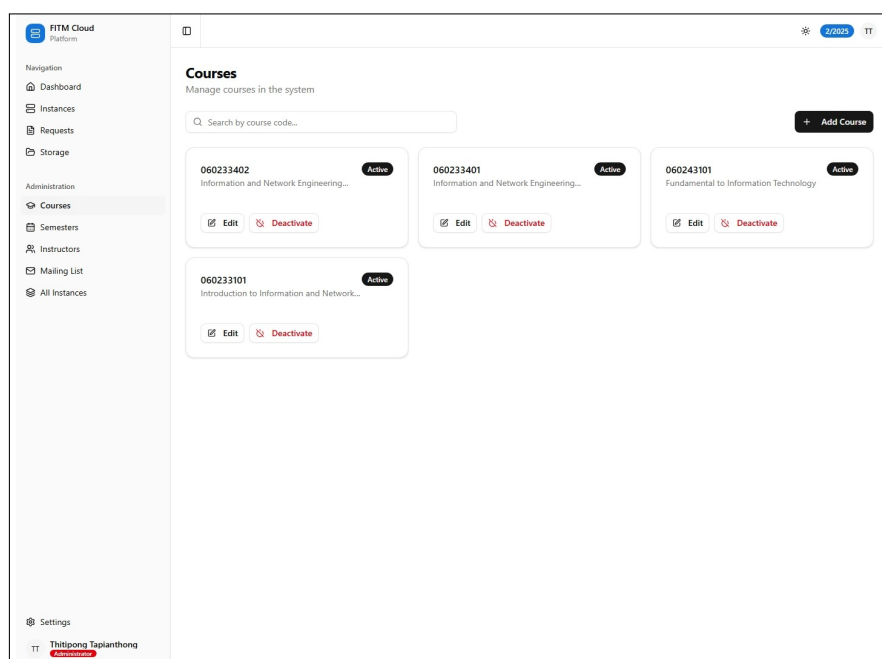
4.5.2 Dashboard สำหรับอาจารย์และเจ้าหน้าที่

สำหรับอาจารย์และเจ้าหน้าที่ ระบบได้จัดเตรียม Dashboard ที่มีฟังก์ชันการจัดการชั้นสูงมากขึ้น รวมถึงการอนุมัติคำขอ การจัดการหลักสูตร และการตรวจสอบการใช้งานของนักศึกษา ภาพที่ 4-6 แสดง Dashboard หลักสำหรับเจ้าหน้าที่ที่มีข้อมูลสถิติและการควบคุมระบบ

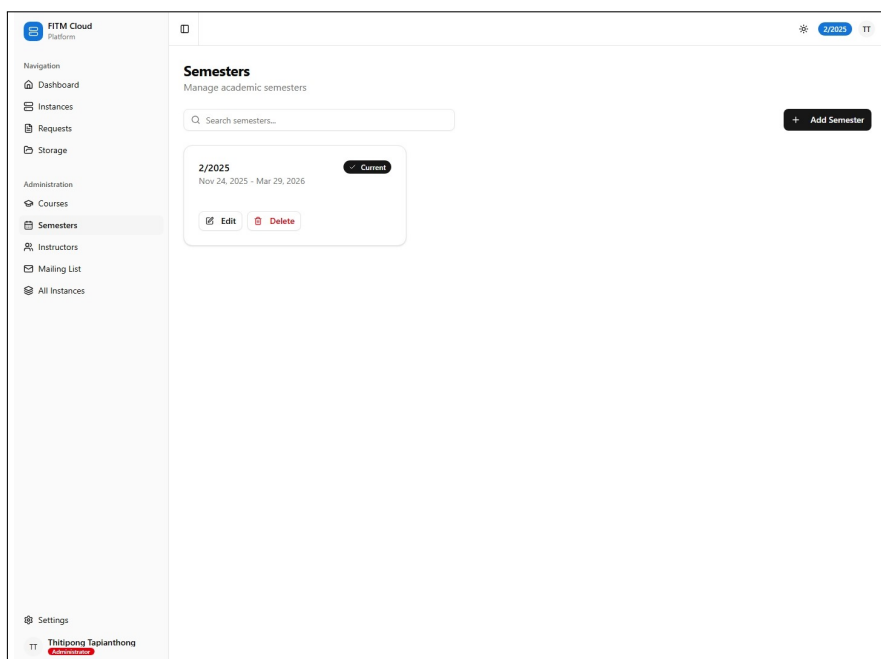


ภาพที่ 4-6 Dashboard หลักสำหรับอาจารย์และเจ้าหน้าที่

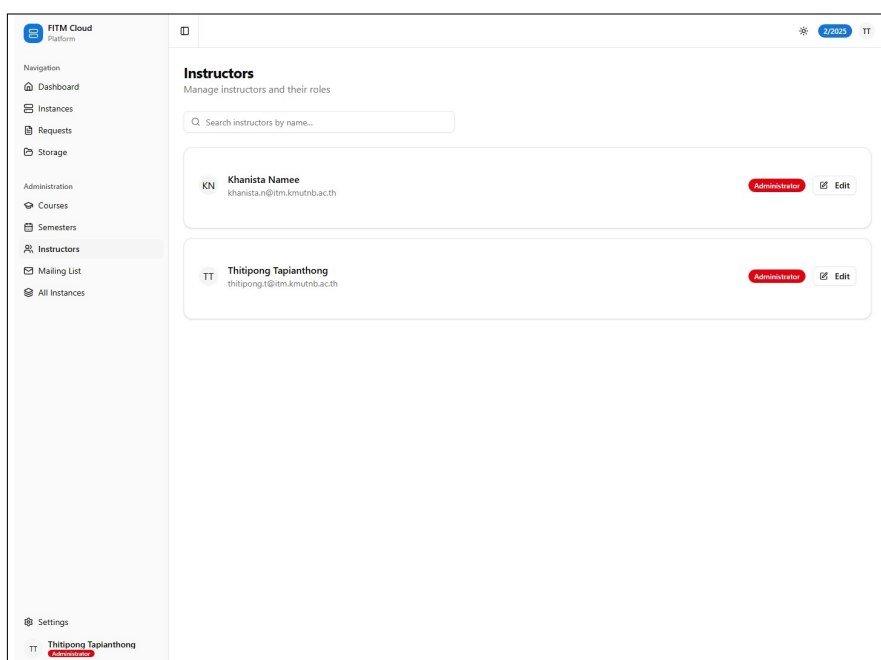
อาจารย์และเจ้าหน้าที่สามารถจัดการหลักสูตร เทอมการศึกษา รวมถึงการจัดการรายชื่อของอาจารย์และเจ้าหน้าที่ ดังแสดงในภาพที่ 4-7 - 4-9 ซึ่งช่วยให้สามารถกำหนดทรัพยากรและสิทธิ์การเข้าถึงสำหรับแต่ละวิชาได้อย่างเหมาะสม



ภาพที่ 4-7 หน้าจัดการหลักสูตรและรายวิชา

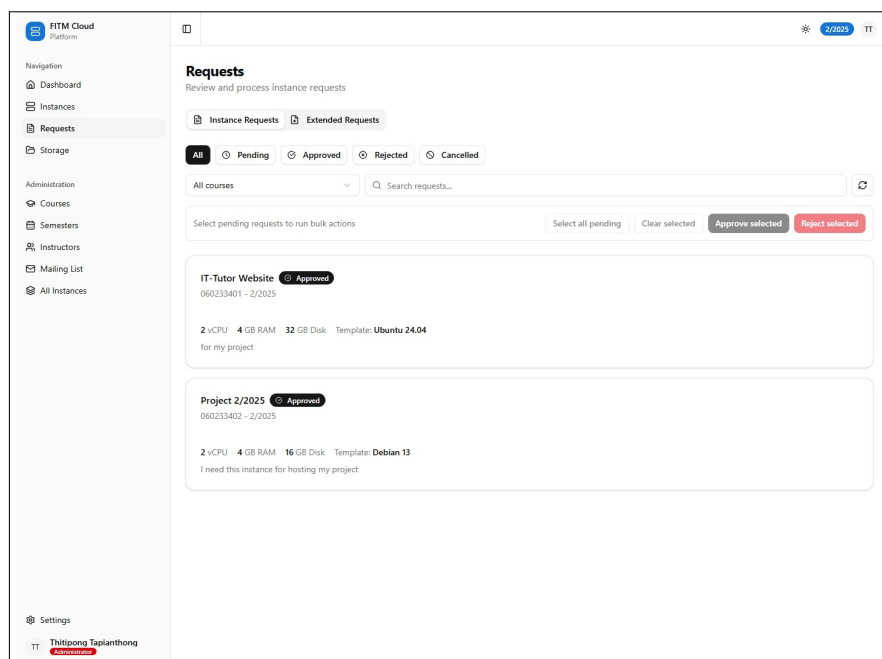


ภาพที่ 4-8 หน้าจัดการเทอมการศึกษา



ภาพที่ 4-9 หน้าจัดการรายชื่ออาจารย์

นอกจากนี้ ระบบยังมีหน้าสำหรับการอนุมัติคำขอต่าง ๆ จากนักศึกษา ดังแสดงในภาพที่ 4-10 ซึ่งช่วยให้อาจารย์สามารถตรวจสอบและอนุมัติคำขอได้อย่างมีประสิทธิภาพ



ภาพที่ 4-10 หน้าอนุมัติคำขอที่รอดำเนินการ

4.5.3 ระบบจัดการแบบ Role-Based Access Control

ระบบได้นำเอา Role-Based Access Control (RBAC) มาใช้ในการจัดการสิทธิ์การเข้าถึงข้อมูล และฟังก์ชันต่าง ๆ โดยแบ่งผู้ใช้เป็น 3 กลุ่มหลัก คือ นักศึกษา อาจารย์ และผู้ดูแลระบบ ซึ่งแต่ละกลุ่ม จะมีสิทธิ์และหน้าที่ที่แตกต่างกันตามความเหมาะสม

การ ออกแบบ ระบบ ใน ลักษณะ นี้ ช่วย ให้ การ จัดการ ความ ปลอดภัย ของ ข้อมูล เป็น ไป อย่าง มี ประสิทธิภาพ และผู้ใช้แต่ละกลุ่มจะเห็นเฉพาะข้อมูลและฟังก์ชันที่เกี่ยวข้องกับหน้าที่ของตนเอง ทำให้ การใช้งานง่ายขึ้นและลดความเสี่ยงในการเข้าถึงข้อมูลที่ไม่เหมาะสม

4.6 ผลการทดสอบประสิทธิภาพระบบ

ส่วนนี้สรุปผลการทดสอบประสิทธิภาพของบริการหลักในระบบ โดยแบ่งตามบริการย่อย เพื่อให้ เห็นภาพรวมและเปรียบเทียบสมรรถนะของแต่ละบริการได้อย่างชัดเจน

4.6.1 ผลการทำงานของ Proxmox Cluster และ การใช้หน่วยความจำร่วมด้วย KSM

ในส่วน ของ โครงสร้าง พื้นฐาน สำหรับ เครื่อง เสมือน ระบบ ถูก ออกแบบ ให้ ทำงาน บน Proxmox Cluster จำนวน ทั้งหมด 10 โหนด โดยแต่ละโหนดมีทรัพยากร 20 CPU และ 32 GB RAM ส่งผลให้ ทั้งคลัสเตอร์มีทรัพยากรรวมเชิงกายภาพเท่ากับ 200 CPU และ 320 GB RAM ดังตารางที่ 4-1 ซึ่ง

เพียงพอสำหรับรองรับการใช้งานเครื่องเสมือนจำนวนมากในบริบทของห้องปฏิบัติการ รายวิชา และงานทดสอบที่มีรูปแบบการใช้งานคล้ายกันในหลายเครื่องพร้อมกัน

ตารางที่ 4-1 สรุปทรัพยากรของ Proxmox Cluster ที่ใช้ในระบบ

รายการ	ค่า
จำนวนโหนดในคลัสเตอร์	10 โหนด
CPU ต่อโหนด	20 CPU
หน่วยความจำต่อโหนด	32 GB RAM
CPU รวมทั้งคลัสเตอร์	200 CPU
หน่วยความจำรวมทั้งคลัสเตอร์	320 GB RAM

ผลลัพธ์ที่สำคัญของคลัสเตอร์นี้ไม่ได้มาจากจำนวนทรัพยากรเชิงกายภาพเพียงอย่างเดียว แต่ยังมาจากความสามารถของ Proxmox VE ในการใช้ **Kernel Samepage Merging (KSM)** เพื่อรวมหน้า memory ที่มีข้อมูลซ้ำกันระหว่าง VM หลายตัวเข้าด้วยกัน เมื่อเครื่องเสมือนจำนวนมากในระบบปฏิบัติการตระกูลเดียวกัน หรือใช้ distro และ version ที่ใกล้เคียงกัน เช่น Ubuntu, Debian หรือ Linux รุ่นเดียวกัน จะมีส่วนของ page memory ที่ซ้ำกันอยู่เป็นจำนวนมาก KSM จะช่วยลดการเก็บข้อมูลซ้ำเหล่านี้ ทำให้หน่วยความจำที่ระบบต้องใช้จริงต่ำกว่าผลรวมของหน่วยความจำที่จัดสรรให้ VM แต่ละเครื่อง

คุณสมบัตินี้ทำให้คลัสเตอร์สามารถรองรับการใช้งานแบบ **memory overcommit** ได้อย่างมีประสิทธิภาพ กล่าวคือ แม้ผลรวมของ RAM ที่กำหนดให้กับ VM บนโหนดหนึ่ง ๆ จะมากกว่า **32 GB** ซึ่งเป็นหน่วยความจำจริงของเครื่อง แต่ระบบยังสามารถทำงานได้อย่างเสถียรราบเท่าที่ working set ที่ถูกใช้งานจริงไม่ได้แตกต่างกันทั้งหมดทุกเครื่อง ในบริบทของระบบนี้ที่มักมี VM จำนวนมากสร้างจาก template เดียวกันและใช้งานแพ็คเกจพื้นฐานคล้ายกัน KSM จึงช่วยเพิ่มประสิทธิภาพการใช้ RAM ของโหนดได้อย่างชัดเจน ลดโอกาสที่ต้องใช้ swap เร็วเกินไป และช่วยให้สามารถจัดสรรเครื่องเสมือนได้หนาแน่นขึ้นโดยยังรักษาความต่อเนื่องของบริการ

ในเชิงผลลัพธ์ต่อการใช้งานจริง แนวทางดังกล่าวเหมาะกับสภาพแวดล้อมการศึกษามาก เนื่องจากผู้ใช้งานจำนวนมากมักเริ่มต้นจาก image มาตรฐานชุดเดียวกัน ทำให้เกิดประโยชน์จาก KSM สูงกว่างานที่มี workload หลากหลายมาก ระบบจึงสามารถใช้ทรัพยากรหน่วยความจำได้คุ้มค่ากว่าเดิม รองรับการทำงานพร้อมกันจำนวนมาก และทำให้การบริหารคลัสเตอร์ขนาด 10 โหนดมีความยืดหยุ่นมากขึ้นโดยไม่ต้องเพิ่มหน่วยความจำเชิงกายภาพในทันทีทุกครั้งที่มีการขยายจำนวน VM

4.6.2 ผลการทดสอบประสิทธิภาพส่วนติดต่อผู้ใช้ (Midori)

ทำการทดสอบสมรรถนะของ Midori (Frontend/Next.js) โดยมีเป้าหมายการทดสอบคือ <https://dev.fitm.cloud> ระยะเวลารวม 30 วินาที และระดับความขนานต่อเส้นทาง (Path-Level Parallelism) 100 พร้อมทั้งบันทึกรายละเอียดคำขอรายครั้งลงในไฟล์ log เพื่อใช้วิเคราะห์ความสม่ำเสมอของเวลาตอบสนองและพฤติกรรมในช่วง tail latency

ภาพรวมของการทดสอบสะท้อนปริมาณคำขอรวม 3,506 คำขอ อัตราความสำเร็จ 96.55% และอัตราการส่งผ่านโดยเฉลี่ย 109.67 RPS ดังตารางที่ 4-2

ตารางที่ 4-2 สรุปผลภาพรวมการทดสอบ Midori

เวลาเริ่ม	ระยะเวลา (s)	Requests	สำเร็จ	ล้มเหลว	Throughput (RPS)
2026-03-12 02:08:19Z	30	3,506	3,385	121	109.67

การทดสอบครอบคลุมเส้นทางหลัก ได้แก่ หน้าแรกและหน้าลงชื่อเข้าใช้ ซึ่งเป็นเส้นทางสำคัญของส่วนติดต่อผู้ใช้ สถิติหลักสรุปในตารางที่ 4-3

ตารางที่ 4-3 ผลการทดสอบรายเส้นทาง (Midori)

เส้นทาง	Requests	สำเร็จ	ล้มเหลว	RPS	p50 (ms)	p95 (ms)	p99 (ms)	max (ms)
/	1,730	1,670	60	54.12	699.18	12,858.73	15,008.66	15,015.25
/login	1,776	1,715	61	55.81	705.81	11,971.10	15,006.54	15,016.48

ข้อสังเกตและข้อเสนอแนะเบื้องต้น

- ผลการทดสอบแสดงให้เห็นว่า Midori ยังให้บริการได้ต่อเนื่องภายใต้โหลด 100 concurrent users ต่อเส้นทาง แต่มี tail latency สูงมาก โดยค่า p95 และ p99 ของทั้งสองหน้าแต่ละระดับ 11.9–15.0 วินาที และมีค่าขอล้มเหลวรวม 121 ครั้ง
- จากไฟล์บันทึกคำขอ พบว่าคำขอส่วนหนึ่งในช่วงต้นและช่วงกลางของการทดสอบตอบกลับได้ในช่วงประมาณ 80–900 ms แต่มีอีกกลุ่มหนึ่งชนเพดาน timeout ของสคริปต์ที่ประมาณ 15 วินาที สอดคล้องกับค่า max latency ที่สูงมากในผลสรุป
- หน้าแรกและหน้าลงชื่อเข้าใช้มีสมรรถนะใกล้เคียงกัน โดย p50 อยู่ราว 700 ms แสดงว่าคอขวดน่าจะอยู่ที่ขั้นการ render หรือการเรียกทรัพยากรรวมมากกว่าจะเป็นเพียงหน้าใดหน้าหนึ่งโดยเฉพาะ
- หากต้องการปรับปรุงผลลัพธ์ ควรตรวจสอบการทำ caching ของ Next.js, การโหลด asset และ data fetching ที่เกิดในฝั่ง server, รวมถึงการตั้งค่า reverse proxy และ connection reuse ระหว่าง Midori กับบริการปลายทาง เพื่อลด tail latency และลดจำนวนคำขอที่ timeout

4.6.3 ผลการทดสอบประสิทธิภาพบริการ API (Momoi)

เพื่อ ประเมิน สมรรถนะ ของ บริการ Momoi (API Service) ได้ ดำเนิน การ ทดสอบ ภาระ งาน ด้วยการ ยิง คำขอ พร้อม กัน ต่อ เส้น ทาง (per-path concurrency) โดยมี เป้าหมาย การ ทดสอบ คือ <https://dev.fitm.cloud> ที่ ระยะเวลา รวม 30 วินาที และ ระดับ ความ ขนาน ต่อ เส้น ทาง 100 พร้อม บันทึก ผล ราย คำขอ ทุก ครั้ง ลง ไฟล์ log เพื่อ ใช้ วิเคราะห์ เสถียร ภาพ ของ API ภาย ใต้ โหลด จริง

ภาพรวม ของ การ ทดสอบ สะท้อน ปริมาณ คำขอ รวม 7,216 คำขอ อัตรา ความ สำเร็จ 100% และ อัตรา การ ส่ง ผ่าน โดย เฉลี่ย 234.87 RPS ดัง ตาราง ที่ 4-4

ตารางที่ 4-4 สรุปผลภาพรวมการทดสอบ Momoi API

เวลาเริ่ม	ระยะเวลา (s)	Requests	สำเร็จ	ล้มเหลว	Throughput (RPS)
2026-03-12 02:12:46Z	30	7,216	7,216	0	234.87

จาก ข้อมูล สรุป ครั้ง นี้ คำขอ ทั้งหมด สำเร็จ (ค่า ok= 7,216) สะท้อน ว่า เส้น ทาง API ที่ เลือก ทดสอบ ยัง คง ทำงาน ได้ ถูก ต้อง ภาย ใต้ เงื่อนไข การ ทดสอบ ดัง กล่าว ตาม ข้อมูล ใน ตาราง ที่ 4-5

ตารางที่ 4-5 ผลการทดสอบรายเส้นทาง (Momoi API)

เส้นทาง	Requests	สำเร็จ	ล้มเหลว	RPS	p50 (ms)	p95 (ms)	p99 (ms)	max (ms)
/api/openapi/json	4,228	4,228	0	137.73	566.46	987.09	1,479.75	1,688.93
/api/academic/semesters/current	2,988	2,988	0	97.25	987.90	1,308.27	1,740.05	2,736.64

ข้อสังเกตและข้อเสนอแนะเบื้องต้น

- Momoi มี เสถียร ภาพ สูง ภาย ใต้ โหลด 100 concurrent users ต่อ เส้น ทาง โดย ไม่ มี คำขอ ล้มเหลว เลย และ ยัง คง รักษา throughput รวม ได้ ประมาณ 234.87 RPS บน โดเมน พัฒนา
- เส้น ทาง /api/openapi/json มี throughput สูง กว่า เส้น ทาง /api/academic/semesters/current แม้ ว่า จะ มี ขนาด ข้อมูล ตอบ กลับ สูง มาก โดย จาก log ราย คำขอ พบ ว่า ขนาด ตอบ กลับ ของ OpenAPI อยู่ ที่ ประมาณ 218,193 ไบต์ ต่อ คำขอ และ สร้าง ปริมาณ ข้อมูล รวม มากกว่า 880 MB ตลอด การ ทดสอบ
- เส้น ทาง /api/academic/semesters/current มี payload ขนาด เล็ก มาก ประมาณ 195 ไบต์ ต่อ คำขอ แต่ กลับ มี p50 สูง กว่า เส้น ทาง OpenAPI อย่าง ชัด เจน สะท้อน ว่า คอขวด หลัก น่า จะ เกิด จาก การ ประมวลผล ใน handler หรือ การ เข้า ถึง ข้อมูล backend มากกว่า ขนาด response เพียง อย่าง เดียว
- จาก log ราย คำขอ ใน ช่วง ต้น ของ การ ทดสอบ พบ ว่า ค่า latency ของ /api/openapi/json ส่วน ใหญ่ อยู่ ประมาณ 680–1,160 ms ขณะที่ /api/academic/semesters/current กระจุย อยู่ ราว 1,224–1,740 ms ซึ่ง สอดคล้อง กับ ค่า percentile ใน ผล สรุป และ บ่งชี้ ว่า ระบบ มี ความ สม่า เสมอ แม้อยู่ ภาย ใต้ โหลด ต่อ เนื่อง

- หากต้องการลด latency ของเส้นทางเชิงธุรกิจ ควรตรวจสอบ query path ของ endpoint ภาคการศึกษาปัจจุบัน การใช้ cache สำหรับข้อมูลที่เปลี่ยนไม่บ่อย และการแยกเส้นทางเอกสาร OpenAPI ออกจาก traffic ใช้งานจริงเมื่อทำการทดสอบเชิงเปรียบเทียบในอนาคต

4.6.4 ผลการทดสอบประสิทธิภาพบริการจัดการเครื่องเสมือน (Yuzu)

เพื่อประเมินสมรรถนะของบริการ Yuzu ได้ทำการทดสอบการโคลนเครื่องเสมือน (VM Clone) เปรียบเทียบระหว่างวิธี *Linked Clone* และ *Full Clone* โดยใช้แม่แบบ VM เดียวกันและกระจายงานไปยังโหนดปลายทางหลายเครื่อง

ในการตีความผลการทดสอบส่วนนี้ ควรระบุนขอบเขตของการวัดเวลาให้ชัดเจนว่าเวลาที่รายงานครอบคลุมเฉพาะช่วงการสร้างหรือโคลนเครื่องเสมือนจนกระทั่งงานฝั่งโครงสร้างพื้นฐานเสร็จสิ้นในระดับที่ระบบสามารถส่งต่อไปยังขั้นตอนถัดไปได้เท่านั้น แต่ยังไม่ได้รวม เวลาของกระบวนการ *cloud-init* หลังบูตเครื่อง เช่น การตั้งค่าเริ่มต้น การอัปเดตแพ็คเกจ และการอัปเดตแพ็คเกจภายใน guest OS ซึ่งจากการสังเกตการใช้งานจริงอาจใช้เวลาเพิ่มได้อีกราว 3 นาที ก่อนที่เครื่องจะอยู่ในสภาพพร้อมใช้งานเต็มรูปแบบ

ภาพรวมการทดสอบประกอบด้วยทั้งหมด 20 เคส โดยเป็น Linked Clone 10 เคส และ Full Clone 10 เคส สถานะสำเร็จ/ล้มเหลวและเวลาที่ใช้สรุปได้ดังตารางที่ 4-6 และข้อสังเกตที่ตามมา

ตารางที่ 4-6 สรุปผลการทดสอบการโคลน VM: Linked vs Full

วิธีโคลน	จำนวนนับ	สำเร็จ	ล้มเหลว	เวลาเฉลี่ย (ms)	ช่วงเวลา (min/max ms)
Linked	10	10	0	5,931.49	1,492.12 / 10,423.62
Full	10	3	7	172,493.80 ¹	167,395.68 / 175,525.81

เมื่อเปรียบเทียบเวลาเฉลี่ย วิธี **Linked Clone** เร็วกว่าวิธี **Full Clone** โดยเฉลี่ยประมาณ 166,562.31 ms หรือคิดเป็นร้อยละ 96.56 ของเวลาที่ลดลง ซึ่งสอดคล้องกับสถาปัตยกรรมการใช้งาน *copy-on-write* ของ Linked Clone ที่ไม่คัดลอกดิสก์เต็มทั้งลูกในขั้นตอนเริ่มต้น

อย่างไรก็ตาม หากพิจารณาในมุมของ *time-to-usable-instance* หรือเวลาดังแต่เริ่ม provision จนผู้ใช้สามารถเข้าใช้งานเครื่องได้จริง ระยะเวลาที่รับรู้ได้จริงจะยาวกว่าค่าที่แสดงในตาราง เนื่องจากยังต้องรวมช่วง boot และขั้นตอน *cloud-init* ภายในเครื่อง โดยเฉพาะงาน *packages update/upgrade* ที่อาจเพิ่มเวลาได้อีกราว 180–240 วินาที ทำให้แม้ Linked Clone จะเสร็จในระดับโครงสร้างพื้นฐานภายในไม่กี่วินาที แต่เวลาพร้อมใช้งานจริงของเครื่องอาจอยู่ในระดับประมาณ 3–4 นาที ขณะที่ Full Clone อาจสูงกว่านั้นอย่างมีนัยสำคัญ

¹เวลาเฉลี่ยของ Full Clone คำนวณจากเคสที่สำเร็จ (3 เคส) ตามข้อมูลในไฟล์ผลการทดสอบ

นอกจากผลการเปรียบเทียบชนิดของการโคลนแล้ว ยังพบข้อสังเกตด้าน **throughput ของคิวงาน** ว่าประสิทธิภาพของ Yuzu ไม่ได้ขึ้นกับจำนวน worker pod บน Kubernetes เพียงอย่างเดียว แต่ขึ้นกับ **ผลคูณระหว่างจำนวน Pods และค่า concurrency ของแต่ละคิว** ด้วย ในสภาพแวดล้อมทดสอบนี้ได้กำหนดจำนวน worker ไว้ที่ **3 Pods** และกำหนดค่า concurrency ปริยายของคิว *provision* เท่ากับ 5, *deprovision* เท่ากับ 5 และ *toggle status* เท่ากับ 10 ดังนั้นความสามารถเชิงทฤษฎีในการดึงงานขึ้นมาประมวลผลพร้อมกันจึงอยู่ที่ประมาณ **15 งาน** สำหรับ *provision*, **15 งาน** สำหรับ *deprovision* และ **30 งาน** สำหรับ *toggle status* หากงานเหล่านั้นไม่ชน resource lock เดียวกัน

ผลลัพธ์ที่สังเกตได้คือ เมื่อภาระงานยังไม่เกินความจุรวมของคิวที่เกี่ยวข้อง ระยะเวลาการเริ่มประมวลผลของแต่ละงานจะใกล้เคียงกับเวลาที่วัดจากการทดสอบ clone โดยตรง แต่เมื่อจำนวนงานสูงเกินกว่าความจุรวมของ worker สำหรับคิวนั้น หรือมีหลายงานอ้างอิง ‘instanceld’ หรือ ‘VMID’ เดียวกัน งานลำดับถัดไปจะยังไม่ล้มเหลวทันที แต่จะเข้าสู่สถานะรอคิว (*queued/waiting*) หรือรอ lock ของทรัพยากรนั้นจนกว่าจะพร้อมประมวลผล ดังนั้นคอขวดในช่วงนี้ไม่ได้เกิดจากตรรกะภายใน Yuzu เพียงจุดเดียว แต่เกิดจากทั้งขีดจำกัดของจำนวน worker ที่พร้อมดึงงานจาก BullMQ และข้อจำกัดด้านความปลอดภัยของ resource locks ที่ใช้ป้องกันไม่ให้เกิดการจัดการ VM เดียวกันซ้อนกัน

อย่างไรก็ตาม การเพิ่ม concurrency เพียงอย่างเดียวไม่เพียงพอสำหรับงานที่แต่ละทรัพยากรเดียวกันใน Proxmox VE เช่น การสั่ง *provision/deprovision/toggle* บนเครื่องเสมือนเดียวกัน ด้วยเหตุนี้ Yuzu จึงเพิ่มกลไก **Redis-based distributed locks** เพื่อครอบ resource สำคัญสองระดับ ได้แก่ lock ตาม ‘instanceld’ สำหรับกันไม่ให้ job หลายประเภทเข้ามาแก้ไข instance เดียวกันพร้อมกัน และ lock ตาม ‘VMID’ สำหรับกันไม่ให้มีคำสั่งหลายชุดเข้าจัดการเครื่องเสมือนหมายเลขเดียวกันพร้อมกัน ไม่ว่าจะมาจากคิวชนิดใดก็ตาม แนวทางนี้ทำให้ระบบยังคงเปิดโอกาสให้หลายงานทำงานบน Proxmox node เดียวกันได้ หากเป็นคนละ ‘VMID’ และไม่ชนกับเครื่องเดิม รวมถึงยังเอื้อให้การทำ Linked Clone จาก template เดียวกันสามารถเกิดขึ้นแบบขนานได้ เพราะ lock จะอ้างอิงที่ ‘VMID’ ปลายทางของเครื่องใหม่ ไม่ใช่ template ต้นทาง

ตารางที่ 4-7 พฤติกรรมของ Yuzu เมื่อจำนวนงานพร้อมกันสัมพันธ์กับจำนวน Pod และค่า concurrency

ลักษณะของภาระงาน	สถานะของการประมวลผล	ผลกระทบที่สังเกตได้
จำนวนงานไม่เกินความจุรวมของคิวที่เกี่ยวข้อง และไม่ชน lock	ทุก Pod สามารถดึงงานขึ้นมาประมวลผลได้พร้อมกันตามค่า concurrency ของคิวนั้น	เวลาตอบสนองใกล้เคียงเวลาประมวลผลจริงของงาน และใช้ทรัพยากรของ worker ได้คุ้มค่าขึ้น
จำนวนงานเกินความจุรวมของคิว	งานส่วนเกินต้องรอใน BullMQ จนกว่าจะมี slot ว่างจาก Pod ใด Pod หนึ่ง	เวลารวมเพิ่มขึ้นจากระยะเวลารอคิว แม้งานแต่ละชิ้นจะยังทำงานได้ปกติ
จำนวนงานชน 'instanceId' หรือ 'VMID' เดียวกัน	งานที่ชน resource เดียวกันจะถูกหน่วงไว้จนกว่างานก่อนหน้านี้จะปล่อย lock	throughput รวม ของ ระบบ ยัง สูง ได้ แต่ การ จัดการ VM เดียวกันจะไม่ถูกทำซ้ำจนจนเกิด race condition

ตารางที่ 4-8 กลไกควบคุมการทำงานพร้อมกันของ Yuzu ฉบับปัจจุบัน

กลไก	หน้าที่	ผลลัพธ์ที่คาดหวัง
Queue Concurrency ต่อคิว	ปรับ จำนวน งาน ที่ หนึ่ง Pod ของ Yuzu รับพร้อมกันได้ในแต่ละ คิว ผ่าน environment variables	เพิ่ม throughput ต่อ Pod และลดเวลารอคิวเมื่อ work-load กระจายได้ดี
Lock ตาม 'instanceId'	กันไม่ให้ job หลายชนิด เข้ามา เปลี่ยน สถานะ หรือ จัดการ เครื่องเดียวกันพร้อมกัน	ลดความเสี่ยงจาก race condition และ สถานะ VM ไม่สอดคล้องกัน
Lock ตาม 'VMID'	กันไม่ให้หลาย job เข้าจัดการ เครื่องเสมือนหมายเลขเดียวกัน พร้อมกัน แม้จะอยู่คนละคิว	ลดโอกาสสร้าง ลบ หรือเปลี่ยน สถานะ VM เดียวกันซ้อนกันจนเกิดความไม่สอดคล้อง

ข้อสังเกตและการวิเคราะห์

- อัตราความสำเร็จ: Linked Clone สำเร็จ 100% (10/10) ในขณะที่ Full Clone สำเร็จ 30% (3/10)
- สาเหตุความล้มเหลวของ Full Clone: พบข้อความผิดพลาด เช่น "cfs-lock 'storage-rbd' error: got lock request timeout" และ "failed with exit status: WARNINGS: 1" ซึ่งบ่งชี้ถึงปัญหาการล็อกทรัพยากรของระบบจัดเก็บข้อมูลแบบ RBD/คลัสเตอร์ หรือข้อจำกัดด้านโควตา/นโยบายของสตอเรจ
- ความแปรปรวนของเวลา: เคส Linked Clone มีช่วงเวลาตั้งแต่ 1.49 วินาที ถึง 10.42 วินาที ขึ้นอยู่กับโหนดปลายทางและภาระงาน ณ ขณะนั้น ขณะที่ Full Clone ที่สำเร็จใช้เวลาประมาณ 167 – 175 วินาที (2.8 – 2.9 นาที) ต่อเคส
- ขอบเขตของเวลาในผลทดสอบ: ตัวเลขในตารางยังไม่รวมระยะเวลา cloud-init หลังบูตเครื่อง ดังนั้นเวลาพร้อมใช้งานจริงของเครื่องเสมือนจะสูงกว่าค่าที่รายงาน โดยเฉพาะกรณีที่ guest OS มีการสั่ง *update* และ *upgrade* แพ็กเกจระหว่างเริ่มต้นระบบ
- ผลกระทบต่อการออกแบบระบบ: สำหรับงานสอนที่ต้องสร้าง VM จำนวนนักศึกษาจำนวนมากในเวลาจำกัด Linked Clone ให้เวลาโปรวิชันรวดเร็วอย่างมีนัยสำคัญ และลดความเสี่ยงจากคอขวดด้าน IO ของสตอเรจส่วนกลาง
- ผลกระทบของจำนวน Pods ต่อคิวงาน: เมื่อเปิดใช้งาน Yuzu เพียง 3 Pods ระบบจะรองรับการประมวลผลพร้อมกันได้ดีที่สุดที่สุทธราว 3 งานในช่วงเวลาเดียวกัน หากมีงานมากกว่านี้ queue latency จะเพิ่มขึ้นทันทีแม้ตัว worker และ Proxmox VE ยังทำงานได้ปกติ
- ผลกระทบของ concurrency ภายใน Pod: การเปิดให้หนึ่ง instance ของ Yuzu ประมวลผลได้หลายงานพร้อมกันช่วยเพิ่มประสิทธิภาพการใช้ทรัพยากรของ Pod แต่จำเป็นต้องทำงานร่วมกับ resource locks เพื่อไม่ให้ throughput ที่เพิ่มขึ้นกลายเป็น contention กับ VM เดิมหรือทำให้สถานะของเครื่องเสมือนไม่สอดคล้องกัน
- ผลกระทบของ distributed locks: กลไก lock ตาม 'instancetype' และ 'VMID' ช่วยให้การเพิ่ม concurrency มีความปลอดภัยมากขึ้น โดยเฉพาะในงานที่มีโอกาสชน resource ซ้ำ เช่น provision ซ้อนกับ deprovision ของเครื่องเดิม หรือ toggle/deprovision ที่อ้างถึง VM ตัวเดียวกัน

ข้อเสนอแนะเชิงปฏิบัติ

- ใช้ Linked Clone เป็นค่าเริ่มต้นในการโปรวิชัน VM จำนวนมาก โดยเฉพาะกรณีแล็บการสอนหรือกิจกรรมที่เน้นความเร็วในการเริ่มใช้งาน
- หากต้องการวัดประสิทธิภาพใช้งานจริงของผู้ใช้ ควรเพิ่มตัวชี้วัดจาก *clone completed* ไปเป็น *instance ready after cloud-init* เช่น รอจน cloud-init เสร็จและบริการพื้นฐานตอบสนองได้ เพื่อสะท้อนเวลาพร้อมใช้งานจริงของเครื่อง

- สำหรับ Full Clone ให้พิจารณาโหมดคิวงาน/การจัดคิวแบบจำกัดพร้อมกัน (throttling) และปรับ *timeout* ของการล็อกสตอเรจ เพื่อหลีกเลี่ยง *cfs-lock timeout*
- ตรวจสอบคอนฟิกสตอเรจ RBD/คลัสเตอร์ (เช่น พารามิเตอร์ Ceph/RBD, คิวงานบน Proxmox VE, และเครือข่ายสตอเรจ) เพื่อบรรเทาข้อความผิดพลาดประเภท WARNINGS และเพิ่มอัตราสำเร็จของ Full Clone
- พิจารณาปรับภาพแม่แบบให้ลดงานที่ต้องทำในช่วง cloud-init เช่น เตรียมแพ็คเกจพื้นฐานไว้ล่วงหน้า หรือแยกงาน *update/upgrade* ที่ใช้เวลานานออกจากเส้นทาง provision หลักเพื่อลดเวลารอจริงของผู้ใช้
- หากคาดว่าจะมีการส่งงานพร้อมกันเกิน 3 งานเป็นประจำ ควรพิจารณาเพิ่มจำนวน Yuzu Pods หรือทำ autoscaling ตามความลึกของคิว เพื่อให้ queue waiting time ไม่กลายเป็นคอขวดหลักของระบบ
- ปรับค่า concurrency ของแต่ละคิวแยกจากกันตามลักษณะงานจริง เช่น จำกัด provision/deprovision มากกว่างาน toggle status ที่เบากว่า และติดตามผลกระทบหลังการปรับค่าทุกครั้ง
- ใช้ข้อมูลจาก lock timeout, queue depth และ active jobs ร่วมกันในการตัดสินใจว่าจะเพิ่ม Pod หรือเพียงปรับ concurrency ภายใน Pod เพื่อหลีกเลี่ยงการเพิ่มโหนดบน storage หรือ node โดยไม่จำเป็น
- เฝ้าติดตามตัวชี้วัด IO/ล็อกของสตอเรจระหว่างช่วงพีก และทดสอบซ้ำหลังปรับแต่ง เพื่อยืนยันการปรับปรุง

บทที่ 5

สรุปผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

โครงการ "Cloud-Based Platform for Supporting Teaching and Academic Activities" ได้พัฒนาแพลตฟอร์มคลาวด์สำหรับสนับสนุนการเรียนการสอนและกิจกรรมทางวิชาการในภาควิชา เทคโนโลยีสารสนเทศให้สามารถใช้งานได้จริงในลักษณะ Microservices Architecture โดยมีบริการหลัก 3 ส่วน ได้แก่ Midori สำหรับส่วนติดต่อผู้ใช้ Momoi สำหรับ API ตรรกะทางธุรกิจ และการเชื่อมต่อด้านการยืนยันตัวตน และ Yuzu สำหรับการประมวลผลงานจัดการเครื่องเสมือน ร่วมกับบริการสนับสนุนด้านฐานข้อมูล ระบบคิวงาน พื้นที่จัดเก็บไฟล์ และระบบติดตามการทำงาน ทั้งหมดถูกจัดวางและติดตั้งใช้งานผ่านรีโพซิทอรี Yuuka ซึ่งทำหน้าที่ดูแล deployment manifests และ observability stack ของแพลตฟอร์ม

ผลลัพธ์สำคัญของโครงการคือการสร้างแพลตฟอร์มที่รองรับวงจรการใช้งานเครื่องเสมือนตั้งแต่การยื่นคำร้อง การอนุมัติ การจัดสรรทรัพยากร การเปิดใช้งาน การควบคุมสถานะ ไปจนถึงการลบคืนทรัพยากร โดยเชื่อมโยงกระบวนการดังกล่าวเข้ากับข้อมูลทางวิชาการ เช่น รายวิชา ภาคการศึกษา และผู้สอน ทำให้การให้บริการเครื่องเสมือนสอดคล้องกับบริบทการเรียนการสอนจริงของภาควิชา ในเชิงสถาปัตยกรรม Momoi ทำหน้าที่บันทึกข้อมูลธุรกรรมและประสานงานกับบริการภายนอก ขณะที่งานโครงสร้างพื้นฐานที่ใช้เวลานานจะถูกส่งเข้าสู่ Redis-backed BullMQ เพื่อให้ Yuzu รับไปดำเนินการบน Proxmox VE แบบ asynchronous ซึ่งเป็นแนวทางที่ช่วยให้ระบบตอบสนองต่อผู้ใช้ได้รวดเร็วขึ้น

นอกจากนี้ ระบบยังรองรับการควบคุมสิทธิ์แบบ Role-Based Access Control (RBAC) สำหรับผู้ใช้กลุ่ม STUDENT, INSTRUCTOR และ ADMIN มีเอกสาร API แบบ OpenAPI สำหรับการพัฒนาและทดสอบ มีส่วนจัดการไฟล์บน S3-compatible storage และมีชุด observability สำหรับติดตาม metrics, logs และ traces ของบริการหลัก โดยเส้นทางการทำงานหลักของระบบถูกออกแบบให้ Midori ติดต่อกับ Momoi สำหรับทั้ง application API และ auth integration ทำให้ขอบเขตของ frontend, application API และ worker orchestration มีความชัดเจน ผลการดำเนินงานโดยรวมจึงสะท้อนว่าระบบสามารถเป็นพื้นฐานของแพลตฟอร์มดิจิทัลสำหรับสนับสนุนการเรียนการสอนและกิจกรรมทางวิชาการได้อย่างเป็นรูปธรรม

ในด้านสมรรถนะ ผลการทดสอบประสิทธิภาพบริการหลักบนสภาพแวดล้อม <https://dev.fitm.cloud> พบว่า Midori รองรับโหลดรวมได้ประมาณ 109.67 คำขอต่อวินาที (RPS) ภายใต้การทดสอบ 30

วินาทีที่ระดับความขนาน 100 ต่อเส้นทาง โดยมีอัตราความสำเร็จ 96.55% และค่า latency median ของหน้า / และ /login อยู่ราว 699–706 ms แต่มี tail latency สูงมากจนค่า p95 และ p99 แต่ละช่วงประมาณ 12–15 วินาที ขณะที่ Momoi รองรับได้ประมาณ 234.87 RPS ด้วยอัตราความสำเร็จ 100% โดยเส้นทาง /api/openapi/json มีค่า p50 ประมาณ 566 ms และเส้นทาง /api/academic/semesters/current มีค่า p50 ประมาณ 988 ms ซึ่งสะท้อนว่า API หลักยังคงมีเสถียรภาพภายใต้โหลดต่อเนื่อง แม้เส้นทางเชิงธุรกิจบางส่วนยังมี latency สูงกว่าที่ควร ส่วนการทดสอบ Yuzu พบว่า Linked Clone มีอัตราความสำเร็จ 100% และเร็วกว่า Full Clone โดยเฉลี่ยกว่า 96% ในระดับเวลาการโคลนและจัดสรรโครงสร้างพื้นฐาน อีกทั้ง implementation ปัจจุบันยังรองรับการกำหนด concurrency ของแต่ละคิวผ่าน environment variables และใช้ Redis-based distributed locks เพื่อกันงานที่ชน 'instanceId' หรือ 'VMID' เดียวกันไม่ให้งานพร้อมกันโดยไม่จำเป็น ขณะเดียวกันงานหลายชิ้นยังสามารถทำงานบน Proxmox node เดียวกันแบบขนานได้ หากเป็นคณะ 'VMID' ซึ่งช่วยให้การทำ Linked Clone จาก template เดียวกันยังคงมี throughput สูง อย่างไรก็ตาม ตัวเลขดังกล่าวยังไม่รวมเวลาหลังบูตของ cloud-init เช่น การ *update* และ *upgrade* แพ็กเกจภายในเครื่อง ซึ่งอาจใช้เวลาเพิ่มอีกราว 3 นาที ทำให้เวลาพร้อมใช้งานจริงของเครื่องเสมือนสูงกว่าค่าที่รายงาน ผลการทดสอบดังกล่าวบ่งชี้ว่าระบบสามารถใช้งานจริงได้ในบริบทการเรียนการสอนของภาควิชา แต่ยังมีประเด็นด้าน tail latency ของ frontend, latency ของบาง API endpoint และเวลาการเตรียมเครื่องหลัง provision ที่ควรได้รับการปรับปรุงเพิ่มเติม

5.2 ปัญหาและอุปสรรค

- 5.2.1 ความซับซ้อนของการประสานงานระหว่างบริการหลายส่วน เนื่องจากระบบประกอบด้วย frontend, API, worker และบริการสนับสนุนจำนวนมากที่ต้องทำงานร่วมกันผ่าน HTTP API, Redis และฐานข้อมูล ทำให้การดีบั๊กและติดตามปัญหาต้องอาศัยความเข้าใจภาพรวมของระบบ
- 5.2.2 การจัดการเครื่องเสมือนผ่าน Proxmox VE API มีรายละเอียดเชิงเทคนิคสูง ทั้งด้านสถานะของ VM, การเลือกโหนด, การจัดสรรเครือข่าย และการจัดคิวงานที่ใช้เวลานาน โดยเฉพาะงานประเภท clone หรือ reprovision ที่มีผลกระทบต่อทรัพยากรของระบบโดยตรง
- 5.2.3 การออกแบบโมเดลข้อมูลให้รองรับทั้งบริบททางวิชาการและการควบคุมสิทธิ์ของผู้ใช้เป็นโจทย์สำคัญ เนื่องจากระบบไม่ได้จัดการเพียงบัญชีผู้ใช้ แต่ยังต้องเชื่อมโยงข้อมูลรายวิชา ภาควิชา การศึกษา ผู้สอน คำร้อง และเครื่องเสมือนเข้าด้วยกันอย่างถูกต้อง
- 5.2.4 การใช้เทคโนโลยีใหม่ในหลายชั้นของระบบ เช่น Bun Runtime, Elysia.js, BullMQ, Prisma และแนวทางการสร้าง type-safe client จาก OpenAPI ต้องใช้เวลาในการศึกษา ทดลอง และกำหนดแนวทางที่เหมาะสมกับโครงการ

- 5.2.5 การติดตามประสิทธิภาพและความผิดปกติของระบบในสภาพแวดล้อมที่มีหลายบริการ ยังเป็นความท้าทาย โดยเฉพาะเมื่อต้องวิเคราะห์ความล่าช้าจากหลายจุด เช่น frontend proxy, API handler, queue worker และการเรียกใช้งาน Proxmox VE
- 5.2.6 การทำเอกสารสถาปัตยกรรมให้สอดคล้องกับระบบที่ติดตั้งใช้งานจริงมีความท้าทาย เนื่องจากระบบปัจจุบันแยกเป็นหลายรีโพสิทอรีและมีความแตกต่างบางส่วนระหว่าง implementation ที่ตั้งใจไว้กับ workaround ที่ใช้ในบางสภาพแวดล้อม ทำให้เอกสารที่อธิบายภาพรวมของระบบต้องอ้างอิงทั้ง source code และ deployment manifests ควบคู่กัน

5.3 การแก้ปัญหา

- 5.3.1 แยกความรับผิดชอบของระบบออกเป็นรีโพสิทอรีและบริการที่ชัดเจน ได้แก่ Midori, Momoi, Yuzu และ Yuuka พร้อมใช้ Dockerfile และไฟล์ deployment เพื่อให้สามารถพัฒนา ทดสอบ และติดตั้งใช้งานแต่ละส่วนได้อย่างเป็นอิสระ
- 5.3.2 ใช้ OpenAPI เป็นมาตรฐานสำหรับการอธิบาย API ของ Momoi และนำข้อมูลดังกล่าวไปใช้สร้างชนิดข้อมูลฝั่ง client ช่วยลดความคลาดเคลื่อนระหว่าง frontend และ backend รวมถึงช่วยให้เอกสารประกอบการพัฒนามีความชัดเจนมากขึ้น
- 5.3.3 พัฒนา Yuzu ให้ทำงานเป็น background worker ที่รับงานจาก Redis-backed BullMQ แทนการประมวลผลแบบ synchronous ภายใน API โดย Momoi จะส่งเพียงข้อมูลอ้างอิงที่จำเป็นเข้าสู่คิว จากนั้น Yuzu จึงอ่านข้อมูลเพิ่มเติมจากฐานข้อมูลก่อนดำเนินการกับ Proxmox VE ทำให้การ retry และการจัดการงานระยะยาวทำได้เหมาะสมขึ้น อีกทั้งยังสามารถกำหนด concurrency ต่อคิวและใช้ distributed locks เพื่อควบคุมงานที่ชนทรัพยากรร่วมกันได้
- 5.3.4 ออกแบบฐานข้อมูลด้วย Prisma ORM ให้รองรับทั้งข้อมูลผู้ใช้ โครงสร้างวิชาการ คำร้องขอ เครื่องเสมือน reverse proxy และพื้นที่จัดเก็บไฟล์ พร้อมกำหนดบทบาทของผู้ใช้อย่างชัดเจน และเชื่อมโยงการยืนยันตัวตนกับบริการ auth ที่ใช้งานร่วมกับแพลตฟอร์ม
- 5.3.5 ใช้ OpenTelemetry และติดตั้งชุด observability เช่น Prometheus, Grafana, Loki และ Jaeger เพื่อเก็บ metrics, logs และ traces ของแต่ละบริการ ช่วยให้สามารถวิเคราะห์ปัญหาเชิงระบบและติดตามประสิทธิภาพได้มีประสิทธิภาพมากขึ้น
- 5.3.6 จัดทำและปรับปรุงเอกสารให้สอดคล้องกับ deployment topology จริง โดยแยกบทบาทของ Midori, Momoi, Yuzu และ Yuuka ให้ชัดเจน รวมถึงอธิบายเส้นทาง ingress, queue flow และขอบเขตระหว่าง authentication กับ authorization เพื่อลดความคลาดเคลื่อนในการสื่อสารและบำรุงรักษาระบบ

5.4 ข้อเสนอแนะ

- 5.4.1 พัฒนาระบบ auto-scaling, load balancing และการแยกทรัพยากรตามลักษณะงาน เพื่อรองรับจำนวนผู้ใช้งานที่เพิ่มขึ้น โดยเฉพาะในช่วงที่มีการสร้างเครื่องเสมือนพร้อมกันจำนวนมากจากกิจกรรมการเรียนการสอน
- 5.4.2 เพิ่มความสามารถด้าน VM template lifecycle เช่น การจัดการ template ตามรายวิชา การทำ snapshot, backup และการกู้คืนสถานะ เพื่อให้รองรับการใช้งานจริงในรายวิชาที่มีความต้องการแตกต่างกันมากขึ้น
- 5.4.3 ปรับปรุงกระบวนการ queue worker และการจัดการงานบนสโตนจอร์หรือคลัสเตอร์ให้รองรับการทำงานพร้อมกันได้ดีขึ้น โดยเฉพาะกรณี full clone หรือการดำเนินงานที่ใช้ทรัพยากรสูง ซึ่งจากผลการทดสอบพบว่ายังมีข้อจำกัดด้านเวลาและอัตราความสำเร็จเมื่อเทียบกับ linked clone แม้ในปัจจุบันจะเริ่มมีการควบคุมด้วย concurrency tuning และ distributed locks แล้วก็ตาม
- 5.4.4 เพิ่มระบบ CI/CD และขยายชุดการทดสอบอัตโนมัติให้ครอบคลุมมากขึ้น ทั้ง unit test, integration test, end-to-end test และ performance regression test เพื่อให้การพัฒนาและปรับปรุงระบบในอนาคตมีความมั่นคงมากขึ้น
- 5.4.5 ต่อยอดการติดตั้งใช้งานบนแพลตฟอร์ม orchestration ระดับ production เช่น Kubernetes อย่างเต็มรูปแบบ พร้อมกำหนดนโยบายด้านความปลอดภัย การจัดการ secret การสำรองข้อมูล และ disaster recovery เพื่อเพิ่มความพร้อมสำหรับการใช้งานในระดับองค์กร
- 5.4.6 พัฒนาระบบ notification และเพิ่ม tracing coverage ฝั่ง worker ให้สมบูรณ์ยิ่งขึ้น เพื่อให้สามารถติดตามเหตุการณ์สำคัญ เช่น การอนุมัติคำร้อง การเริ่ม provision และความล้มเหลวของงานได้ครบถ้วนตลอดเส้นทางการทำงาน

บรรณานุกรม

- Anderson, T. (2008). The theory and practice of online learning (2nd).
- Chya, J., & Jiang, X. (2024). Building a cloud infrastructure for virtual machine scheduling in datacenters [Accessed 25 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 25, 2025, from <https://ieeexplore.ieee.org/document/10427797>
- Clark, R. C., & Kwinn, A. (2016). The new virtual classroom: Evidence-based guidelines for synchronous e-learning (2nd).
- Garcia, C., & Rodriguez, M. (2023). Performance evaluation of hypervisor technologies in educational environments: Vmware vsphere vs. proxmox ve vs. kvm. *Journal of Network and Computer Applications*, 203, 103–121. <https://doi.org/10.1016/j.jnca.2022.103401>
- Kaiser, S., Sadunhaq, M., Tosun, A. A., & Korkmaz, T. (2022). Container technologies for arm architecture: A comprehensive survey of the state-of-the-art [Accessed 23 June 2025.]. *IEEE Communications Surveys and Tutorials*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/9852232>
- Karen Scarfone, P. H., Murugiah Souppaya. (2011). *Guide to security for full virtualization technologies*. Retrieved June 23, 2025, from <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-125.pdf>
- Lee, D., & Chen, W. (2021). Container technology in software development education: A comparative study of docker and kubernetes. *IEEE Transactions on Education*, 64(2), 187–194. <https://doi.org/10.1109/TE.2020.3015678>
- Nayak, N., Grewe, D., & Schildt, S. (2023). Automotive container orchestration: Requirements, challenges and open directions [Accessed 23 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/10136278>
- Suamsiri, R., & Janmanee, K. (2024). Progress tracking system for special project documents [Accessed 23 June 2025.]. Retrieved June 23, 2025, from <http://202.44.47.109/project2/2567-1/671015-1.pdf>

บรรณานุกรม (ต่อ)

- T., D. (2015). *Cloud computing technology the architecture overview*. Retrieved June 23, 2025, from https://www.dga.or.th/wp-content/uploads/2015/05/file_c08274b0a8d4c29561b7e7314afaa102.pdf
- Tao, Y., & Chen, G. (2016). An extensible universal reverse proxy architecture [Accessed 23 June 2025.]. *Proceedings of IEEE Conference*. Retrieved June 23, 2025, from <https://ieeexplore.ieee.org/document/7945939>

ภาคผนวก

ภาคผนวก ก.

การเตรียม Workspace สำหรับโครงงานแบบหลายรีโพสิทอรี

ก.1 ภาพรวมของโครงสร้างแบบหลายรีโพซิทอรี

โครงการ Cloud-Based Platform ในปัจจุบันไม่ได้จัดเก็บซอร์สโค้ดไว้ในรีโพซิทอรีเดียว แต่แยกออกเป็นหลายรีโพซิทอรีตามขอบเขตความรับผิดชอบของบริการ เพื่อให้พัฒนา ทดสอบ และติดตั้งใช้งานได้ยืดหยุ่นมากขึ้น รีโพซิทอรีที่เกี่ยวข้องกับระบบในฉบับปัจจุบันประกอบด้วย Midori, Momoi, Yuzu, Satsuki และ Yuuka

- 1) **Midori** เป็นส่วนติดต่อผู้ใช้ พัฒนาด้วย Next.js และ React
- 2) **Momoi** เป็นบริการ API หลัก พัฒนาด้วย Bun, Elysia.js และ Prisma
- 3) **Yuzu** เป็น worker service สำหรับงาน VM lifecycle และ queue processing
- 4) **Satsuki** เป็นบริการ automation สำหรับ reverse proxy และ SSH access บนเครื่อง Nginx ภายนอก
- 5) **Yuuka** เป็นรีโพซิทอรีสำหรับ manifests, Docker Compose, observability และเอกสารที่เกี่ยวข้องกับ deployment

ก.2 ข้อกำหนดของระบบ

ก่อนเริ่มต้นใช้งาน ควรติดตั้งซอฟต์แวร์พื้นฐานดังนี้

- 1) Bun เวอร์ชัน 1.3 หรือสูงกว่า
- 2) Docker และ Docker Compose Plugin
- 3) Git สำหรับโคลนแต่ละรีโพซิทอรี
- 4) PostgreSQL และ Redis ในกรณีที่ไม่งานผ่าน Docker Compose ของ Yuuka
- 5) Proxmox VE สำหรับการทดสอบงานที่เกี่ยวข้องกับการจัดการ VM จริง

ก.3 การโคลนรีโพซิทอรีทั้งหมด

ผู้พัฒนาควรสร้างโฟลเดอร์หลักหนึ่งตำแหน่ง แล้วโคลนแต่ละรีโพซิทอรีลงในโฟลเดอร์ระดับเดียวกัน ดังตัวอย่างต่อไปนี้

```
mkdir cloud-based-platform
cd cloud-based-platform
git clone https://github.com/akikungz/midori.git
git clone https://github.com/akikungz/momoi.git
git clone https://github.com/akikungz/yuzu.git
git clone https://github.com/akikungz/satsuki.git
git clone https://github.com/akikungz/yuuka.git
```

ภาพที่ ก-1 ตัวอย่างคำสั่งสำหรับโคลนรีโพซิทอรีของระบบ

เมื่อโคลนครบแล้ว workspace จะประกอบด้วยหลายโฟลเดอร์ระดับเดียวกัน เช่น **midori**, **momoi**, **yuzu**, **satsuki** และ **yuuka**

ก.4 การติดตั้ง dependencies ของแต่ละรีโพซิทอรี

หลังจากโคลนซอร์สโค้ดแล้ว ต้องติดตั้ง dependencies แยกตามแต่ละรีโพซิทอรี เช่น

```
cd midori && bun install
cd ../momoi && bun install
cd ../yuzu && bun install
cd ../satsuki && bun install
```

ภาพที่ ก-2 การติดตั้ง dependencies ของแต่ละบริการ

ก.5 การเตรียมไฟล์ Environment

ค่ากำหนดของระบบถูกแยกเก็บตามรีโพซิทอรี และในรีโพซิทอรี Yuuka ยังมีไฟล์ตัวอย่างในโฟลเดอร์ **docs/env** เพื่อใช้ประกอบเอกสารและ onboarding ผู้พัฒนาควรใช้ไฟล์ตัวอย่างจากรีโพซิทอรีจริงของแต่ละบริการเป็นหลัก แล้วตรวจสอบให้ค่าต่อไปนี้สอดคล้องกัน

- 1) PostgreSQL connection string
- 2) Redis URL
- 3) S3-compatible storage endpoint และ credentials
- 4) API endpoint ที่ Midori เรียกใช้งาน
- 5) Proxmox VE credentials ของ Yuzu
- 6) path และ reload command ของ Satsuki บนเครื่อง Nginx ภายนอก

ก.6 ข้อสังเกตสำหรับการพัฒนาใน workspace เดียวกัน

แม้ระบบจะเป็นหลายรีโพซิทอรี แต่สามารถเปิดใช้งานร่วมกันใน workspace เดียวของโปรแกรมพัฒนาได้ ซึ่งช่วยให้ค้นหาโค้ด อ้างอิงไฟล์ และติดตามการไหลของข้อมูลระหว่างบริการได้สะดวกขึ้น โดยเฉพาะเมื่อต้องตรวจสอบ ความสอดคล้องของ schema, API contract, queue names, environment variables และ deployment manifests

ภาคผนวก ข.

การเริ่มต้นระบบแบบ Development ในสภาพแวดล้อมหลายรีโพสิทอรี

ข.1 การเริ่มต้นบริการประกอบจากรีโพซิทอรี Yuuka

รีโพซิทอรี Yuuka ทำหน้าที่เก็บไฟล์สำหรับโครงสร้างพื้นฐานของระบบ เช่น Docker Compose, Kubernetes manifests, ตัวอย่าง environment และเครื่องมือ observability ดังนั้นในสภาพแวดล้อมพัฒนา ผู้พัฒนาควรเริ่มต้นบริการประกอบจากรีโพซิทอรีนี้ก่อน เพื่อให้ PostgreSQL, Redis, RustFS และเครื่องมือสำหรับติดตามระบบพร้อมใช้งาน

```
cd yuuka
docker compose -f docker/docker-compose.yaml up -d
```

ภาพที่ ข-1 การเริ่มต้นบริการประกอบจากรีโพซิทอรี Yuuka

ข.2 การเริ่มต้นบริการหลักของระบบ

หลังจากบริการประกอบพร้อมใช้งานแล้ว ให้เริ่มต้นบริการหลักของระบบใน terminal แยกกัน เพื่อให้สามารถตรวจสอบ log และควบคุมการทำงานของแต่ละบริการได้อย่างชัดเจน

- 1) เริ่ม Momoi จากรีโพซิทอรี **momoi**
- 2) เริ่ม Yuzu จากรีโพซิทอรี **yuzu**
- 3) เริ่ม Midori จากรีโพซิทอรี **midori**

```
cd momoi
bun run dev
```

ภาพที่ ข-2 การเริ่มต้น Momoi ในโหมดพัฒนา

```
cd yuzu
bun run dev
```

ภาพที่ ข-3 การเริ่มต้น Yuzu ในโหมดพัฒนา

สำหรับ Satsuki โดยทั่วไปจะไม่จำเป็นต้องรันในเครื่องพัฒนาเดียวกันทุกครั้ง เพราะบริการนี้ถูกออกแบบให้ทำงานบนเครื่อง Nginx ภายนอกที่ต้องมีสิทธิ์เขียนไฟล์ configuration และ **authorized_keys** จริง หากต้องการทดสอบส่วนนี้ ผู้พัฒนาควรใช้เครื่องทดสอบแยกหรือสภาพแวดล้อมที่เตรียมสิทธิ์ไว้แล้ว

ข.3 ลำดับการเริ่มต้นที่เหมาะสม

ลำดับการเริ่มต้นระบบที่เหมาะสมสำหรับการพัฒนา คือ เริ่มจาก Yuuka เพื่อให้บริการประกอบพร้อมก่อน จากนั้นเริ่ม Momoi และ Yuzu แล้วจึงเริ่ม Midori เป็นลำดับสุดท้าย เนื่องจาก Midori

```
cd midori
bun run dev
```

ภาพที่ ข-4 การเริ่มต้น Midori ในโหมดพัฒนา

ต้องอาศัย API จาก Momoi และ Yuzu ต้องอาศัย Redis รวมถึงฐานข้อมูลในการประมวลผลงานจากคิว

ข.4 จุดเชื่อมต่อสำคัญในสภาพแวดล้อมพัฒนา

เมื่อติดตั้งและเริ่มบริการครบแล้ว ผู้พัฒนาสามารถเข้าถึงบริการสำคัญได้จากพอร์ตมาตรฐานดังนี้

- 1) Midori ที่ <http://localhost:3000>
- 2) Momoi ที่ <http://localhost:3000>
- 3) Grafana ที่ <http://localhost:3002>
- 4) RustFS Console ที่ <http://localhost:9001>
- 5) Prometheus ที่ <http://localhost:9090>
- 6) Jaeger ที่ <http://localhost:16686>

ข.5 การอัปเดตชนิดข้อมูลระหว่าง frontend และ backend

เมื่อมีการเปลี่ยนแปลง OpenAPI ของ Momoi ควรเริ่ม Momoi ให้พร้อมก่อน แล้วจึงสั่งสร้างชนิดข้อมูลฝั่ง Midori ใหม่ เพื่อให้ type definitions ของ frontend สอดคล้องกับ API ล่าสุด

```
cd midori
bun run api:gen
```

ภาพที่ ข-5 การสร้างชนิดข้อมูล API ของ Midori ใหม่จาก OpenAPI

ข.6 การแก้ไขปัญหาเบื้องต้นในสภาพแวดล้อมพัฒนา

แนวทางการตรวจสอบปัญหาที่พบบ่อยในระหว่างการพัฒนา มีดังนี้

- 1) Port ถูกใช้งานแล้ว
 - 1.1) ตรวจสอบ process ที่ใช้งานพอร์ตอยู่
 - 1.2) หยุด process ดังกล่าวหรือเปลี่ยนค่าพอร์ตในไฟล์ environment
- 2) Docker services ไม่สามารถเริ่มได้
 - 2.1) ตรวจสอบสถานะด้วยคำสั่ง `docker compose -f docker/docker-compose.yaml ps`

- 2.2) ดู log ด้วยคำสั่ง `docker compose -f docker/docker-compose.yaml logs [service_name]`
- 2.3) รีสตาร์ทบริการด้วยคำสั่ง `docker compose -f docker/docker-compose.yaml restart [service_name]`
- 3) Frontend ไม่สามารถเชื่อมต่อ API ได้
 - 3.1) ตรวจสอบว่า Momoi ทำงานอยู่ที่พอร์ตที่กำหนดจริง
 - 3.2) ตรวจสอบค่า API URL ใน Midori
 - 3.3) ตรวจสอบ CORS และ cookie settings ของ Momoi
- 4) Yuzu ไม่ประมวลผลงาน
 - 4.1) ตรวจสอบ Redis, queue state และ log ของ Yuzu
 - 4.2) ตรวจสอบค่า concurrency ของแต่ละคิว
 - 4.3) ตรวจสอบ resource lock ที่เกี่ยวข้องกับ `instanceId` หรือ VMID

ภาคผนวก ค.

หน้าที่ของแต่ละรีโพซิทอรีและแนวทางการตรวจสอบปัญหาเบื้องต้น

ค.1 หน้าที่ของแต่ละรีโพซิทอรีในระบบ

การแยกระบบออกเป็นหลายรีโพซิทอรีช่วยลด coupling และทำให้ทีมพัฒนาสามารถดูแลแต่ละส่วนของแพลตฟอร์มได้อย่างเป็นอิสระมากขึ้น อย่างไรก็ตาม ผู้พัฒนาต้องเข้าใจความรับผิดชอบของแต่ละรีโพซิทอรีอย่างชัดเจนเพื่อให้ตรวจสอบปัญหาได้ถูกต้อง

- 1) **Midori** รับผิดชอบส่วนติดต่อผู้ใช้ การเรียก API ของระบบ และประสบการณ์ผู้ใช้งานเว็บ
- 2) **Momoi** รับผิดชอบตรรกะทางธุรกิจ การเชื่อมต่อฐานข้อมูล การเปิดเผย OpenAPI การยืนยันตัวตนผ่าน ‘/api/auth’ และการส่งงานเข้าสู่คิว
- 3) **Yuzu** รับผิดชอบการประมวลผลงานเบื้องหลัง เช่น provision, deprovision และการเปลี่ยนสถานะเครื่องเสมือน
- 4) **Satsuki** รับผิดชอบการสร้างไฟล์ reverse proxy configuration และ ‘authorized_keys’ จากข้อมูลในฐานข้อมูล โดยติดตั้งบนเครื่อง Nginx ภายนอกที่เป็น reverse proxy หน้าด่านของระบบ
- 5) **Yuuka** รับผิดชอบ manifest, Docker Compose สำหรับบริการประกอบ, ตัวอย่าง environment และการตั้งค่า observability ของระบบ

ค.2 ส่วนประกอบสนับสนุนที่เกี่ยวข้องกับหลายรีโพซิทอรี

แม้ว่าบริการหลักเชิงธุรกิจจะมีเพียงสามตัว แต่การทำงานจริงยังขึ้นกับบริการสนับสนุนหลายตัวที่เชื่อมโยงกันระหว่างรีโพซิทอรี เช่น Satsuki สำหรับ reverse proxy/SSH automation บนเครื่อง Nginx ภายนอก, PostgreSQL สำหรับข้อมูลเชิงธุรกรรม Redis และ BullMQ สำหรับคิวงาน RustFS สำหรับพื้นที่จัดเก็บไฟล์ และ Jaeger, Prometheus, Grafana, Loki สำหรับการติดตามสถานะของระบบ

ค.3 แนวทางการตรวจสอบปัญหาเบื้องต้น

เมื่อเกิดปัญหาระหว่างการพัฒนา ควรเริ่มจากการแยกก่อนว่าปัญหาเกิดในรีโพซิทอรีใด หรือเกิดจากการเชื่อมต่อข้ามรีโพซิทอรี ดังแนวทางต่อไปนี้

- 1) **เปิดหน้าเว็บไม่ได้หรือข้อมูลไม่แสดง** ให้ตรวจสอบ Midori ก่อนว่าเริ่มต้นได้สมบูรณ์ และค่า endpoint ของ Momoi ถูกต้อง
- 2) **เข้าสู่ระบบไม่ได้หรือ session ผิดปกติ** ให้ตรวจสอบ Momoi, การตั้งค่า Google OAuth, cookie domain/path และการเชื่อมต่อฐานข้อมูลของข้อมูลผู้ใช้
- 3) **API ตอบกลับผิดพลาด** ให้ตรวจสอบ log ของ Momoi รวมถึงสถานะของ PostgreSQL, Redis และค่าที่อยู่ในไฟล์ environment

- 4) งาน **provision** หรือเปลี่ยนสถานะ VM ไม่ทำงาน ให้ตรวจสอบ Yuzu, Redis queue, การเชื่อมต่อไปยัง Proxmox VE และสถานะของ resource locks ว่ามีงานอื่นถือ lock ของ ‘instanceld’ หรือ ‘VMID’ เดียวกันอยู่หรือไม่
- 5) **reverse proxy** หรือการเข้าใช้งานผ่าน SSH มีปัญหา ให้ตรวจสอบ Satsuki, trigger/notification จาก PostgreSQL, ไฟล์ configuration ที่ถูกสร้างบนเครื่อง Nginx ภายนอก และสถานะการ reload ของบริการ proxy
- 6) **อัปโหลดไฟล์หรือเรียกไฟล์ไม่ได้** ให้ตรวจสอบการตั้งค่า RustFS/S3-compatible storage ทั้งฝั่ง Momoi และบริการประกอบใน Yuuka
- 7) **ข้อมูล observability ไม่ขึ้น dashboard** ให้ตรวจสอบการ export metrics และ traces ของแต่ละบริการ รวมถึงสถานะของ Prometheus, Loki และ Jaeger

ค.4 ตัวอย่างรายการตรวจสอบอย่างเป็นลำดับ

ในทางปฏิบัติ การไล่ตรวจสอบจากชั้นนอกเข้าสู่ชั้นในจะช่วยลดเวลาในการหาสาเหตุของปัญหาได้ โดยอาจเริ่มจาก browser, frontend, API, queue worker, database และ infrastructure ตามลำดับ ดังนี้

- 1) ตรวจสอบว่า Midori ทำงานที่ localhost:3000
- 2) ตรวจสอบว่า endpoint ของ Momoi ตรงกับค่าที่ตั้งในไฟล์ environment ของ Midori
- 3) ตรวจสอบบริการประกอบจากคำสั่ง **docker compose -f docker/docker-compose.yaml ps** ในรีโพสิทอรี Yuuka
- 4) ตรวจสอบ Redis และ PostgreSQL
- 5) ตรวจสอบ log ของ Yuzu, ค่า concurrency ของแต่ละคิว และสถานะ queue/lock บน Redis
- 6) ตรวจสอบ log ของ Satsuki และผลลัพธ์ของไฟล์ reverse proxy/authorized_keys บนเครื่อง Nginx ภายนอกหรือ bastion ปลายทาง
- 7) ตรวจสอบ Jaeger, Prometheus และ Grafana

ค.5 ข้อควรระวังเรื่องความสอดคล้องระหว่างรีโพสิทอรี

เนื่องจากแต่ละรีโพสิทอรีพัฒนาแยกจากกัน จึงควรระมัดระวังความสอดคล้องของ schema ฐานข้อมูล, ตัวแปร environment, เส้นทาง API, queue name และข้อมูลอ้างอิงที่ใช้ร่วมกัน เพราะหากค่าหนึ่งค่าผิดไปเพียงจุดเดียว อาจทำให้เกิดปัญหาต่อเนื่องข้ามหลายบริการได้

ค.6 เอกสารอ้างอิงที่ควรใช้ประกอบ

ในการทำงานจริง ผู้พัฒนาควรอ่าน README และเอกสารในแต่ละรีโพสิทอรีร่วมกัน โดยเฉพาะเอกสารใน Yuuka ที่สรุปตัวอย่าง environment และการติดตั้งเครื่องมือประกอบ ส่วน Midori, Momoi, Yuzu และ Satsuki ควรใช้ร่วมกับเอกสาร API, queue, authentication flow, reverse proxy และ observability เพื่อให้เห็นภาพรวมของระบบครบถ้วน

1) Port ถูกใช้งานแล้ว

- 1.1) ตรวจสอบ Port ที่ใช้งานด้วยคำสั่ง: `lsof -i :3000` (สำหรับ macOS/Linux)
- 1.2) หยุดกระบวนการที่ใช้ Port หรือเปลี่ยน Port ในไฟล์ .env

2) Docker Services ไม่สามารถเริ่มได้

- 2.1) ตรวจสอบสถานะด้วยคำสั่ง: `docker compose -f docker/docker-compose.yaml ps`
- 2.2) ดู Log ด้วยคำสั่ง: `docker compose -f docker/docker-compose.yaml logs [service_name]`
- 2.3) รีสตาร์ท Services ด้วยคำสั่ง: `docker compose -f docker/docker-compose.yaml restart [service_name]`

3) Database Connection Error

- 3.1) ตรวจสอบว่า PostgreSQL Container ทำงานอยู่
- 3.2) ตรวจสอบการตั้งค่าใน Environment Variables
- 3.3) รันคำสั่ง Database Migration (หากจำเป็น)

4) Frontend ไม่สามารถเชื่อมต่อ API ได้

- 4.1) ตรวจสอบว่า Momoi API Service ทำงานอยู่ตาม endpoint ที่กำหนดในไฟล์ Environment
- 4.2) ตรวจสอบการตั้งค่า API URL และ AUTH API URL ในไฟล์ Environment ให้ชี้มายังบริการ Momoi อย่างสอดคล้องกัน
- 4.3) ตรวจสอบ CORS Settings และ cookie configuration ของบริการปลายทาง

การเข้าถึงข้อมูลเพิ่มเติม

- 1) อ่าน Documentation เพิ่มเติมใน README.md ของแต่ละ Component
- 2) ตรวจสอบ Log Files สำหรับข้อมูลการ Debug
- 3) ใช้ Jaeger UI สำหรับติดตามการทำงานของ API
- 4) ใช้ Grafana Dashboard สำหรับติดตามประสิทธิภาพระบบ